

MARS：具執行時期任務切換之超低位元串流推論 可重組加速器架構

MARS: A Reconfigurable Accelerator Architecture with Runtime Task Switching for Ultra-Low-Bit Streaming Inference

作者：黃柏鈞（Po-Chun Huang）

指導教授：熊博安（Pao-Ann Hsiung）

黃駿賢（Chun-Hsien Huang）

2026 年 7 月

摘要

邊緣 FPGA 平台，以及更廣義之固定式邊緣加速器，皆須在嚴格資源限制下支援多個 AI 任務。於 FPGA 上，所調查之 PR 類 CNN 方法需重組態控制器與每任務部分 bitstream，切換延遲為毫秒等級；固定特徵萃取器設計則無法於執行時期調整特徵萃取器本身。將超低位元量化與參數高效率遷移學習（PEFT）結合於串流資料流加速器原可解此，但於所評估組態中，單分支 adapter 之效果於 1-bit 之退化顯著大於 2 bit 以上。

本論文提出 **MARS**（Multi-Branch Adapter Reconfigurable Streaming-accelerator），一種超低位元 FPGA 加速器，將自研 Adapter IP 整合至串流資料流主幹。MARS 以多分支 Adapter 擴充有效 adapter 容量、緩解二值化造成之遷移準確率損失，以**殘差校正（RC）**——即 adapter 既有之 down-projection 偏置；其硬體映射不增加額外運算單元或管線級，儲存與組態位元組計入 adapter 成本——緩解 1-bit 所造成的顯著準確率退化，並以執行時期組態介面於單一 bitstream 上重新編程所有任務相依參數，免任何 FPGA fabric 重組態控制器即可切換任務。據我們所知，於所調查之 FPGA/PEFT 加速器研究中，本研究為首個同時整合超低位元共用主幹、特徵萃取器層級之 adapter 任務適應、以及板上驗證之單一 bitstream 執行時期任務切換之加速器。功能正確性以三層級驗證：模組層級 RTL 模擬、與軟體參考一致之頂層資料流模擬，以及板上端到端驗證。

於凍結之 1-bit VGG-like 主幹上，多分支 Adapter 擴展相較單分支基線，於二十組 backbone-target 配對之單一 seed 軟體實驗中平均提升 1-bit 遷移準確率 3.4 個百分點（最高 11.3，以遷移至 SVHN 最顯著）；於 CIFAR-10→SVHN 上，RC 於單分支部署幾何貢獻 +23.0 個百分點、於 $M=1-4$ 間為 +20 至 +23 個百分點，於其餘目標增益較小（+0.6 至 +7.4）。硬體方面，*compactness* build 於單一 PYNQ-Z2 bitstream 上執行時期切換五組任務專屬模型（各對應一資料集），各資料集板上準確率與其軟體參考差距均在 0.69 個百分點內，單次任務切換之組態重寫為 1.86 ms（約 25 KB；切換後推論正確性另行驗證）、無 fabric 重組態； $M \geq 2$ 之多分支 Adapter 組態因超出 XC7Z020 容量，僅於 RTL 模擬與合成後層級驗證。另於協定對齊之部署層級 1W1A SVHN 比較（資料集、精度、輸入尺寸與 batch 對齊，惟 adapter 幾何不同：FPGA 執行部署版 MARS、GPU/Jetson 執行寬版 Conv-Adapter）中，*throughput* build 持續輸出 1,866 img/s、2.214 W（842.7 img/s/W；FPGA 功耗為 Vivado 估計值、GPU 為實測值）；逐平台比較詳見第五章。

關鍵詞：參數高效率遷移學習（PEFT）、超低位元量化、FPGA 加速器、串流資料流、執行時期任務切換、殘差校正。

Contents

術語對照表	5
1 緒論	11
1.1 研究背景	11
1.2 動機與問題定義	12
1.2.1 資源受限邊緣 FPGA 上之多應用部署：現行方法與其限制	12
1.2.2 部署密度與 1-bit 準確率落差	13
1.3 方法概述	14
1.3.1 超低位元量化下的準確率強化	14
1.3.2 硬體設計與整合	14
1.3.3 執行時期任務切換	15
1.4 研究目標	15
1.5 貢獻	16
1.6 論文組織	18
2 相關研究	19
2.1 超低位元 FPGA CNN 加速器與資料流架構	19
2.2 參數高效率遷移學習與 Adapter 方法	20
2.3 可重組態、自適應與共享 FPGA 架構	22
3 方法論	25
3.1 問題形式化與設計動機	25
3.2 二值化 Adapter 瓶頸架構	29
3.3 殘差校正 (RC) 與 1-bit 之經驗發現	33
3.3.1 定義與硬體實作	33
3.3.2 RC 為何於 1-bit 下重要	34

3.3.3	經驗發現：RC 於 1-bit 下之 disproportionate role	34
3.4	多分支 Adapter 擴展	35
3.4.1	架構	35
3.4.2	理論運算成本	36
3.4.3	透過分支擴展恢復準確率	37
4	MARS NPU 架構設計	38
4.1	系統架構概觀	38
4.2	MVAU + Adapter Super Wrapper	41
4.3	Adapter IP 內部架構	43
4.3.1	Down-Projection 管線	43
4.3.2	Up-Projection 路徑	44
4.3.3	記憶體組織	45
4.4	Stream Adder + Threshold 模組	45
4.4.1	貢獻 LUT：離線預計算與執行時期可寫入之儲存	45
4.4.2	二級管線	46
4.5	執行時期配置中樞 (cfg_hub)	47
4.5.1	位址映射	49
4.5.2	執行時期切換流程	49
4.5.3	RTL 修改	50
4.6	MVAU + Adapter 整合系統之管線化	51
4.7	驗證方法論	51
5	實驗與評估	52
5.1	端到端設計流程	52
5.2	實驗設定	53
5.2.1	資料集與任務	53
5.2.2	平台與工具	54
5.2.3	量測指標	54
5.3	軟體端遷移學習結果	55
5.3.1	1-bit 遷移之準確率落差	55
5.3.2	Down-convolution 偏置作為 1-bit 專屬之校正項 (RC)	55
5.3.3	多分支擴展與軟體參考值	56

5.3.4	backbone (來源) 資料集之影響	57
5.3.5	跨資料集泛化	59
5.3.6	Multi-seed 變異性	60
5.3.7	FPGA-deployed 配置與 SW-HW 差距	61
5.4	FPGA 資源使用與功耗	68
5.4.1	跨配置資源比較	68
5.4.2	功耗	70
5.4.3	逐層 PE 配置：throughput 對 compactness	70
5.4.4	多分支擴展成本與準確率—資源權衡	72
5.4.5	Golden 模型驗證	77
5.5	跨平台效能比較	77
5.5.1	分析	78
5.5.2	跨平台比較之公平性與範圍	79
5.6	執行時期模型切換	80
5.6.1	cfg_hub 架構	81
5.6.2	實作層觀察：Vivado OOC 合成快取	82
5.6.3	compactness 變體：板上 N 任務切換與可擴展性	83
5.7	任務數擴展分析	85
5.8	證據層級總結	88
5.9	限制	89
6	結論與未來工作	92

術語對照表

1W1A	1-bit 權重、1-bit 啓動值（二值化區段）。
Ablation（消融研究）	切換某一元件以孤立其貢獻之受控實驗。
Adapter	插入凍結主幹之小型任務特定模組; 本論文採 Conv-Adapter 瓶頸結構（down-conv $\rightarrow$ 二值啓動 $\rightarrow$ up-conv）。
Backbone（主幹）	凍結之任務無關特徵萃取器（本論文為以 CIFAR-10 預訓練之 1W1A VGG-like CNN；即 FINN 參考之 CNV 網路、6 卷積 + 3 FC）。
Baseline（基線）	單分支 $M=1$ Conv-Adapter, 為先前工作之參考起點; 見第 5.3 節之「基線政策」。
寬版幾何（wide geometry）	較大之純軟體 adapter 幾何（ 3×3 down-conv、per-channel α 、mid = $C_{\text{out}}/4$ ）; 於 CIFAR-10 主幹上準確率最高, 優劣排序取決於來源（第 5.3.4 節）。
部署幾何（deployed geometry）	部署至 FPGA 之 adapter 幾何（ 1×1 down-conv、scalar α 、mid = $C_{\text{in}}/4$ ）; 即帶入 bitstream 之配置。
Bitstream（位元流）	FPGA 編譯後之組態二進位檔（.bit）。
Cfg_hub	本論文自訂之 64 KB 配置中樞, 於 runtime 重新編程所有任務相依參數。
Down-/up-projection	包覆於 Adapter 二值啓動兩側之兩個 1×1 卷積。
DFX	Dynamic Function eXchange, AMD 之部分重組態介面; 於比較基線中作為代表性 PR 機制。
NPU	神經處理單元; 本論文指部署於 FPGA 之 MARS 加速器。
QAT	量化感知訓練; 訓練時模擬量化, 使網路適應低位元權重與啓動值。
memstream	FINN 之晶片內權重串流模組, 將參數送入 MVAU。
MMIO	記憶體映射 I/O; 主機 CPU 經系統位址映射存取 FPGA 暫存器與記憶體。

AXI-Lite	輕量級 AXI 介面, 用於低頻寬之暫存器／組態存取（此處用於組態中樞）。
OOC	Out-Of-Context 合成; Vivado 為每個 IP 保留之獨立合成快取, 與頂層 run 分離。
WNS	Worst Negative Slack; 後佈局時序餘裕（正值代表時序達成）。
FINN	AMD 開源之串流資料流神經網路編譯器, 用於生成主幹 IP。
MVAU	Matrix-Vector-Activation Unit, FINN 卷積核心, Adapter IP 整合於其中。
<i>M</i>	每層平行 Adapter 分支數（ $M=1$ 基線, $M=2-4$ 本論文擴展）。
Throughput compactness build	/ Throughput build：FINN 預設之異質 PE 折疊（最高 32）, 吞吐量／能效配置。Compactness build：均勻 $PE=1$, 為 runtime N 任務切換刻意收小資源。兩者皆為 pipelined 串流資料流設計。
RC（殘差校正）	Adapter 之 down convolution Int8 量化偏置, 於 runtime 吸收至累加器 reset 值; 與一般 ResNet 之「殘差連接」skip-link 屬不同概念。
Q8	8-bit 定點運算, 僅用於 adapter 貢獻與閾值路徑; 主幹與 Adapter 資料路徑仍為 1W1A 二值。
Super Wrapper	將 Adapter IP 整合至 FINN MVAU 之 RTL 包裝器, 採解耦閾值處理與 FIFO 同步。
PR / PCAP / ZyCAP	經 Processor Configuration Access Port（PCAP）或其高吞吐變體（ZyCAP）之部分重組態; 於比較表中作為切換延遲基線。

List of Figures

3.1	MARS 軟體架構。	29
3.2	Conv-Adapter [22] 瓶頸二值化至 1W1A。	29
3.3	多分支 adapter: M 條平行二值分支於閾值化前相加。	36
4.1	MARS 加速器之系統頂層架構。	39
4.2	FINN 生成、未經 Adapter 修改之串流推論管線。	39
4.3	MARS 串流資料流管線。	40
4.4	未修改之 FINN MVAU 內部架構。	42
4.5	MVAU + Adapter Super Wrapper。	42
4.6	Adapter IP 內部架構 (單一支; 匯流排標註為位元寬度)。	43
4.7	RC 之硬體實現: 累加器初始值由 ram_rc 載入 (無額外加法器或管線級)。	44
4.8	Function Switch Controller 之系統層整合。	48
4.9	Function Switch Controller 之內部電路。	48
4.10	cfg_hub 64 KB 位址映射。	49
4.11	端到端執行時期任務切換序列。	49
4.12	部署成本對任務/變體數之示意	50
5.1	MARS 端到端軟體到硬體流程。	53
5.2	RC 之機制層級探查	67
5.3	MARS compactness build 於 XC7Z020 之實作後佈局。	69
5.4	兩種 build 之折疊差異: PE = 1(compactness) 對 PE 平行通道 (throughput, 以 PE = 4 為例)。	71
5.5	多分支準確率增益對合成後 LUT 需求	74
5.6	各分支數之總晶片功耗 (合成後估計)。	76

List of Tables

1.1	研究問題、所提方法, 以及回應各問題之證據。	16
2.1	超低位元 FPGA CNN 加速器之代表性文獻整理 (依年份排序, 由新至舊)	20
2.2	PEFT 與 Adapter 類方法之代表性文獻整理 (依年份排序, 由新至舊) . . .	22
2.3	可重組態與任務切換 FPGA 架構之代表性文獻整理 (依年份排序, 由新至舊)	23
2.4	研究缺口分析:MARS 之定位	24
3.1	單 Conv-Adapter [22] 之 bit-width 掃描: 含 (RC) /不含 (No-RC) down-projection 偏置 vs. Full-FT。	26
3.2	單 Conv-Adapter [22](RC) 跨更廣 backbone-target 之 bit-width 掃描。 . . .	27
3.3	各層 Adapter 瓶頸配置	32
3.4	主幹運算成本分解 (CIFAR-10, $32 \times 32 \times 3$ 輸入)	37
4.1	MARS 部署串流資料流分區內各模組之來源。	41
4.2	Adapter RAM 配置 (範例: Super4, PE= 4)	45
5.1	本論文使用之資料集 (皆調整為 $32 \times 32 \times 3$ 輸入)。	54
5.2	CIFAR-10 $\rightarrow$ SVHN(1W1A)1-bit 殘差校正消融。	56
5.3	CIFAR-10 backbone 之多分支擴展 (寬版幾何、軟體、1W1A、RC)。 . . .	57
5.4	多分支擴展對其他來源 backbone 之推廣 (寬版幾何)。	58
5.5	CIFAR-10 $\rightarrow$ SVHN(1W1A) 多 seed 測試準確率。	60
5.6	$M=1 \rightarrow 4$ 增益之五-Seed 配對摘要 (部署幾何, Seeds 2024–2028)。	61
5.7	部署 MARS 相對 Conv-Adapter [22] 所改之五個設計軸。	62
5.8	寬版與部署幾何於各分支數之對照 (全部 20 組 backbone-target 配對); Δ_{M4} = 部署 – 寬版 ($M=4$)。第一部分: CIFAR-10、SVHN、STL10 來源 (接續 次頁)。	64

5.8 (續) 寬版與部署幾何於各分支數之對照。第二部分:FashionMNIST、CINIC10 來源。	65
5.9 部署 MARS 之殘差校正消融 (1-bit)。	66
5.10 MARS 跨資料集遷移準確率 vs. 分支數 M 。	66
5.11 MARS 多分支擴展貢獻 (四個 target)。	67
5.12 四個 MARS build 配置之 FPGA 資源使用。	68
5.13 四個 MARS build 配置之功耗。	70
5.14 throughput 與 compactness build 之逐層 PE 折疊。	71
5.15 多分支 MARS 於 XC7Z020 之合成後資源與功耗擴展。	72
5.16 compactness build 之每分支參數量與 post-synthesis 成本。	73
5.17 $M=1$ build 之合成與實作階段使用率對照。	74
5.18 二十組主幹-目標遷移對之部署幾何多分支準確率。	75
5.19 純主幹: 跨平台效能 (CIFAR-10, batch 1000)。	78
5.20 資料集、精度、輸入尺寸與 batch 對齊、惟 adapter 幾何不同之部署層級比較:MARS (FPGA) 對 Conv-Adapter [22] (GPU/Jetson), SVHN, batch 1000。	78
5.21 2 任務與 N 任務切換對照。	81
5.22 cfg_hub 位址映射 (基址 0x40010000, 64 KB)	81
5.23 compactness build: 板上 N 任務切換準確率。	84
5.24 compactness N -task bitstream: XC7Z020 後佈局資源。	85
5.25 任務切換 FPGA 推論: MARS 與已發表方法之比較。	86
5.26 T 個執行時期任務之 bitstream 儲存需求。	87
5.27 重組態控制器需求 vs. MARS。	87
5.28 引用範圍內 1-bit Adapter 之硬體驗證狀態。	88
5.29 主張—證據總結: 各主張之證據層級、工作負載與限制。	89

List of Algorithms

3.1	二值化 Adapter 前向運算	33
4.1	Stream Adder + Threshold 融合	47

Chapter 1

緒論

1.1 研究背景

為滿足即時反應、資料隱私與離線運作的需求，深度學習模型正逐步由雲端伺服器遷移至邊緣裝置。然而邊緣部署受限於運算能力、記憶體容量與功耗預算，如何在有限硬體資源下有效支援多種 AI 應用已成為核心挑戰 [1–3]。

AI 硬體加速研究沿多條設計路徑演進。其中 FPGA 類加速器具備可重組性、高能效與可客製化之資料路徑，特別適合邊緣推論 [4, 5]。在超低位元設定下（如 1-bit / 2-bit），二值化與低精度網路可將乘法替換為 XNOR 與 popcount 運算以降低算術複雜度，並透過 PE/SIMD 並行維持吞吐量 [6–8]。串流資料流工具鏈將量化網路映射至硬體，其中矩陣-向量-啟動單元（MVAU）為核心運算元件 [9–11]。

選擇將推論工作交由客製化硬體（本論文之 FPGA-instantiated NPU）而非交由通用 GPU 跑同一個模型，並非僅為了「峰值吞吐量更高」這麼簡單。實際邊緣 AI 工作負載中有相當大一部分位於 GPU 無論用什麼軟體堆疊皆可能難以滿足之功耗、延遲或外型尺寸範圍內。代表性實際案例如下：

- **常時開啓之耳戴式裝置與智慧音訊：**耳機與 TWS 裝置之關鍵字偵測與主動降噪推論之常時推論功耗預算極低，遠低於典型行動 GPU 之功耗剖面，因此驅動專屬低功耗 NN 加速器之客製化 [12]。
- **工業機器視覺：**產線高吞吐瑕疵檢測於無風扇外殼之低功耗包絡，行動 GPU 之散熱難以滿足；二值化量化之 FPGA 串流加速器即為此 regime 之主流方案 [13–15]。
- **無人機與自主行動機器人：**每張推論之最差情況延遲有界要求不利於尾延遲變異與 deadline 同量級之 GPU；確定性 FPGA 串流資料流管線提供週期精準之資料路

徑時序（完整系統 deadline 仍含 DMA 與主機軟體）[5,16]。

- **太空與抗輻射部署**：目前出貨之太空合格運算元件多為 FPGA，市面上幾無太空版 GPU 可選 [2]。
- **穿戴式醫療／IoT 裝置**：連續生理訊號推論之多日電池壽命須使用極低功耗加速器，此 regime 亦明確推動專屬之 FPGA/ASIC NN 推論硬體 [1-3]。

同時，模型任務適應的需求快速增加。傳統遷移學習通常進行整模型微調或僅替換分類頭；隨模型規模與任務多樣性攀升，此策略的訓練成本、儲存需求與部署複雜度皆顯著上升 [17,18]。參數高效率遷移學習（PETL/PEFT）凍結主幹並僅訓練少量任務專屬參數，藉此降低額外開銷並維持競爭力 [19-21]。在視覺領域，Adapter 類方法通常插入由 down-projection、非線性、up-projection 組成的瓶頸模組，僅需切換輕量參數即可完成任務適應 [22,23]。

1.2 動機與問題定義

1.2.1 資源受限邊緣 FPGA 上之多應用部署：現行方法與其限制

於單一邊緣 FPGA 部署多個 AI 應用在文獻中已被大量研究，目前主要分三類範式，各自皆於原始論文中揭示其限制，本論文正面回應之。

部分重組態時間多工 ACNNE [24] 透過 PCAP 介面在執行時期置換卷積層硬體區塊，其每一分割之重組態延遲顯著。Rangsikunpum 等 [25] 之 coarse-to-fine 方案改用較快的 ZyCAP 控制器，但其每次切換延遲據作者指出已主導單次推論的執行時間。Boudjadar 等 [26] 與 Kang & Park [27] 亦採動態部分重組態（DPR），因此承繼底層 PR 控制器之延遲，且需於目標裝置上長駐一組重組態控制器與部分 bitstream 倉庫。綜上，現行 PYNQ 等級 FPGA 之 PR 切換尚未有次毫秒之報導。

固定特徵萃取器配可程式後端 FixyNN [28] 將特徵萃取器硬接線而僅保留後端可程式性，明確主張邊緣端無法為每一任務完整複製模型、特徵萃取器應被共享。此框架具影響力，但特徵萃取器於 tape-out 即凍結，無法在執行時期進行任務特定之特徵適應，而這恰為超低位元（1W1A）主幹最容易損失準確率的地方。

晶片內動態模型選擇 NeuroAdaptiveNet [29] 將一組固定、於合成時決定之精簡模型變體之權重與偏置全部存於晶片內; 每次推論以 k-Nearest-Centroid 選擇器 (僅增加 0.23–0.67 μ s 延遲) 挑選變體, 並將其參數多工至共用 PE 陣列——無部分重組態、無 DFX 控制器。其晶片內成本因此隨常駐變體集合成長, 且集合於合成／訓練時即固定; 所報 52.85% LUT 節省為相對於以最大模型為尺寸之靜態設計。其為對小型表格資料之每輸入整模型選擇, 而非特徵萃取器層級之任務適應。

本論文欲解決之缺口 綜觀上述已發表系統, 於所引用之研究中, 我們未見一套能在資源受限之邊緣 FPGA / ASIC 上, 不需任何結構層級 (fabric-level) 重組態控制器、即可於單一 bitstream 上以 $O(1)$ 晶片成本服務任意數量之合格同主幹、同輸入尺寸、同輸出 head 分類任務 (主機端組態儲存隨任務數線性成長), 且同時支援特徵萃取器層級任務適應 (而非僅分類器層級適應) 之設計。於 1W1A 量化下, 固定式或僅分類器式方法難以補回二值化所損失之準確率; 而 PR 類方法之切換延遲則不容許細粒度時間分享。MARS 正面處理此缺口: 在主幹各層插入任務特定 conv-adapter (恢復 1W1A 準確率) 配合參數重載切換 (單一 bitstream 服務任意數量之合格任務、免重組態控制器)。與上述系統的量化正面比較詳見第 5.7 節 (表 5.25)。

1.2.2 部署密度與 1-bit 準確率落差

第 2.3 節所回顧之邊緣平台, 其主導限制並非原始吞吐量而是任務密度——固定資源預算之裝置可服務多少個不同下游任務。所回顧之每一類方法皆付出每任務之硬體代價: PR 類為每任務配置部分 bitstream (並需重組態控制器與部分 bitstream 倉庫), FixyNN 式設計於 tape-out 凍結特徵萃取器且每任務複製後端, NeuroAdaptiveNet 等晶片內模型選擇方案則將固定之合成時模型變體集合常駐晶片、每輸入選一個, 晶片成本隨集合成長。因此可部署之 AI 應用數隨晶片面積呈線性 (或更差) 成長; 於固定預算下可部署數量小。最大化單一裝置上之可部署任務密度即為本論文之核心動機。

量化與此密度問題以特定方式互動。儘管 PEFT 於軟體端已被廣泛研究, 且 Conv-Adapter [22] 與 Jie 等 [23] 之精度冗餘分析皆指出輕量 Adapter 可承受較激進量化, 於 ≥ 2 -bit 下與 full fine-tuning 接近之單分支 Adapter, 卻僅在 1-bit 出現顯著準確率下滑——此落差為 1-bit 專屬, 而非一般低精度現象。如何補回此 1-bit 落差, 使 1-bit 之密度優勢得以在不犧牲遷移準確率下使用, 即本論文以**多分支 Adapter 擴展**所針對之演算法問題; 支撐之位元寬度研究與完整評估分別見第 3 與 5 章。

本論文之超低位元沿用二值化網路文獻之慣用範圍 [7, 8]: 權重/啓動值精度至多 2

位元。於 1W1A, 乘法直接化約為 XNOR-popcount; 2 位元運算一般需多個位元平面或等效低位元操作。MARS 方法本身——adapter 插入、殘差校正、多分支擴展與單一 bitstream 參數切換——並不綁定 1 位元, 亦適用於此範圍 (乃至更高) 之精度; 惟本論文所有實作與硬體結果均採 1-bit 權重、1-bit 啟動 (1W1A) 之設計點: 一因其資源最精簡, 二因 RC 於此操作點之實測效果在所評估掃描中顯著較大 (第 3.3 節)。同理, 標題中之 reconfigurable 全文皆指於單一固定 bitstream 上之執行時期任務參數重組態, 而非 FPGA fabric 部分重組態。

因此本論文核心問題為: 在超低位元 FPGA 推論架構下, 如何讓任意數量之合格任務共用單一主幹與單一 bitstream 並提供特徵萃取器層級之適應, 同時達成快速、免重組態控制器之任務切換?

1.3 方法概述

本論文透過演算法強化與硬體設計創新, 同時處理準確率與可部署性挑戰。

1.3.1 超低位元量化下的準確率強化

本論文在準確率面之貢獻為: 於超低位元 (1W1A) 量化下將遷移準確率提升至單分支 Conv-Adapter 基線 [22] 之上, 主要由**多分支 Adapter 擴展**達成: 將單一支改為 $M \in \{2, 3, 4\}$ 組平行 Adapter 分支, 於二值化造成準確率退化之處擴充有效 adapter 容量。另一方面, 本論文分析單分支 adapter 本即帶有之 down-projection 偏置: 據我們所知, 前人未曾隔離其作用, 實證上, 無偏置 adapter 於 ≥ 2 -bit 與 full fine-tuning 接近、於 1-bit 則否, 可能係因額外量化階層留有補償空間。本論文將此既有項命名為**殘差校正 (RC)**; 受控 ablation (詳見第 5 章) 顯示移除此項會使準確率專於 1-bit 崩落, 且以折入累加器初始值之方式於 FPGA 上以零額外運算/資料路徑成本實現 (其儲存與組態容量計入 adapter 預算)。單分支 Adapter ($M=1$, 偏置開啓) 即先前工作之 Conv-Adapter 基線, 不列為本工作之貢獻。

1.3.2 硬體設計與整合

硬體核心貢獻為: 透過 Super Wrapper 架構將**資源節約、零 DSP 之 Adapter IP** (以 LUT 二值算術與分散式 RAM 實作) 整合至串流資料流資料路徑。此設計具備解耦閾值處理、零 DSP 貢獻量縮放、以及週期精準的管線同步。完整設計流程由量化感知訓練, 經主幹生成, 到自訂 RTL 整合與 FPGA 上板驗證。

1.3.3 執行時期任務切換

在 Adapter 架構之上, 本論文進一步貢獻**執行時期任務切換**基礎架構, 整套機制皆於單一 bitstream 內運作。所有任務相依參數透過統一之晶片內組態介面對外暴露。因所有合格任務共用同一凍結超低位元主幹, 單一 bitstream 即可在 $O(1)$ 晶片成本下服務任意數量之合格同主幹、同輸入尺寸、同輸出 head 分類任務——受主機端組態儲存 (隨任務數 $O(T)$ 成長) 與凍結主幹之遷移能力所限——: 每任務切換僅重新編程一小段每任務配置, 不需 FPGA 重組態控制器。此機制以五份 host 常駐任務影像於板上展示, 詳細切換結果見第 5 章; 架構上可在 $O(1)$ 晶片成本下擴展至任意數量之同主幹、同輸入尺寸、同輸出 head 分類任務。

1.4 研究目標

基於前述動機, 本論文目標如下:

1. 主要透過多分支 Adapter 擴展、輔以不增加額外運算之殘差校正, 提升超低位元微調準確率, 並在多個遷移學習資料集上驗證。
2. 建立可控的設計流程, 將串流資料流產生之主幹與自訂 RTL Adapter 模組整合。
3. 提出並實作針對核心串流運算單元 (MVAU) 之驗證方法, 以確保軟體到硬體之行爲一致性。
4. 於 FPGA 上實作並驗證完整加速器, 具備端到端推論能力。

上述目標透過一組具體研究問題達成, 每一問題均對應所提之機制與回應該問題之實驗證據。表 1.1 整理此對應關係, 作為本論文可防守之骨幹: 每一宣稱皆可追溯至特定方法與特定結果。

Table 1.1: 研究問題、所提方法, 以及回應各問題之證據。

研究問題	所提方法	證據
1W1A 下 adapter 遷移準確率相對 ≥ 2 -bit 是否顯著退化?	單分支 ($M=1$) 基線與 bit-width 掃描	bit-width 掃描與 1-bit 落差 (第 5.3.1 節; 表 3.1)
在零額外運算/資料路徑成本 (儲存計入 adapter 預算) 下, 可補回多少因 1-bit 顯著退化而損失之準確率?	殘差校正 (RC) 作為無額外算術之累加器初始化項	受控 bias 開/關消融, 寬版與 deployed 兩種幾何 (第 5.3.2 節)
多分支 Adapter 擴展能否補回 1W1A 量化下損失之遷移準確率?	多分支 Adapter 擴展 ($M=2-4$)	分支掃描 (第 5.3.3 節; 表 5.18)
軟體 adapter 能否整合進串流資料流資料路徑?	Super Wrapper 加性側流 + RTL Adapter IP	RTL 與資料流設計與驗證 (第四章; per-MVAU 模擬於第 5.4 節)
能否於單一不變 bitstream 上服務任務切換而不需重組態控制器?	執行時期組態任務相依參數 blob	上板任務切換 (第 5.6.3 節; 表 5.23–5.24)
相較 PR 類重組態, 是否更適合低延遲任務切換?	整包晶片內組態改寫 vs. PR/DFX 部分 bitstream 載入	對 ZyCAP/PCAP 之切換延遲比較 (第 5.7 節; 表 5.25)

1.5 貢獻

本論文之貢獻為相對於前人工作之架構與設計創新; 全文所報告之量測數值僅為這些設計之效果佐證, 而非貢獻本身。各項貢獻以「前人如何做、本論文改變了什麼、為何此改變本質上不同、由此得到何種效益」之結構陳述。

1. 將參數高效 adapter 整合進超低位元串流資料流加速器。前人之參數高效 adapter

(如 Conv-Adapter [22]) 僅存在於軟體、以全精度或較高位元運作, 而串流資料流加速器 (如 FINN [9]) 將單一固定網路編譯入電路、無任務適應路徑; 兩者從未結合。本論文將 adapter 設計為正交的加性側分支, 相加進主幹資料路徑卻完全不更動凍結之主幹。不同於改寫權重之方法 (如 LoRA [20]), 此側分支完全不更動主幹權重, 故能在不重新編譯或重新合成主幹的情況下, 為超低位元串流資料流加速器加上任務適應能力。於本論文所綜述之文獻範圍內, 此為前人未曾展示之、具晶片內任務適應能力之串流資料流加速器。

2. **針對超低位元情境之多分支 adapter 擴展。**參考 adapter 採單一瓶頸分支, 於主幹二值化後遷移準確率受限。本論文將 adapter 拓寬為 M 條平行分支——各分支獨立完成 down 投影、二值啟動與 up 投影, 其縮放後貢獻於共用閾值級前相加——以擴充有效 adapter 容量, 使遷移準確率一致高於單分支基線 (二十組 backbone-target 配對平均 +3.4 pp、最高 +11.3 pp; 第 5 章), 同時維持小的每任務參數負擔。
3. **將任務身分與 bitstream 解耦, 達成單一 bitstream 執行時期任務切換。**前人基於部分重組態之任務切換 FPGA 推論——ZyCAP [25]、PCAP/ACNNE [24]——將每個任務綁定其專屬之部分 bitstream、經重組態控制器載入, 故晶片成本隨任務數成長, 且切換需完整 fabric 重組態。本論文將任務身分與 bitstream 解耦: 共用之超低位元主幹一次性凍結入電路, 所有任務相依參數搬移至執行時期可寫之晶片內組態記憶體。故切換任務僅為單一不變 bitstream 上之參數記憶體改寫——無 fabric 重組態、無重組態控制器。因所有合格任務共用同一凍結主幹, 單一 bitstream 即可在 $O(1)$ 晶片成本下服務任意數量之合格同主幹、同輸入尺寸、同輸出 head 分類任務 (主機端儲存 $O(T)$); 此性質以五份 host 常駐任務影像於板上展示 (第 5.6.3 節)。

「硬體」之範圍。 雖然本論文之實作與上板驗證採用 FPGA, 所提出之架構——Adapter IP、Super Wrapper 整合、執行時期組態切換機制——在概念上可移植至相同資料路徑之 ASIC 實作, 惟本論文僅驗證 FPGA 實現。本論文中 FPGA 特定討論僅限於以 FPGA fabric 重組態 (ICAP/PCAP/ZyCAP 部分重組態) 作為比較基線之處, 因該控制器與部分 bitstream 之需求為 FPGA fabric 所本有。

文獻比較之範圍。 本論文之比較宣稱皆於明確劃定之先前已發表系統集合內評估: (i) PR 類任務切換 FPGA 推論——ACNNE [24]、CoarseToFine [25]、Boudjadar 等 [26]、

Kang & Park [27];(ii) 固定特徵萃取器設計——FixyNN [28];(iii) 晶片內動態模型選擇——NeuroAdaptiveNet [29];(iv) 軟體端 PEFT/量化——Conv-Adapter [22]、Jie 等 [23]、LoRA 家族 [20, 30, 31]。此集合透過近年於 arXiv、IEEE Xplore、ACM Digital Library、MDPI Open Access 之關鍵字檢索（“FPGA task-switching inference”、“partial reconfiguration AI accelerator”、“parameter-efficient transfer learning”、“adapter quantisation”）建立。後續章節之逐項主張僅以「據作者所知」表述，正式比較與每項宣稱之確切範圍見第 5.7 節（表 5.25）。

1.6 論文組織

本論文其餘章節組織如下：

- **第 2 章（相關研究）**：回顧超低位元 FPGA 推論、串流資料流架構、ConvNet 之 PEFT 與 Adapter 方法，以及任務切換／可重組態 FPGA 部署方案。
- **第 3 章（方法論）**：詳述 Adapter 瓶頸架構、RC 機制與多分支 Adapter 擴展策略，並提供形式化之演算法描述。
- **第 4 章（MARS NPU 架構設計）**：描述 Adapter IP 之 RTL 設計、Super Wrapper 整合、管線同步與端到端 FPGA 設計流程。
- **第 5 章（實驗與評估）**：報告跨平台之準確率、資源使用、功耗與吞吐量結果，並分析任務切換擴展特性。
- **第 6 章（結論與未來工作）**：彙整研究發現、討論限制並規劃未來延伸方向。

Chapter 2

相關研究

本章回顧與本論文最相關的三條研究脈絡：超低位元 FPGA CNN 加速、參數高效率遷移學習（PEFT）與 Adapter 方法，以及可重組態／任務切換 FPGA 架構。各節末附整理表格，標注關鍵文獻。

2.1 超低位元 FPGA CNN 加速器與資料流架構

以 FPGA 為基礎的 CNN 加速由單純的硬體映射進展到整合考慮資料流設計、低位元量化與自動化工具鏈。綜合性文獻指出主要議題包含並行策略、資料重用、記憶體頻寬最佳化、資料流映射與能效 [1, 3, 4]。Li [5] 進一步回顧了 Weight-Stationary、Output-Stationary 與 Row-Stationary 三種資料流範式，並著重於 FINN 風格的串流架構。

在超低位元加速器中，FINN [9] 為最具代表性之框架之一。FINN 針對二值化神經網路推論提出異質串流架構，硬體資源按層客製並經由晶片上資料流銜接。FINN-R [10] 將工具鏈泛化以支援任意精度之量化網路。Alam 等 [11] 分析 FINN MVU/MVAU 之 RTL 實作，證實 RTL 可提供較短的關鍵路徑與較佳的資源效率。Arend 等 [32] 提出 FINN 產生加速器中 MVAU 模組的資源預測模型。Jungemann 等 [16] 將 FINN 擴展至 HPC 環境，於大型 FPGA 平台上達成顯著吞吐量提升。Tasci 等 [33] 於 PYNQ Z1 上以 1W1A 與 2W2A 量化展示 FINN+Brevitas 的實務部署，於 2.05 W 達成 953 kFPS。

除 FINN 之外，其他工作由不同角度探索低位元 FPGA 加速。FP-BNN [34] 展現二值化神經網路於 FPGA 之實務可行性。Su 等 [13] 全面綜述 BNN 於 FPGA 之實作與多種工具鏈。Zhan 等 [14] 提出利用四向並行之邊緣 BNN 加速器。於模型層，XNOR-Net [6]、BinaryConnect [35]、Binarized Neural Networks [7]、Bi-Real Net [36] 與 ReActNet [37] 共同證實：於 1-bit 設定下維持準確率需搭配特殊的啟動函數、殘差連線與訓練策略。

Qin 等 [8] 提供系統性的 BNN 綜述。硬體設計層面, 具線緩衝組織的串流架構 [38] 與平衡資料流策略 [39] 進一步提升輕量 CNN 的吞吐效率。硬體觀點下的量化由 Zhang 等 [40] 系統性回顧, Nagel 等 [41] 則提出量化白皮書。hls4ml 框架 [42] 為另一重要之開源 ML-to-FPGA 編譯平台。

Table 2.1: 超低位元 FPGA CNN 加速器之代表性文獻整理 (依年份排序, 由新至舊)

年份	文獻	位元寬度	平台／工具	主要貢獻
2026	hls4ml [42]	混合	HLS / 多 FPGA	開源 HLS-based ML 編譯器
2025	Arend et al. [32]	1–8 bit	FINN / Zynq	MVAU 模組資源預測模型
2025	Tasci et al. [33]	1W1A, 2W2A	FINN+Brevitas / PYNQ Z1	實務 QNN 部署, 953 kFPS、2.05 W
2025	Jungemann et al. [16]	混合	FINN / Alveo U250	將 FINN 擴展至 HPC 環境
2025	Zhang et al. [40]	混合	綜述	由硬體觀點回顧量化技術
2024	Zhao et al. [39]	8-bit	自訂 RTL	輕量 CNN 之平衡資料流
2023	Alam et al. [11]	1–8 bit	FINN RTL / Zynq	FINN MVU 之 RTL 與 HLS 比較
2023	Su et al. [13]	1-bit	綜述	BNN FPGA 工具鏈全面性綜述
2023	Zhan et al. [14]	1-bit	自訂 RTL	四向並行之 BNN 加速器
2023	Kang & Yang [38]	8-bit	自訂 RTL	串流式線緩衝 CNN 加速器
2020	ReActNet [37]	1W1A	軟體 (PyTorch)	精確 BNN 之廣義啟動函數
2020	Qin et al. [8]	1-bit	綜述	BNN 技術系統性回顧
2018	FINN-R [10]	任意	FINN / 多 FPGA	將 FINN 泛化至任意精度 QNN
2018	FP-BNN [34]	1-bit	自訂 RTL / Zynq	FPGA 上實務 BNN 實作
2018	Bi-Real Net [36]	1W1A	軟體 (PyTorch)	二值化網路中加入實數捷徑
2017	FINN [9]	1-bit	FINN / PYNQ-Z1	BNN 之異質串流架構
2016	XNOR-Net [6]	1W1A	軟體 (Torch)	ImageNet 之 XNOR 二值化卷積
2015	BinaryConnect [35]	1W	軟體 (Theano)	權重二值化、梯度仍為實數
2016	Binarized NN [7]	1W1A	軟體 (Theano)	權重與啟動值皆二值化

儘管此領域進展顯著, 現有超低位元 FPGA 加速器研究仍大多聚焦於單模型部署。主流假設為加速器服務於單一固定模型, 最佳化目標為該工作負載的吞吐量、延遲與資源效率。於既有低位元資料路徑上加入任務特定適應且同時保留共享主幹的方法, 仍付之闕如。

2.2 參數高效率遷移學習與 Adapter 方法

隨著大型預訓練模型廣被採用, 如何在不複製所有模型參數的情況下適應下游任務已成為關鍵研究題。Adapter 範式最早由 Houlsby 等 [19] 為 NLP 提出, 於 Transformer 層間插

入瓶頸模組。LoRA [20] 後續提出以權重更新之低秩分解作為另一種 PEFT 策略。Visual Tuning 綜述 [17] 將近期視覺微調方法歸為五類——fine-tuning、prompt tuning、adapter tuning、parameter tuning、remapping tuning——為視覺 PEFT 建立清晰的方法論地圖。

針對 ConvNet, Conv-Adapter [22] 最具代表性。作者主張 ConvNet 中間表示為具空間局部性的結構化特徵圖, Adapter 必須保留此局部性, 不能直接沿用 Transformer 風格的設計。Conv-Adapter 引入基於深度可分離卷積的瓶頸模組, 證實僅需約 3.5%–5% 主幹參數即可在 23 個跨領域分類資料集上媲美或超越全模型微調。Jie 等 [23] 的精度冗餘分析進一步顯示, Adapter 相較全微調具有較平坦的損失地景, 意味其對量化雜訊更具韌性; 其實驗顯示僅將 Adapter 量化為 1-bit 於軟體端幾乎無性能下降, 說明低位元 Adapter 是有意義的建模方向。

近期工作將 PEFT 擴展至更廣脈絡。LoRA-C [30] 將低秩分解套用於卷積層, 供 IoT 部署。LoRA-Edge [31] 提出以 tensor-train 協助的 LoRA, 用於 Jetson Orin Nano 等邊緣裝置上的 CNN 微調, 以 1.49% 可訓練參數達成 $3.8\times$ 更快收斂。PEFT-Bench [21] 提供統一評測多種 PEFT 方法的標準化基準。Cache-Adapter [43] 將 Adapter 微調延伸至影片動作辨識, 凍結主幹並僅訓練插入之 Adapter 區塊。Liu 等 [12] 綜述資源受限環境下的輕量深度學習技術, 涵蓋架構與壓縮兩大面向。

本論文為何採用 adapter 而非其他 PEFT 家族。 本論文之硬體目標選擇 adapter (而非 LoRA、prompt tuning 等其他 PEFT 家族), 基於兩項文獻支持之考量。第一, 串流資料流目標於合成時將主幹權重編譯為固定 memstream [9, 10], 因此凡是在標準部署 (合併) 形式下將更新折入主幹權重之每任務 PEFT 方法 (如 LoRA 之 $W+BA$ 形式 [20, 30, 31]), 皆需每任務重新產生 FINN memstream (因而需重 build bitstream); 若保留低秩旁路不合併雖可避免改寫, 卻留下一條稠密全精度路徑, 無法映射至 XNOR–popcount MVAU 原語。adapter 式 PEFT 則插入一條加性側流, 主幹權重不動, 並可二值化映射至同一串流原語 (第 3 章), 自然以側 FIFO + 累加器融合方式與串流資料流運算單元整合 (第 4.3 節)。第二, 特定於 ConvNet, Conv-Adapter [22] 證實 depthwise/ 1×1 bottleneck adapter 保留特徵圖之空間區域性——此性質為 Transformer 式 adapter 或 prompt tuning 所不具——且以約 3.5% 之主幹參數即可於 23 個分類任務匹配或超越完整微調; Jie 等 [23] 並指出 adapter 具更平坦損失景觀, 僅將 adapter 量化至 1-bit 於軟體端衰減可忽略 (其 *backbone* 維持全精度); 於本論文所針對之 $1W1A$ *backbone* 下, 單分支 1-bit adapter 已不足, 此即多分支 Adapter 擴展所欲補足之落差 (第 5 章)。此兩項文獻支持之考量——側流整合性與 adapter 特有之 1-bit 量化韌性——共同使 adapter 成為 FINN 式串流硬體加速器上最直接可部署之 PEFT 類別。

Table 2.2: PEFT 與 Adapter 類方法之代表性文獻整理（依年份排序, 由新至舊）

年份	文獻	領域	方法類型	可訓練比例	主要貢獻
2025	Cache-Adapter [43]	影片	Adapter	<5%	影片動作辨識之 Adapter 微調
2025	LoRA-Edge [31]	視覺	LoRA+TT	1.49%	邊緣 CNN 微調之 tensor-train LoRA
2025	PEFT-Bench [21]	多	Benchmark	多樣	具成本度量之統一 PEFT 基準
2024	Conv-Adapter [22]	視覺	Adapter	3.5–5%	結合深度卷積之 ConvNet 專用 Adapter
2024	Visual Tuning [17]	視覺	綜述	多樣	視覺 PEFT 五大類別分類法
2024	LoRA-C [30]	IoT	LoRA	<2%	IoT 部署之低秩卷積適應
2024	Liu et al. [12]	通用	綜述	多樣	受限裝置輕量 DL 綜述
2023	Jie et al. [23]	視覺	Adapter	<5%	精度冗餘分析, 1-bit Adapter 可行
2022	LoRA [20]	NLP	LoRA	0.1–1%	權重更新之低秩分解
2019	Houlsby et al. [19]	NLP	Adapter	~3%	Transformer 原始 Adapter 瓶頸

儘管軟體端證據充足, Adapter 文獻仍存關鍵限制: 研究焦點集中於建模與訓練方法, 未處理 Adapter 如何映射至 FPGA 串流資料路徑、如何與底層資料流運算架構共同設計、如何落實於超低位元共享主幹之上。此缺口為 MARS 的主要動機之一。

2.3 可重組態、自適應與共享 FPGA 架構

與本論文緊密相關的第三條脈絡為 FPGA 平台上之可重組與任務切換支援。AC-NNE [24] 提出利用部分重組態（PR）在執行時期動態重組卷積層硬體區塊的 Adaptive CNN Engine。Rangsikunpum 等 [25] 以 PR 於外部記憶體中切換多個細粒度分類模型。FixyNN [28] 將特徵萃取器固定、後端保留可程式性, 肯定部分主幹共享的硬體價值。

近期, ElBouazzaoui 等 [29] 提出具動態模型選擇之 NeuroAdaptiveNet, 透過自適應分類器選擇取得 52.85% LUT 節省。Boudjadar 等 [26] 於 Xilinx ZYBO 上展示結合 CNN 自適應與動態 PR 的共同設計方法。Kang 與 Park [27] 提出以遮罩式部分更新結合動態重組態的執行時期穩健邊緣推論。Manca 等 [44] 提出的 ONNX-to-hardware 流程, 進一步顯示自適應 FPGA 推論的持續熱度。Khaki 與 Choi [15] 研究經 Vitis-AI 部署於 FPGA 的

遷移學習; 更廣的邊緣 AI 部署景觀亦有多篇近期綜述 [2]。

Table 2.3: 可重組態與任務切換 FPGA 架構之代表性文獻整理 (依年份排序, 由新至舊)

年份	文獻	重組方法	平台	主要貢獻
2025	NeuroAdaptiveNet [29]	晶片內模型選擇	Zynq	動態分類器/變體選擇, LUT 節省 52.85%
2025	Boudjadar et al. [26]	部分重組態	ZYBO	結合動態 PR 之 CNN 自適應共同設計
2025	Kang & Park [27]	遮罩式更新	Zynq	部分遮罩式穩健邊緣推論
2025	Khaki & Choi [15]	Vitis-AI	ZCU104	FPGA 遷移學習部署
2024	ACNNE [24]	部分重組態	Zynq	利用層級 PR 之 Adaptive CNN engine
2024	Coarse-to-Fine [25]	部分重組態	Zynq	以 PR 切換細粒度任務模型
2024	Manca et al. [44]	設計流程	多	自適應邊緣推論之 ONNX-to-hardware
2019	FixyNN [28]	權重重載	ASIC	固定前端 + 可程式後端 (約 60/40%)

此類工作的共通模式為: 自適應透過模型層級或配置層級切換達成——置換整個模型、重組硬體分區、或於預先編譯的執行檔中擇一。所有方法皆未將任務特定差異壓縮為共享超低位元主幹上的參數級差異 (parameter-level deltas)。

除資源開銷外, PR 類任務切換設計還面臨一個較少被討論的根本性任務切換延遲問題。Rangsikunpum 等 [25] 測得使用 ZyCAP (Zynq 上最高吞吐之控制器) 之重組態延遲為 5.2 ms; 值得注意的是此延遲已超過其管線中任一推論階段之執行時間, 意即一旦需要模型切換, 重組態步驟即成為主導時間。因此其設計將 PR 限於不頻繁的分類目標轉換 (例如切換監視系統之觀察目標) 以避免影響整體吞吐量。使用更普及之 PCAP 介面時, 開銷更嚴重: Huang 等之 ACNNE [24] 每個可重組分割回報 97–114 ms 之配置延遲。此結果顯示, 即便 PR 類邊緣推論研究活躍, 重組態延遲本身仍為一項大致未解的阻礙, 對需要快速或頻繁任務切換之應用尤其如此。

MARS 位於此設計空間之另一點: 其排除 fabric 重組態、不依賴合成時固定之變體集合, 並以輕量每任務參數 blob (約 25 KB, 經 MMIO) 提供特徵萃取器層級之 adapter 適應, 達成約 1.86 ms 經驗證之任務轉換、免重組態控制器且不改動 FPGA 邏輯, 單一 bitstream 服務任意數量之合格任務 (共用凍結主幹、相同輸入與輸出 head 介面; 第 5.6.3 節); 其相對選擇式家族之優勢在於結構可擴充性與適應層級, 而非選擇器延遲。

本章小結

表 2.4 整理了 MARS 相對於前述三條研究脈絡的定位。三條脈絡各自已解決問題的一部分: 低位元 FPGA 加速器提供高效之 1W1A 串流推論, 但將單一固定任務編譯進邏輯; PEFT/adapter 方法將任務差異限縮於小的參數增量, 但僅存在於全精度或較高位元之軟體; 可重組態 FPGA 方法可於執行期切換任務, 但需 fabric 重組態控制器與每任務 bitstream, 或於合成時固定變體集合。尚無工作同時具備此三性質; MARS 即位於此交集——超低位元串流推論、adapter 式任務表示、單 bitstream 執行期切換——並完成上板驗證。

Table 2.4: 研究缺口分析:MARS 之定位

能力	低位元 FPGA 加速器	PEFT / Adapter	可重組態 FPGA	MARS
超低位元 (1W1A) 推論	✓	—	✓	✓
Adapter 式任務表示	—	✓	—	✓
共享主幹、參數增量任務	—	✓	△	✓
單 bitstream 執行期任務切換 (無 PR)	—	—	△	✓
端到端 FPGA 驗證	✓	—	✓	✓

✓: 已處理; △: 部分處理 (PR 類切換需 fabric 重組態控制器與每任務部分 bitstream; 晶片內模型選擇之變體集合於合成時固定); —: 未處理。

Chapter 3

方法論

本章介紹本論文提出之演算法與架構方法。第 3.1 節將問題形式化; 第 3.2 節說明 Adapter 瓶頸設計; 第 3.3 節介紹殘差校正機制; 第 3.4 節介紹多分支 Adapter 擴展。

3.1 問題形式化與設計動機

本節梳理 MARS 設計過程所遇到的挑戰鏈, 並說明每項挑戰如何促成特定架構決策。

挑戰 1: 現行任務切換邊緣 FPGA 方法皆伴隨控制器層級開銷或 tape-out 層級剛性。 任務切換邊緣 FPGA 文獻收斂於三種策略——部分重組態 (ACNNE [24]、CoarseToFine [25]、Boudjadar [26]、Kang & Park [27])、固定特徵萃取器配可程式後端 (FixyNN [28])、以及晶片內動態分類器選擇 (NeuroAdaptiveNet [29])。每類方法於其原始論文中皆有記錄其特定之運作代價: PR 類需長駐重組態控制器與部分 bitstream 倉庫, 並承受前述延遲; FixyNN 於 tape-out 凍結特徵萃取器, 無法進行特徵層級適應; NeuroAdaptiveNet 則將固定之合成時模型變體集合全部常駐晶片、以選擇器每輸入選一個, 晶片成本隨集合成長且集合於合成時固定。基於這些記錄之代價, MARS 的設計目標是: 結合特徵萃取器層級的任務適應 (在 1W1A 這個公認困難的設定下恢復準確率所必須 [6, 7]) 與單一 bitstream、免重組態控制器、可服務任意數量合格任務 (第 5.6.3 節; 主機端儲存隨任務數 $O(T)$ 成長) 之任務切換。此延遲主張僅限 PR 類重組態基線; 相對 NeuroAdaptiveNet, MARS 之區別在結構可擴充性與特徵萃取器層級適應, 而非選擇器延遲。

挑戰 2: 尚無經 RTL 驗證之 Adapter 硬體報導。 雖然 PEFT Adapter 已於軟體端被廣泛研究 (LoRA [20]、Conv-Adapter [22]、Houlsby Adapter [19]), 於本論文所調查之研

究中, 我們未見經 RTL 驗證之 Adapter 模組整合至 FPGA 串流資料流 CNN 管線。現有 FPGA 加速器研究聚焦單模型部署, 未處理 Adapter 瓶頸運算如何映射至硬體處理元。因此 MARS 必須從零設計客製化 Adapter 硬體。

挑戰 3: 單一 1-bit Adapter 分支並不足夠。 於邊緣 FPGA 建置客製化硬體要求積極量化以壓低資源。雖 1-bit 權重與 1-bit 啟動值 (1W1A) 在 CNN 主幹已被廣泛接受 [6, 7], 但將相同超低位元量化套用於單分支 Conv-Adapter ($M=1$) 並不足夠: 本論文之實驗觀察到, 僅於 1-bit 時 (表 3.1), 單分支 Adapter 之整體貢獻顯著下滑。沿用自 Conv-Adapter [22] 之單分支設計因此僅保留為基線; 在將 Adapter 路徑投入超低位元硬體之前, 必須另行提出專屬之準確率恢復機制。

表 3.1 為 bit-width 掃描, 將單分支 Conv-Adapter [22] ($M=1$) 與 *Full-FT* (解凍全網路共同 fine-tuning, 作為同訓練預算下之實務參考) 並列, 且 Adapter 以兩種形式評估: 已發表之配置 (含 down-projection 偏置——即本論文所隔離並命名之殘差校正 (RC, 第 3.3 節)) 與移除該偏置之 *No-RC* 版本。此掃描揭露兩個問題, 皆專屬於 1-bit。其一, 該偏置於已發表配置中預設開啓、據我們所知前人從未隔離, 卻恰於 1-bit 為承重元件: 移除它於 1-bit 損失最高 16.2 pp (SVHN 79.23 $\rightarrow$ 63.06), 於 ≥ 2 -bit 至多 1.4 pp (多數落於 ± 0.3 pp 內; 唯一較大偏移為 STL10 於 8-bit 之 +1.4 pp)。此前所未見之 1-bit 相依性為本論文之發現; 第 3.3 節將其轉化為不增加額外運算/資料路徑成本之累加器重置機制, 第 5 章並做完整消融 (第 5.3.2 節)。其二, 即便開啓該偏置, 單分支於 ≥ 2 -bit 與 Full-FT 差距僅 ≈ 1 -5 pp, 卻於 1-bit 仍存明顯落差 (如 SVHN 79.23% 對 94.91%、FashionMNIST 82.71% 對 92.36%) ——此殘餘不足由本論文之多分支擴展加以解決。

Table 3.1: 單 Conv-Adapter [22] 之 bit-width 掃描: 含 (RC) / 不含 (No-RC) down-projection 偏置 vs. Full-FT。

Bit	SVHN			FashionMNIST			STL10			CINIC10		
	No-RC	RC	Full-FT	No-RC	RC	Full-FT	No-RC	RC	Full-FT	No-RC	RC	Full-FT
1	63.06	79.23	94.91	77.94	82.71	92.36	66.91	67.50	71.05	64.20	64.50	68.02
2	95.45	95.54	96.07	93.27	93.26	94.11	72.91	72.67	77.44	75.93	75.83	78.24
4	95.75	95.82	96.07	94.07	93.90	94.48	75.84	76.25	79.47	77.74	77.85	80.01
8	95.83	95.79	96.13	93.94	93.78	94.53	74.45	75.89	78.91	77.94	77.79	80.14

此 1-bit 專屬之落差不限於 CIFAR-10 主幹或這四個 target。表 3.2 將單 Adapter (RC) 之逐位元掃描擴及全部二十組 backbone $\rightarrow$ target 組合; 於每一組合, 1-bit 準確

率皆遠低於其 2/4/8-bit 對應值，顯示所觀察之落差不限於所評估 VGG-like 家族中之單一轉移。

Table 3.2: 單 Conv-Adapter [22](RC) 跨更廣 backbone-target 之 bit-width 掃描。

Backbone	Target	1-bit	2-bit	4-bit	8-bit
CIFAR-10	SVHN	79.23	95.54	95.82	95.79
CIFAR-10	STL10	67.50	72.67	76.25	75.89
CIFAR-10	FashionMNIST	82.71	93.26	93.90	93.78
CIFAR-10	CINIC10	64.50	75.83	77.85	77.79
SVHN	CIFAR-10	46.34	79.34	82.88	82.93
SVHN	STL10	35.16	54.16	59.38	58.76
SVHN	FashionMNIST	81.47	92.74	93.64	93.39
SVHN	CINIC10	32.76	64.96	69.04	69.75
STL10	CIFAR-10	68.92	83.18	86.21	86.50
STL10	SVHN	73.22	95.42	95.76	95.48
STL10	FashionMNIST	84.71	93.07	94.06	94.05
STL10	CINIC10	56.07	70.59	73.56	73.99
FashionMNIST	CIFAR-10	30.42	76.16	82.53	83.52
FashionMNIST	SVHN	39.16	94.65	95.54	95.54
FashionMNIST	STL10	30.84	51.10	58.44	58.98
FashionMNIST	CINIC10	24.82	60.31	67.65	68.84
CINIC10	CIFAR-10	78.05	88.42	90.46	90.54
CINIC10	SVHN	67.43	95.49	95.94	95.72
CINIC10	STL10	66.42	74.90	76.63	74.19
CINIC10	FashionMNIST	79.81	93.38	94.00	93.77

解法: 多分支 Adapter 搭配殘差校正。 MARS 於 1-bit 區段成功將準確率提升至 Conv-Adapter 基線之上, 其機制為引入**多分支 Adapter (MB-Adapter)** 擴展: 不採 Conv-Adapter 單分支 [22] ($M=1$, 先前工作), 改採 $M \in \{2, 3, 4\}$ 組平行 Adapter 分支, 其輸出於閾值化前相加。每一分支學習不同校正子空間, 增加 M 可漸進提升遷移準確率——與恢復 1-bit 量化下損失之有效容量一致——且資源隨 M 線性、可預測地成長 (第 3.4 節)。多分支 Adapter 擴展為本論文達成準確率提升之機制。

作為互補機制，MARS 另納入**殘差校正 (RC)**：每組 Adapter 之 down convolution 之 Int8 量化偏置，作為獨立 FPGA 元件匯出，並於 RTL 層級吸收至累加器初始化（第 3.3 節）。RC 不需額外加法器、DSP 或管線階段。針對四種 multi-branch adapter 寬度、於寬版與 deployed 兩種幾何進行之獨立 ablation（第 5.3.2 節）顯示，於 1W1A 主幹之 CIFAR-10→SVHN 上，RC 於寬版幾何貢獻 +15 至 +17、於 deployed 幾何貢獻 +20 至 +23 個百分點（其餘目標較小，隨其 Full-FT 可回收空間而變），於 ≥ 2 -bit 主幹下 RC 對準確率之影響至多約 1.4 個百分點、多數低於 0.3（表 3.1）。此 1-bit 下之 disproportionate role 與二值 sign 函數失去可學習逐通道偏移項之機制一致，亦支持於部署 bitstream 中無條件納入 RC 之設計決策。

挑戰 4: 完全手寫主幹 RTL 過於耗時。 以手寫 Verilog 實作整個加速器（主幹與 Adapter）將耗時過巨。FINN 框架 [9, 10] 提供自動化管線：將 PyTorch 量化模型編譯為 MVAU 串流資料路徑，於數分鐘內產生主幹之可合成 RTL，而非數月。

挑戰 5: FINN 不支援 Adapter 生成。 FINN 雖能高效率產生主幹資料路徑，卻無內建 Adapter 模組支援。其編譯器無法產生 Adapter 瓶頸 RTL，所產 MVAU 包裝亦未暴露 Adapter 整合所需之內部訊號（累加器值、週期計數器）。另一方面，大規模修改 FINN 產生之 RTL 十分脆弱：FINN 自動化管線於每次重編時覆寫輸出，且產生之程式碼不利人類閱讀——因此任何手動修改都必須極小化、局部化，並施加於產生 RTL 之凍結快照上，而非置於重生成迴圈之內。

MARS 之解法: FINN + 自訂 RTL 混合整合流程。 MARS 藉提出混合設計流程解決此問題。主幹 MVAU RTL 由 FINN 產生；Adapter 模組、Stream Splitter、Stream Adder、FIFO 緩衝區、以及外層 Super Wrapper 皆為手寫 Verilog。自訂整合腳本合併兩者：自 FINN 產物中抽出 MVAU IP，將之包裝於 Super Wrapper 內（以路由資料同時經主幹與 Adapter 路徑），並產生可供合成之統一頂層設計。對 FINN 產生程式碼之唯一手動修改，正是此凍結快照上之最小局部補丁——將各承載 adapter 之 MVAU 的原始部分和外露（表 4.1）——其餘整合皆由 wrapper 於外部完成。此流程保留 FINN 於主幹的高生產力，同時使 Adapter 的時序、權重儲存與多分支累加於 RTL 層級完全可控（實作細節見第 4 章）。

3.2 二值化 Adapter 瓶頸架構

Adapter 模組保留 Conv-Adapter [22] 之瓶頸拓樸——降維、非線性、升維——但將其內部之每一個算術運算全部置換, 使 Adapter 能映射到與 1W1A 主幹相同之二值串流原語。此再設計為本論文之演算法核心, 也是 MARS 與原始全精度 adapter 之根本差異所在。

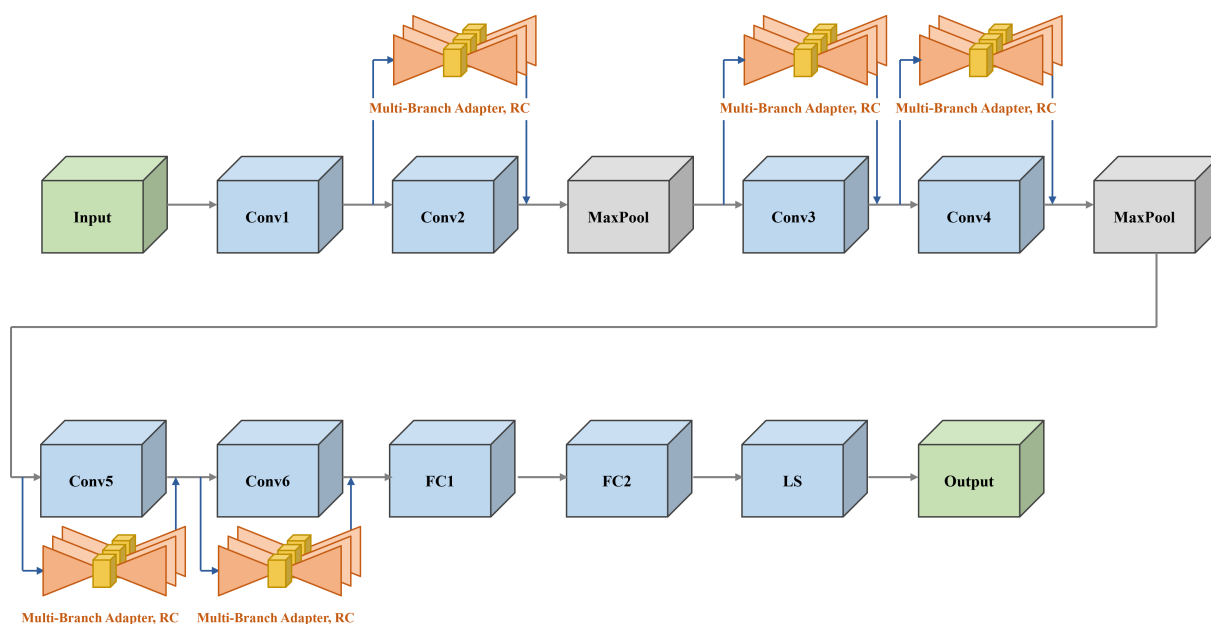


Figure 3.1: MARS 軟體架構。

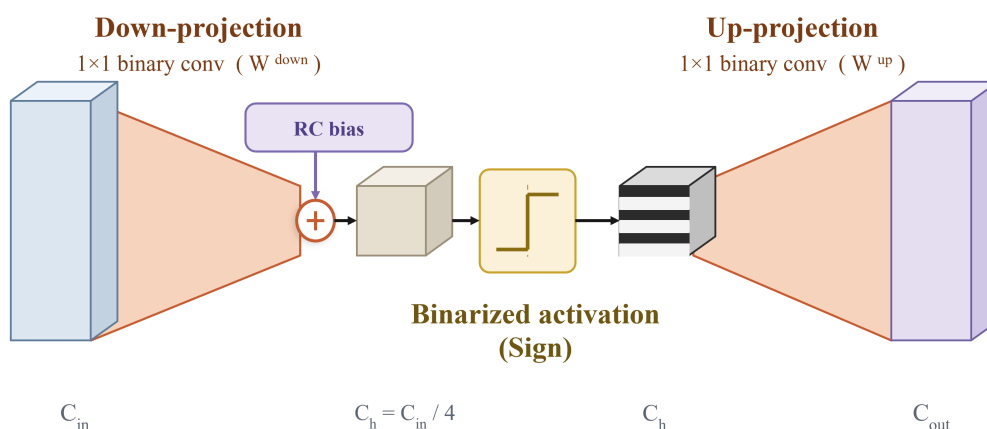


Figure 3.2: Conv-Adapter [22] 瓶頸二值化至 1W1A。

MARS 相對於原始 Conv-Adapter 改了什麼。 原始 Conv-Adapter 為全精度模組: 其 3×3 depthwise down 卷積與 1×1 up 投影為實數乘加卷積, 非線性為實數 ReLU, 輸出再乘以一個學習得到之浮點縮放因子 α 。這些運算若不配置專用乘法器與啟動邏輯, 皆無法置於超低位元串流資料路徑上。MARS 因此於演算法層逐項置換:

- **空間算子:** 3×3 depthwise down 卷積 $\rightarrow 1 \times 1$ 。於 FPGA 部署幾何中, down 卷積由原始之 3×3 depthwise 核縮減為 1×1 pointwise 投影, 完全免除鄰近像素之 line buffer; 軟體寬版幾何保留 3×3 核, 此縮減之準確率代價於第 5.3.7 節 (表 5.7 與 5.8) 量化。
- **投影: 實數乘加 $\rightarrow$ XNOR-popcount。** 於 1-bit 權重與 1-bit 啟動下, 兩運算元皆取值於 $\{-1, +1\}$, 故每個乘法退化為一個位元 XNOR、累加退化為 popcount (式 3.2 與 3.4) ——即與 1W1A 主幹全程所用相同之二值內積原語。位元以 $\{0, 1\}$ 儲存, 於 $\{0, 1\} \leftrightarrow \{-1, +1\}$ 對應下 $\{-1, +1\}$ 內積等於 $2 \cdot \text{popcount} - N$; 常數 $-N$ 吸收進逐通道 bias/threshold, 故下式直接使用 XNOR-popcount 計數。
- **非線性: ReLU $\rightarrow$ 二值 sign。** 實數 ReLU 由二值 sign 函數取代 (式 3.3), 每隱藏通道僅輸出一個位元。正是此置換使隱藏表示成為 1-bit, 並使整條 adapter 路徑維持二值。
- **偏置成為一級且經分析之項 (RC)。** 以二值 sign 取代 ReLU 會丟棄 ReLU 之偏置原本提供的每通道加性平移。MARS 將此偏置獨立提出為殘差校正 (第 3.3 節), 並——不同於原始 Conv-Adapter 默默保留偏置且從未分析——證實其實測效果於 1W1A 顯著大於 2-8 bit。

其結果保留 down-projection $\rightarrow$ 二值啟動 $\rightarrow$ up-projection 之瓶頸拓撲, 並為硬體相容改變其空間運算子與算術, 得到完全二值之資料路徑外加一個小型 Int8 校正項; 每分支之學習縮放 α 沿用自 Conv-Adapter。此再設計所倚賴之兩個準確率機制——多分支擴展 (第 3.4 節) 與 RC (第 3.3 節) ——於後續各節分析; 而這些運算如何映射至串流硬體 (含 RC 之累加器重置實作與吸收 α 之查表) 則留至第 4 章。本節其餘部分形式化此三個二值運算。

給定具 C_{in} 通道的輸入特徵圖, Adapter 進行三階段連續運算:

Down-Projection。 輸入啟動由 C_{in} 通道投影至縮減之隱藏維度 C_h :

$$C_h = \left\lfloor \frac{C_{\text{in}}}{r} \right\rfloor, \quad (3.1)$$

其中 r 為縮減比例, 在 MARS 所有配置皆設為 $r = 4$ 。對每個 hidden unit j , down-projection 在 C_{in} 個輸入通道上計算 down 權重與輸入之二值內積, 並以 RC 偏置初始

化:

$$h_j = b_j + \sum_{i=1}^{C_{in}} \text{XNOR}(W_{j,i}^{\text{down}}, x_i), \quad j = 1, \dots, C_h. \quad (3.2)$$

各符號定義：

- h_j ：hidden 通道 j 之累加值（16-bit 有號暫存器）。
- b_j ：通道 j 之匯出硬體域 8-bit 有號（Int8）RC 偏置（第 3.3 節），符號延伸至 16-bit 累加器寬度，於累加開始前作為累加器初始值載入；此匯出值已吸收下述二值內積偏移，並非原始浮點訓練偏置。
- $W_{j,i}^{\text{down}} \in \{0, 1\}$ ：由輸入通道 i 至 hidden 通道 j 之二值 down-projection 權重，存於分散式 RAM（ram_down）。
- $x_i \in \{0, 1\}$ ：通道 i 之二值輸入啟動。
- $\text{XNOR}(\cdot, \cdot)$ ：位元 XNOR 運算（兩 bit 相符為 1）；求和 $\sum_i \text{XNOR}(W_{j,i}^{\text{down}}, x_i)$ 即 XNOR-popcount 二值內積，硬體上以 XNOR 後接 popcount、依 SIMD 寬度分塊實現（第 4 章）。
- C_{in}, C_h ：輸入與 hidden 通道數。

XNOR-popcount 為 BNN 之標準運算 [6, 7]。MARS 之設計選擇在於其組織方式：累加器以 RC 偏置 b_j （而非 0）初始化，實現免額外硬體之混合精度校正（第 3.3 節）。

二值化啟動。 hidden 表示透過符號提取二值化：

$$a_j = \begin{cases} 1 & h_j \geq 0, \\ 0 & \text{否則}, \end{cases} \quad \mathbf{a} = (a_1, \dots, a_{C_h}). \quad (3.3)$$

此運算只需每個 hidden 累加器的 MSB，無需專門的比較器邏輯。

Up-Projection。 二值 hidden 向量 $\mathbf{a} \in \{0, 1\}^{C_h}$ 投影回 C_{out} 個輸出通道。對每個輸出通道 o ，輸出值為 up 權重與 hidden 向量之二值內積：

$$z_o = \sum_{j=1}^{C_h} \text{XNOR}(W_{o,j}^{\text{up}}, a_j), \quad o = 1, \dots, C_{out}, \quad (3.4)$$

其中：

- z_o : 通道 o 之輸出值 (8-bit 無號 popcount)。
- $a_j \in \{0, 1\}$: 通道 j 之二值 hidden 啓動。
- $W_{o,j}^{\text{up}} \in \{0, 1\}$: 由 hidden 通道 j 至輸出通道 o 之二值 up-projection 權重, 存於分散式 RAM (ram_up)。
- C_h, C_{out} : hidden 與輸出通道數。

硬體上, 輸出通道以每次 P 個 (PE 平行) 方式產生, 並將對應之 P 個權重向量打包成單一寬 RAM 字 (每一輸出步一次); 完整 PE/SIMD 對應見第 4 章。

空間中心選擇。 對於主幹之 3×3 卷積層, Adapter 只對每個 3×3 滑動視窗之中心像素運算, 不處理全九個位置——此即 Adapter 之 1×1 down 卷積於主幹視窗串流上的實現方式。因 FINN MVAU 將主幹視窗之九個 kernel 位置以串流方式逐週期送出 (每週期一個輸入 chunk), Adapter 只需處理中心位置 (位置 4/0-8), 即可減 $9 \times$ 運算量。中心像素以週期計數器辨識:

$$\text{中心啓用} \iff 4K \leq \text{cycle_cnt} < 5K, \quad (3.5)$$

其中 $K = C_{\text{in}}/S$ 為每個 kernel 位置的 chunk 數。Adapter 之有限狀態機據此閘控累加, 僅於中心像素 chunk 處理完成後產生有效輸出。

表 3.3 列出各卷積層之 Adapter 配置。

Table 3.3: 各層 Adapter 瓶頸配置

層	C_{in}	C_{out}	r	C_h
Conv1 (MVAU0)	3	64	—	—
Conv2 (Super1)	64	64	4	16
Conv3 (Super2)	64	128	4	16
Conv4 (Super3)	128	128	4	32
Conv5 (Super4)	128	256	4	32
Conv6 (Super5)	256	256	4	64

Conv1 不插入 Adapter, 因其負責通用低階特徵萃取 [22]。

單一 Adapter 之參數開銷為：

$$\Delta P = \sum_{l=1}^5 (C_{\text{in}}^{(l)} \cdot C_h^{(l)} + C_h^{(l)} \cdot C_{\text{out}}^{(l)} + C_h^{(l)} + 1) = 58,533, \quad (3.6)$$

其中各層之項分別為 down-projection ($C_{\text{in}}^{(l)} C_h^{(l)}$)、up-projection ($C_h^{(l)} C_{\text{out}}^{(l)}$)、Int8 RC 偏置 ($C_h^{(l)}$) 與純量 α (+1)；約為主幹參數之 $\Delta P / P_{\text{backbone}} = 58,533 / 1,546,690 \approx 3.78\%$ 。

完整 Adapter 前向運算彙整於演算法 3.1。

Algorithm 3.1 二值化 Adapter 前向運算

Require: 輸入串流 $\mathbf{x}^{(k)}, k = 0, \dots, K - 1$; 權重 $\mathbf{W}_{\text{down}}, \mathbf{W}_{\text{up}}$; RC 偏置 $\mathbf{b}_{\text{RC}}$

Ensure: 每輸出步之 P 維輸出 popcount 向量 $\mathbf{o}$

```

1: // 階段 1：Down-Projection (對輸入 chunk 疊代)
2: for  $j = 0$  to  $C_h - 1$  do
3:    $h_j \leftarrow b_{\text{RC}}^{(j)}$  ▷ 以 RC 偏置初始化
4: end for
5: for  $k = 0$  to  $K - 1$  do ▷ 僅處理中心像素之 chunk
6:   for  $j = 0$  to  $C_h - 1$  do
7:      $h_j \leftarrow h_j + \text{popcount}(\text{XNOR}(\mathbf{w}_{\text{down}}^{(j,k)}, \mathbf{x}^{(k)}))$ 
8:   end for
9: end for
10: // 階段 2：二值化啟動
11: for  $j = 0$  to  $C_h - 1$  do
12:    $a_j \leftarrow \mathbf{1}[h_j \geq 0]$  ▷ 符號提取
13: end for
14: // 階段 3：Up-Projection (對輸出步疊代)
15: for  $s = 0$  to  $C_{\text{out}}/P - 1$  do
16:   for  $p = 0$  to  $P - 1$  do
17:      $o_p \leftarrow \text{popcount}(\text{XNOR}(\mathbf{a}, \mathbf{w}_{\text{up}}^{(s,p)}))$ 
18:   end for
19:   輸出  $(o_0, o_1, \dots, o_{P-1})$ 
20: end for

```

3.3 殘差校正 (RC) 與 1-bit 之經驗發現

3.3.1 定義與硬體實作

本論文採用 Chen 等人 [22] 之 Conv-Adapter 瓶頸結構作為基礎 Adapter：每一分支由 1×1 down convolution、二值啟動、 1×1 up convolution 與可學習縮放因子 α 組成。本文將 down convolution 之 Int8 量化偏置定名為殘差校正 (Residual Correction, RC)，沿用硬體匯出管線之命名 (匯出為獨立 FPGA 元件 `adapter_{l}_rc.dat`，五個逐

層檔案，8-bit 有號）。本論文之「殘差校正（RC）」專指 Adapter 之 down-convolution Int8 量化偏置（匯出為 adapter_{l}_rc.dat、於累加器 reset 時吸收; 第 3.3 節），與一般 ResNet 式之「殘差連接（skip-link）」屬於不同概念，但保留既有縮寫 RC。此處之偏置 b_j 即式 (3.2) 之 down-projection 累加器中的加項。RTL 層級下 RC 不引入額外加法器、DSP slice 或管線階段：逐通道 Int8 偏置吸收至累加器之重置值（第 4.3 節），校正僅以既有暫存器之控制路徑多工實現；其分散式 RAM 儲存（ram_rc）與累加器初始化控制計入 adapter 開銷。

$$\text{hidden_acc}[j] \xleftarrow{\text{init}} \text{ram_rc}[j], \quad j = 0, \dots, C_h - 1. \quad (3.7)$$

RC 值儲於一小型分散式 RAM（ram_rc, $C_h \times 16$ bits），初始化動作吸收至既有之重置／視窗起始控制邏輯。

3.3.2 RC 為何於 1-bit 下重要

於 1-bit weight、1-bit activation（1W1A）下，down convolution 之輸出送入二值 sign 函數。若無可學習偏移項，sign 函數退化為固定之零閾值決策，Adapter 失去逐通道調整此閾值之能力。一旦啟動寬度增至 2 bit 以上，RC 之實測作用即變得很小（表 3.1）；一個可能的解釋為額外量化階層保留了下游各層可利用之幅值資訊，惟此屬假說而非已驗證之機制。RC 因此並非一般性之增強機制，而是專門用於補回 1-bit 二值啟動所抹除之逐通道偏移項。

3.3.3 經驗發現：RC 於 1-bit 下之 disproportionate role

原 Conv-Adapter 之 ablation 程序對 RC 效果之估計偏低，原因為 PyTorch Conv2d 之 bias=True 預設行為，使得使用者層級之 no_rc 旗標實際上未關掉 bias。本論文重新實作 ablation：RC 關閉時嚴格設定 bias=False, bias_quant=None，RC 開啓時則為 bias=True, bias_quant=Int8Bias，其餘 Adapter 設計維持不變。

兩項發現直接源於此修正後之 ablation（完整表格見表 3.1 及第 5.3.2 與 5.3.5 節）：

1. **1-bit 下 RC 關鍵**：於 CIFAR-10→SVHN 1W1A 下，關閉 RC 於所有 multi-branch adapter 配置（ $M=1-4$ ）下，於寬版幾何準確率下降 15–17 個百分點、於 deployed 幾何下降 20–23 個百分點。此現象與前述二值 sign 函數論述一致。
2. **≥ 2 -bit 下 RC 之實測效果顯著較小**：於 2、4、8 bit 之 backbone activation 寬度

下，於同一 source-target 配對中關閉 RC 之準確率差異至多 1.4 個百分點、多數低於 0.3（表 3.1）。

依此結果，本論文對 RC 之定位重新框定：RC 並非一般性之準確率提升機制，而是於 1W1A 具顯著較大實測效果之校正項；由於其不增加額外運算或資料路徑成本（儲存與組態容量計入 adapter 預算），部署用之 bitstream 中將 RC 無條件納入。

3.4 多分支 Adapter 擴展

3.4.1 架構

多分支 Adapter 擴展為本論文將超低位元遷移準確率提升至 Conv-Adapter 單分支基線 [22] 之上的關鍵機制。將單分支擴展為每層 M 組平行分支，直接針對本論文實驗觀察到的現象——單分支 Adapter 之貢獻專於 1-bit 顯著下滑（表 3.1）——做出對應。各分支具獨立權重與 RC 偏置。對每個輸出通道 o , M 組分支輸出以可學習縮放因子合併：

$$y_o = \sum_{i=1}^M \alpha_i z_o^{(i)}, \quad o = 1, \dots, C_{\text{out}}, \quad (3.8)$$

其中：

- y_o ：通道 o 之多分支合併 Adapter 輸出。
- M ：平行 Adapter 分支數（ $M=1$ 至 $M=4$ ）。
- α_i ：第 i 分支之可訓練縮放因子，微調時一同訓練；部署時量化並吸收至貢獻 LUT。
- $z_o^{(i)}$ ：第 i 分支於通道 o 之輸出，以第 3.2 節之 down-projection、二值啟動、up-projection 管線計算。

訓練方式 所有任務相依參數—— M 條分支與分類器——在主幹凍結下聯合端到端訓練。各分支 down/up 投影權重於前向二值化：

$$W^{\text{down},(i)} = \text{sign}(\widetilde{W}^{\text{down},(i)}), \quad W^{\text{up},(i)} = \text{sign}(\widetilde{W}^{\text{up},(i)}), \quad (3.9)$$

反向採 straight-through estimator

$$\frac{\partial \text{sign}(u)}{\partial u} \approx \mathbf{1}_{|u| \leq 1}, \quad (3.10)$$

故 ADAM 更新的是實數潛在權重 $\widetilde{W}$ 與實數 down-conv 偏置 $b^{(i)}$ (即 RC 項)。分支縮放 α_i 為實數純量、線性作用於分支輸出, 梯度為精確值

$$\frac{\partial L}{\partial \alpha_i} = \left\langle \frac{\partial L}{\partial y}, z^{(i)} \right\rangle, \quad (3.11)$$

不經任何估計器。公式本身不強迫分支相異: 各分支結構相同, 僅因獨立隨機初始化與聯合訓練而分化。匯出時 α_i 量化入 Q8 貢獻 LUT、 $b^{(i)}$ 折入累加器初始值 (第 3.3 節)。

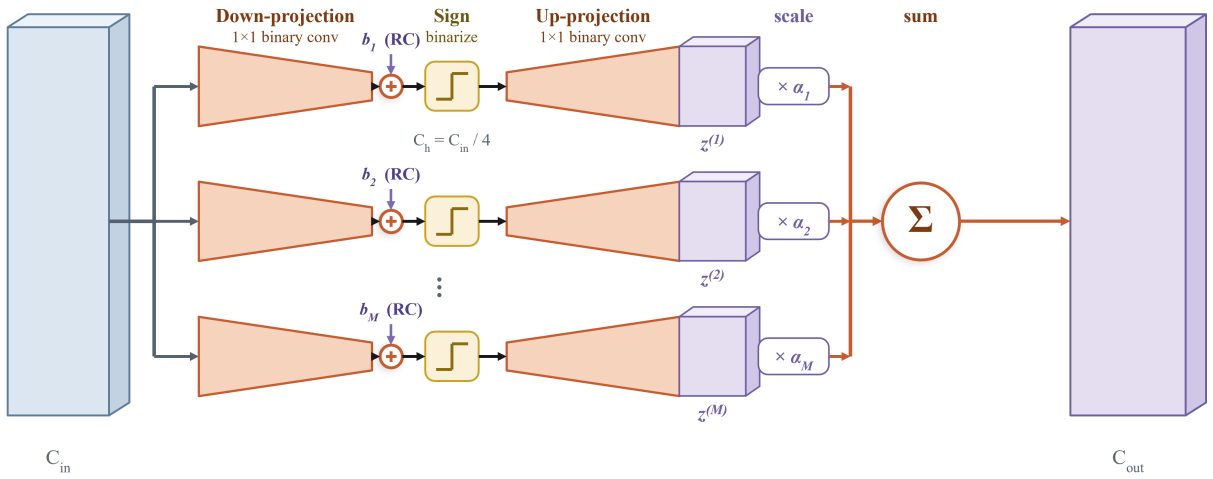


Figure 3.3: 多分支 adapter: M 條平行二值分支於閾值化前相加。

總參數開銷隨 M 線性增長:

$$\Delta P(M) = M \times 58,533, \quad (3.12)$$

即 $\Delta P/P_{\text{backbone}} \approx 3.78\% \times M$ 。配置 $M \in \{1, 2, 3, 4\}$ 。各 M 之參數量與 post-synthesis 資源報告及功耗估計量化於表 5.16。

3.4.2 理論運算成本

主幹與 Adapter 分支之運算成本分析如下。主幹含六個 3×3 卷積層與三個全連接層:

Table 3.4: 主幹運算成本分解 (CIFAR-10, $32 \times 32 \times 3$ 輸入)

層	輸出形狀	MAC 數
Conv1	$30 \times 30 \times 64$	1,555,200
Conv2	$28 \times 28 \times 64$	28,901,376
Conv3	$12 \times 12 \times 128$	10,616,832
Conv4	$10 \times 10 \times 128$	14,745,600
Conv5	$3 \times 3 \times 256$	2,654,208
Conv6	$1 \times 1 \times 256$	589,824
FC1-FC3	512, 512, 10	398,336
合計		59.46 M MACs

每個 Adapter 分支於五層各執行 1×1 down/up projection。此處須區分兩種運算量：軟體參考實作於各宿主層之完整輸入特徵圖上計算兩投影、再裁邊對齊卷積輸出 (compute-then-crop)，每分支實際執行 $4,227,072 \approx 4.23$ M MACs——此數亦作為第 5.5 節跨平台正規化中之每分支 adapter 項；部署 RTL 藉 centre-selection 僅於輸出位置計算，每分支實際執行 $3,010,560 \approx 3.01$ M MACs。跨平台核算統一採用之模型層級運算式為：

$$\text{OPs}_{\text{total}} = 2 \times (59.46 + 4.23 \times M) \text{ MOPs/image}, \quad (3.13)$$

其中定義 $1 \text{ MAC} = 2 \text{ OPs}$ 。 $M = 1$ 時，總計 127.38 MOPs/image。

3.4.3 透過分支擴展恢復準確率

多分支 Adapter 擴展旨在漸進恢復 Adapter 因超低位元量化而損失之準確率：增加 M 讓各分支以前述線性資源成本學習不同校正子空間。預期增加 M 可提升平均遷移準確率、平均增益漸進遞減——因共享瓶頸維度限制整體資訊容量，與額外分支數無關——惟個別遷移軌跡受最佳化變異影響，未必單調。量化此恢復之分支掃描實驗——涵蓋全部二十組 backbone-target 配對與兩種幾何——見第 5 章（第 5.3.3 與 5.3.7 節）。

Chapter 4

MARS NPU 架構設計

本章描述 MARS 加速器之 RTL 層級硬體設計。第 4.1 節介紹系統架構; 第 4.2 節詳述整合 Adapter IP 至 MVAU 之 Super Wrapper; 第 4.3 節介紹 Adapter IP 內部管線; 第 4.4 節說明 Stream Adder + Threshold 模組; 第 5.1 節勾勒端到端設計流程。

4.1 系統架構概觀

MARS 加速器以一個整合 micro-processor 子系統與可程式邏輯 (PL) 之 SoC FPGA 為目標。系統執行頻率為 100 MHz。所用之具體裝置與開發板於第 5 章 (實驗) 說明。

圖 4.1 為頂層架構。micro-processor 子系統透過晶片外記憶體負責系統控制、資料搬運與推論流程管理。PL 實作串流資料流推論管線, 含以下元件:

- **Input/Output DMA** : micro-processor 子系統與 PL 間資料傳輸之串流介面。
- **MVAU0** : 未修改之 FINN 產生 MVAU, 用於第一卷積層 (Conv1)。
- **Super1–Super5** : 五組強化 Adapter 之 MVAU 模組, 為自訂 RTL, 承載 Conv2–Conv6 層。
- **MVAU6–MVAU8** : FINN 產生之全連接層 (FC1–FC3; MVAU8 即分類器)。
- **MaxPool $\times$ 2** : 卷積階段間之池化模組。
- **LabelSelect** : 最終分類輸出。

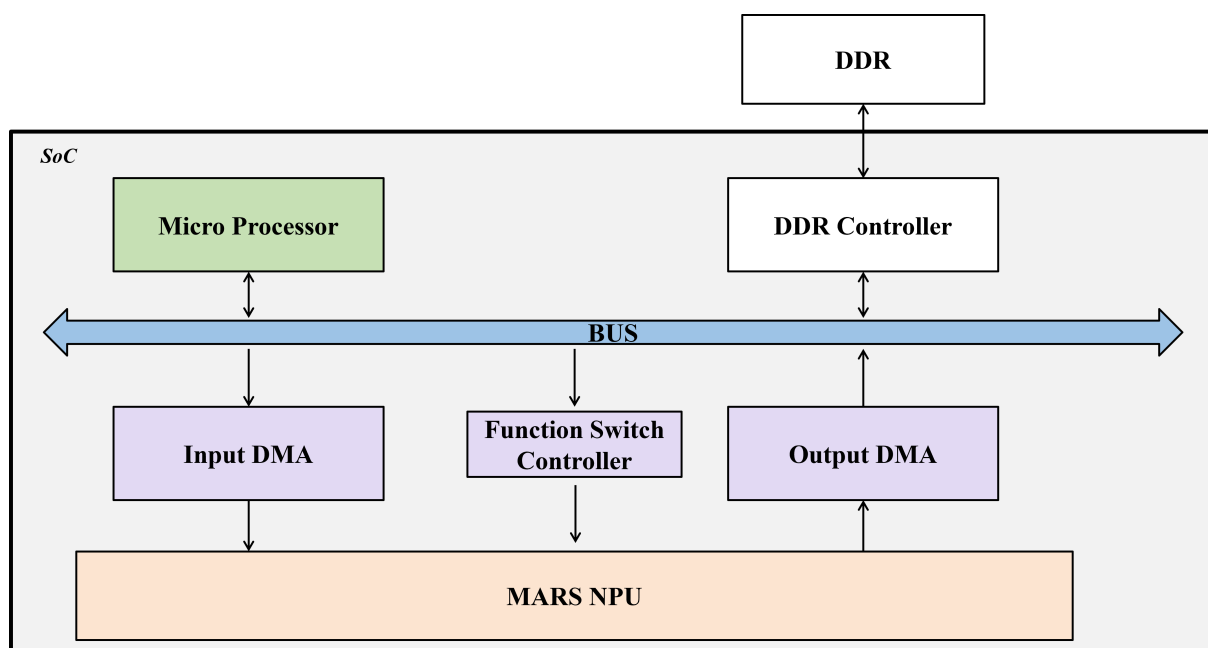


Figure 4.1: MARS 加速器之系統頂層架構。

作為對照，圖 4.2 顯示 FINN 為相同 1W1A VGG-like 主幹直接生成、未經 Adapter 修改之串流管線。Conv1–Conv6 各由一組 FINN MVAU 模組以序列方式串接；第 4.2 節之 Adapter Super Wrapper 即在此管線之基礎上擴展。

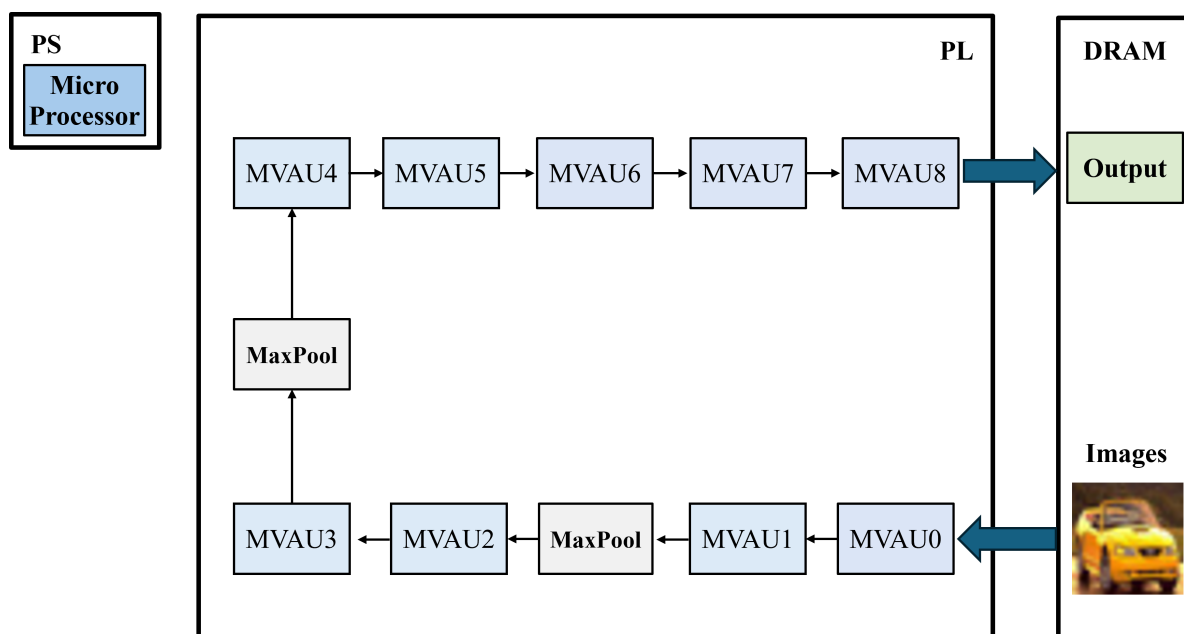


Figure 4.2: FINN 生成、未經 Adapter 修改之串流推論管線。

串流資料流管線以兩列結構佈局: 第一列為 Input DMA → MVAU0 → Super1–Super3 → MaxPool; 第二列為 Super4–Super5 → MVAU6–MVAU8 → Output DMA。模組間連接皆使用串流協定。

圖 4.3 提供完整管線的詳細視圖, 區分 FINN 產生的模組 (藍色) 與自訂 Adapter 強化 Super Wrapper (橘色), 並附 Super Wrapper 內部結構展開。

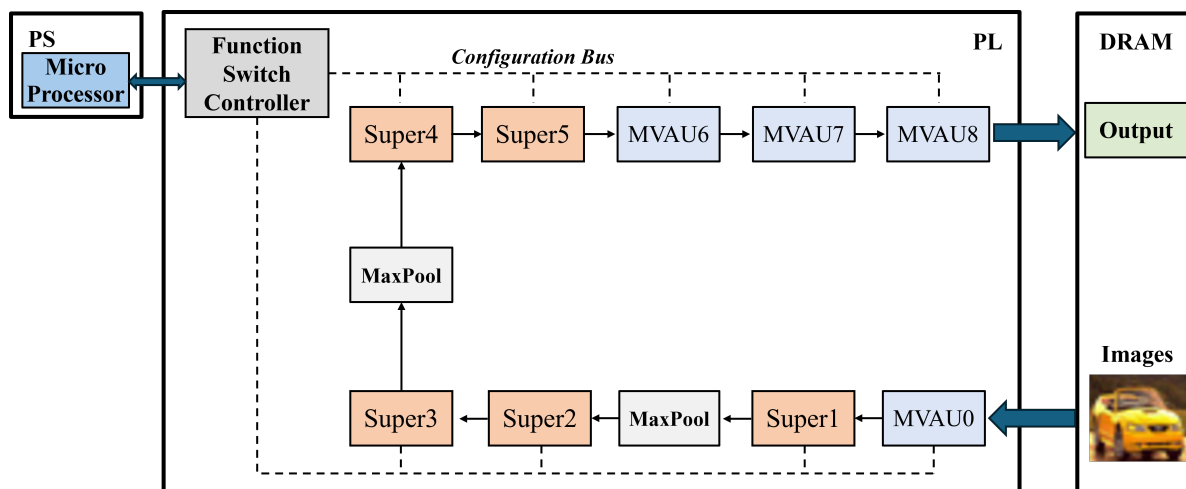


Figure 4.3: MARS 串流資料流管線。

模組來源：自寫 RTL 與 FINN 產生。 MARS 之部署 bitstream 結合兩類來源：由 FINN 資料流編譯器 (第 2.1 節) 產生之主幹類模組，與本論文針對整合及執行時期切換邏輯所自寫之 RTL。表 4.1 列出串流資料流分區內每個模組之來源，方便讀者將本章系統圖中各區塊對應至原始檔案。

Table 4.1: MARS 部署串流資料流分區內各模組之來源。

模組	來源	角色
MVAU_hls_0/6/7/8	FINN 產生	MVAU0（第一卷積）與 MVAU6/7/8（全連接+分類器）之矩陣-向量-啓動單元；未掛載 adapter
ConvolutionInputGenerator_rtl_0..5	FINN 產生	滑動視窗影像區塊產生器
StreamingDataWidthConverter_rtl_0..9	FINN 產生	串流寬度轉換
StreamingFIFO_rtl_0..6	FINN 產生	主幹層間 FIFO
StreamingMaxPool_hls_0..1	FINN 產生	主幹 max-pool
Thresholding_rtl_0	FINN 產生	主幹閾值單元
LabelSelect_hls_0	FINN 產生	最終 label arg-max
IODMA_hls_0	FINN 產生	輸入／輸出 DMA
MVAU_hls_1..5	FINN 產生、手動修改	包入 Super1-5 以承載 adapter 之 MVAU 核心（Conv2-Conv6 層）；模組內閾值已解耦，並將原始 32-bit 部分和對外暴露於輸出串流，使 adapter 貢獻得以在二值化前融合
Adapter_MVAU{1..5}.v	自寫 RTL	Adapter 資料路徑（down/up/RC/popcount/XNOR）
Stream_Adder_Threshold_mvau{1..5}.v	自寫 RTL	Adapter 貢獻融合 + Q8 閾值
Simple_FIFO_mvau{1..5}.v	自寫 RTL	每 Adapter 同步 FIFO（深度 64）
Stream_Splitter_mvau{1..5}.v	自寫 RTL	主幹／Adapter 串流分流
MVAU{1..5}_Super_Wrapper.v	自寫 RTL	整合修改後 MVAU 核心與 adapter 之每層 Super Wrapper
adapter_cfg_hub.v	自寫 RTL	runtime 配置中樞（19 LUT；位址解碼 + 寫入分支）
cls_cfg_bridge.v	自寫 RTL	cfg→memstream 橋（分類器權重）
MVAU_hls_0/6/7/8_imp 包裝	自寫 RTL	FINN 所產 MVAU0/MVAU6/MVAU7/MVAU8 之 cfg 可寫包裝

簡言之，主幹 XNOR/popcount/閾值/DMA 資料路徑為 FINN 所產（表上半段），唯五個承載 adapter 之卷積 MVAU 核心（MVAU_hls_1..5）係 FINN 產生後再手動修改以對外暴露原始部分和（表中段）；所有 Adapter、Super Wrapper、cfg-hub、cfg-bridge 及 cfg 可寫包裝皆為本論文之自寫 RTL（表下半段）。所有自寫 RTL 位於專案之 adapter-IP 原始碼目錄。

4.2 MVAU + Adapter Super Wrapper

在介紹 MARS 如何整合 Adapter 之前，先簡要回顧 Super Wrapper 所替換之原始 FINN MVAU 結構。圖 4.4 為未修改之 FINN MVAU 內部管線：串流輸入經 Convolution Input Generator 切片，與 1-bit 權重 RAM 進行 XNOR、popcount 累加，最後於模組內部與通道閾值 RAM 比較產生單一二值輸出位元。關鍵在於：閾值化與二值化發生於 MVAU 內部——partial-sum 訊號未對外暴露，故任何在 MVAU 之後想加入之修正項（例如 Adapter 貢獻）皆無法於外部融合，因為二值化已將累加器塌縮為一個 bit。

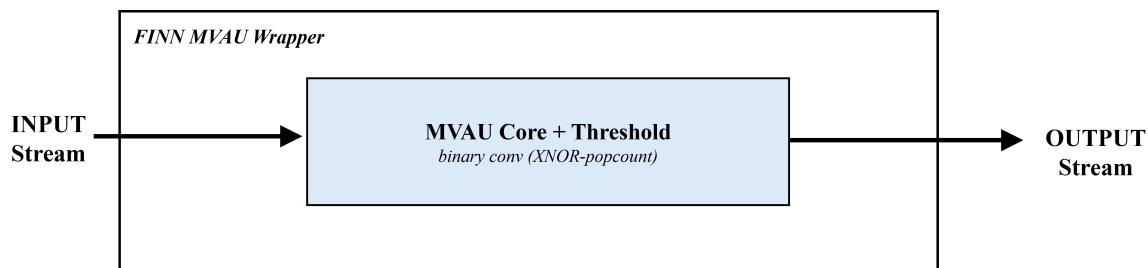


Figure 4.4: 未修改之 FINN MVAU 內部架構。

Super Wrapper 自兩個面向重構此管線。其一，將閾值步驟從 MVAU 核心解耦：圖 4.5 中之 Modified MVAU Core 於累加器處終止，將原始 32-bit partial sum 暴露於輸出串流。其二，新增 Adapter Core 為並行旁路，擁有獨立之 down/up projection RAM；其 8-bit 貢獻先與 MVAU partial sum 相加，再由外部之 Stream Adder + Threshold 模組進行閾值比較與二值化。此一解耦是後置 Adapter 之所以能於串流二值化管線上實現之關鍵。

圖 4.5 為 Super Wrapper 之內部架構。

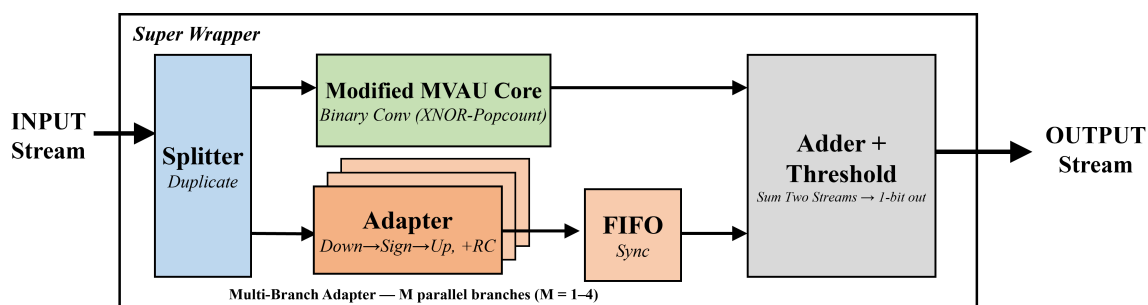


Figure 4.5: MVAU + Adapter Super Wrapper。

Super Wrapper 運作如下：

1. **Stream Splitter**：以組合邏輯將輸入串流複製至兩路。兩路輸出皆須 ready 才能接受輸入：

$$s_axis_tready = m_axis_1_tready \text{ AND } m_axis_2_tready. \quad (4.1)$$

2. **Path A — MVAU Core**：FINN 原產 MVAU 核心經修改後，輸出原始整數部分和（partial sum），不再直接閾值二值化。內部閾值／二值化邏輯被解耦移至 Stream Adder + Threshold 模組，使 Adapter 貢獻可於最終二值決策前融合進 MVAU 結果。
3. **Path B — Adapter Core**：以專用 Adapter IP（第 4.3 節）執行 Adapter 前向運算（演算法 3.1）。

4. **Simple FIFO**：深度 $2^6 = 64$ 的輕量同步 FIFO, 提供 MVAU 與 Adapter 兩路間之相位對齊; 吸收內部管線深度差, 確保融合點之週期精準同步。
5. **Stream Adder + Threshold**：將 MVAU 部分和與 Adapter 貢獻融合, 並進行最終閾值比較以產生二值輸出（第 4.4 節）。

4.3 Adapter IP 內部架構

圖 4.6 為 Adapter IP：一個 Control Unit、cfg 可寫之 RAM 記憶體（ram_down、ram_rc、ram_up 與貢獻 LUT），以及執行 down-projection（XNOR-popcount）、RC 初始化累加、二值 sign、up-projection（XNOR-popcount）與 α 縮放之串流資料路徑。其中 down-projection 路徑本身為四級管線（S0–S3）、up-projection 為二級輸出管線, 詳見下文。

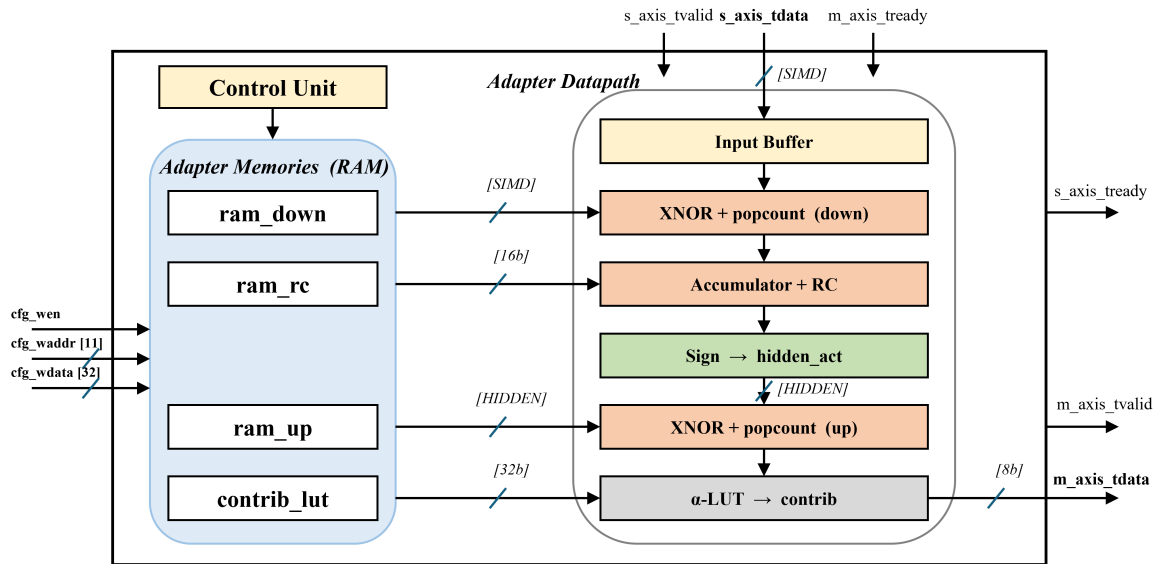


Figure 4.6: Adapter IP 內部架構 (單一支; 匯流排標註為位元寬度)。

4.3.1 Down-Projection 管線

Down-projection 路徑處理每個 3×3 視窗之中心像素, 共四級管線：

- **S0（輸入鎖存 + RAM 讀取）**：同時鎖存輸入啟動並從 ram_down 讀取全部 C_h 個隱藏通道之 down-projection 權重。
- **S1（XNOR）**：對每個隱藏通道 j 計算 $\text{XNOR}(s1_act, s1_down[j])$ 。

- **S2 (Popcount)**：以自 RTL popcount 運算式推論之組合加法樹, 計數每個 XNOR 結果的 set bits。
- **S3 (累加)**：將 popcount 結果加至累加器 (其已以 RC 偏置初始化)。

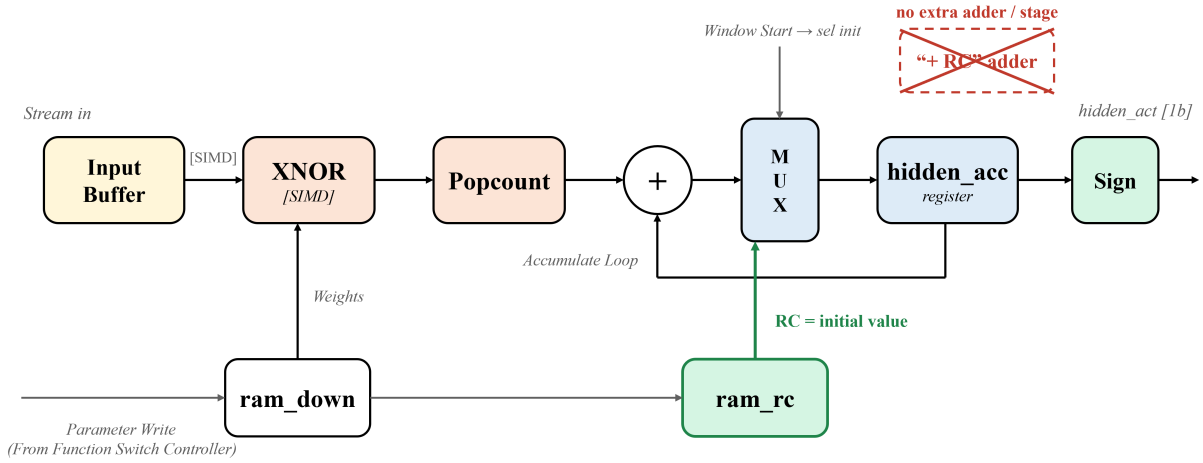


Figure 4.7: RC 之硬體實現: 累加器初始值由 ram_rc 載入 (無額外加法器或管線級)。

圖 4.7 細示 RC 進入此管線之方式: 自 ram_rc 讀出之逐通道 RC 字接至 hidden_acc 暫存器前多工器之初始化輸入——每個視窗起始時 FSM 選擇此輸入, 將 RC 偏置取代常數零載為累加器初始值; 其餘週期多工器選擇累加迴路, 逐次加入 popcount 結果 (S3 級)。由於暫存器本就須於每個輸出視窗載入初始值, 此校正不引入額外加法器、DSP 或管線級——圖中打叉之「+RC」級於 RTL 中並不存在。新增硬體僅 ram_rc 儲存與初始化選擇控制, 皆計入 adapter 開銷 (第 3.3 節), 且皆可經 Function Switch Controller 於執行時期寫入 (第 4.5 節)。

4.3.2 Up-Projection 路徑

所有輸入 chunk 處理完畢後, 二值化之隱藏啟動向量 (hidden_act) 驅動 up-projection:

1. Up-projection 權重打包為寬 RAM 字。以 PE = 4 為例, 四個 32-bit 權重向量串接為單一 128-bit 字, 支援單埠 RAM 存取:

$$\text{ram_up}[s] = \{\mathbf{w}_{\text{up}}^{(s,3)}, \mathbf{w}_{\text{up}}^{(s,2)}, \mathbf{w}_{\text{up}}^{(s,1)}, \mathbf{w}_{\text{up}}^{(s,0)}\}. \quad (4.2)$$

2. 計算 hidden_act 與各 PE 權重切片之 XNOR。

3. Popcount 產生每個 PE 之 8-bit 輸出。

4.3.3 記憶體組織

所有 Adapter 權重使用分散式 RAM (LUTRAM), 以 Verilog 合成屬性 (* ram_style = "distributed" *) 標示。此設計選擇：

- 提供單週期讀取延遲, 避免 BRAM 管線開銷。
- 避免消耗 BRAM, 保留給主幹 MVAU 權重儲存使用。
- 投影權重為 1-bit, 每分支預算維持 58,533 個參數 (混合精度: 1-bit down/up 權重加 Int8 RC 偏置與分支縮放 α)——此為參數個數, 非位元數。

具體 RAM 配置整理於表 4.2。

Table 4.2: Adapter RAM 配置 (範例: Super4, PE= 4)

RAM	深度	寬度	內容
ram_down	$C_h \times K = 128$	32 bits	Down-projection 權重
ram_up	$C_{out}/P = 64$	$P \times 32 = 128$ bits	打包之 up-projection 權重
ram_rc	$C_h = 32$	16 bits	RC 偏置 (有符號)
adp_contrib_lut	$2^8 = 256$	32 bits (有符號)	Q8 縮放後 adapter 貢獻 (α 已吸收)

Int8 RC 偏置於匯出時 sign-extend 存入 16-bit ram_rc 字; 較寬之 RAM 字為配合累加器介面之版面填充, 非額外數值精度。

4.4 Stream Adder + Threshold 模組

Stream Adder + Threshold 模組將 MVAU 部分和與 Adapter 貢獻融合並產生二值輸出, 以預計算查表實現零 DSP 使用。

4.4.1 貢獻 LUT：離線預計算與執行時期可寫入之儲存

Adapter 輸出 (popcount 值 o_p) 在加至 MVAU 輸出之前必須先乘以逐通道帶號縮放因子 α 。若於執行時期執行此乘法, 每一輸出通道需消耗一個 DSP slice (或代價高昂

之 LUT 乘法器)。本論文以位元組查表方式完全避開乘法器：對每一無號位元組值 $i \in [0, 255]$ ，

$$\text{LUT}[i] = \left\lfloor \frac{(i - z) \times \alpha_{Q8} + 128}{256} \right\rfloor, \quad (4.3)$$

其中 z 為零點（popcount 範圍 $[0, C_h]$ 時等於 $C_h/2$ ）， α_{Q8} 為 Q8 量化縮放。runtime-switching bitstream 採每層雙貢獻 LUT——adp_contrib_lut_b0 對應封裝後 adapter 串流之低位元組，adp_contrib_lut_b1 對應高位元組——使每一週期可平行查詢兩組符號/縮放值。

為支援任務切換但無須重新合成，LUT 並未燒入為合成期常數：由部署期軟體（已知 α 、 z 與每任務之縮放符號）離線預計算，載入分散式 RAM 表格，且此表格可經 cfg_hub 於執行時期重寫（第 4.5 節）——切換任務即重寫 LUT 內容，不需重建 bitstream。兩張 256-深表格完全以 LUT-RAM 實現，不耗 BRAM 與 DSP；每通道之符號選擇存於另一張同樣可於執行時期重寫之 RAM，Super5 與其他層相反之 BatchNorm 極性亦由其實現，無需任何專屬邏輯。

整體而言，部署版 bitstream 中之 LUT 既非嚴格合成時常數、亦非完全執行時期計算之值，而為「合成時預設、執行時可重寫」之表格。預設內容對應部署任務之 Eq. (4.3)；任務切換僅需透過 cfg_hub 覆寫該 512 筆項目即可。

4.4.2 二級管線

此模組以二級管線運作：

Stage 0（LUT 查表 + 輸入擷取）：當 MVAU 與 Adapter 輸出皆有效時：

$$\text{contrib}_p = \text{LUT}[\text{s_axis_2_tdata}[(p \times 8) + :8]], \quad p = 0, \dots, P - 1. \quad (4.4)$$

Stage 1（加法 + 閾值）：計算總啟動值，並與每 PE 之閾值比較：

$$\text{total}_p = \text{mvau_pop}_p + \text{contrib}_p, \quad (4.5)$$

$$\text{out}_p = \begin{cases} 1 & \text{若 } \text{total}_p \geq \text{thresh}_p[s], \\ 0 & \text{否則,} \end{cases} \quad (4.6)$$

其中 $\text{thresh}_p[s]$ 為 PE p 於輸出步 s 之閾值，儲於每個 PE 各自之閾值 RAM。

演算法 4.1 整理完整融合流程。

Algorithm 4.1 Stream Adder + Threshold 融合

Require: MVAU 輸出 $\mathbf{m} \in \mathbb{Z}^P$; Adapter popcount $\mathbf{o} \in [0, C_h]^P$; 閾值 RAM; 貢獻 LUT

Ensure: 二值輸出 $\mathbf{b} \in \{0, 1\}^P$

```
1: // Stage 0 : LUT 查表
2: for  $p = 0$  to  $P - 1$  do
3:    $\text{contrib}_p \leftarrow \text{LUT}[o_p]$  ▷ 離線預計算、執行時可重寫
4: end for
5: // Stage 1 : 加法 + 閾值
6: for  $p = 0$  to  $P - 1$  do
7:    $\text{total}_p \leftarrow m_p + \text{contrib}_p$ 
8:    $b_p \leftarrow \mathbf{1}[\text{total}_p \geq \text{thresh}_p[\text{step}]]$ 
9: end for
10: return  $\mathbf{b}$ 
```

4.5 執行時期配置中樞 (cfg_hub)

為使單一編譯 bitstream 能於執行時期服務多個下游任務, MARS 於串流資料流分區內插入輕量配置中樞。可行性建立在一個簡單觀察: 當下游任務共用同一凍結 1W1A VGG-like 主幹時, 二值卷積與 FC 權重相同 (烘入電路), 僅 BatchNorm 融合閾值、分類器輸出投影權重與五組 Adapter 區塊 (enable、down/up 投影 RAM、RC 偏置、閾值、符號、貢獻 LUT) 不同。因此單一 .bit 檔即可服務任意數量之合格任務, 每任務狀態化約為一份 25,088 bytes (約 25 KB) 之主機端 blob, 以 6,757 筆 32-bit cfg-hub 字寫入傳輸 (匯流排 payload 為 27,028 bytes; 窄於 32-bit 之欄位如 byte-packed 分類器權重, 寫入時擴為完整字)。中樞佔用 64 KB 位址空間, 基址 0x40010000——此為系統記憶體映射中可定址視窗之大小, 而非每次傳輸之資料量; 每任務實際僅寫入上述約 25 KB blob, 稀疏分佈於此視窗內。此 64 KB 之 16-bit byte 位址拆解為 3-bit 單元選擇 (byte_addr[15:13]) 與 11-bit 字位址 (byte_addr[12:2])。圖 4.8 展示系統層整合: micro-processor 寫入中樞之控制介面, 由組合邏輯解多工——3-bit 單元選擇, 其中 unit 0 再分高低兩半——至分散於管線各處之九個實體目標 (八個位址單元、九個目的地); 資料路徑完全不被擾動, 整套切換基礎架構與串流資料流正交。

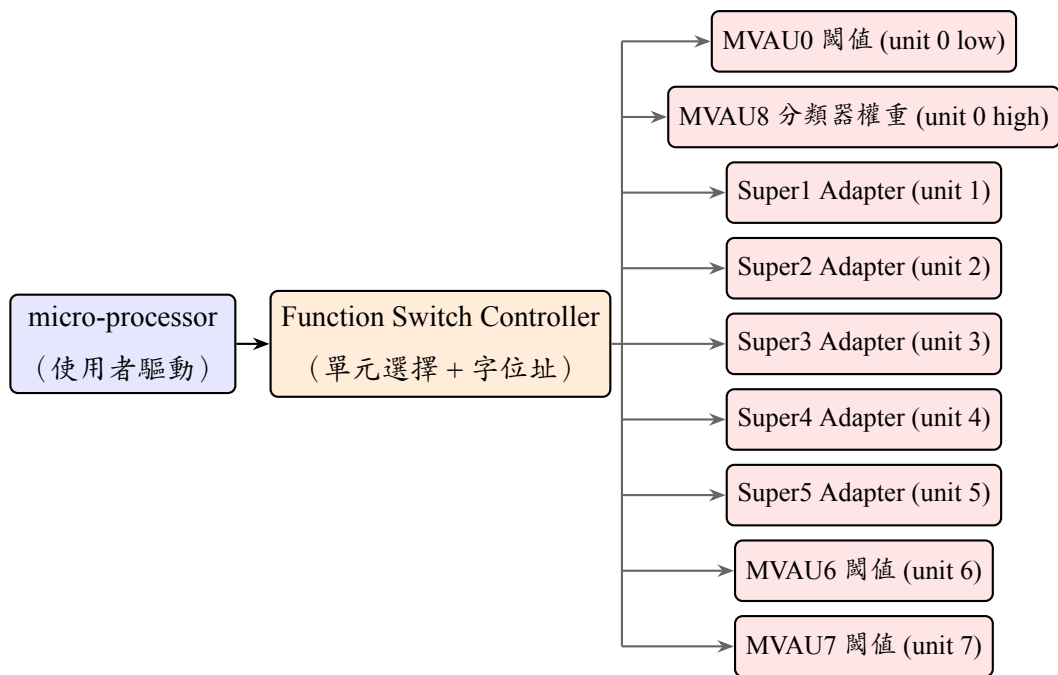


Figure 4.8: Function Switch Controller 之系統層整合。

圖 4.9 為 Function Switch Controller 之內部電路。AW latch 鎖存 16-bit AXI-Lite 寫入位址, Write FSM 追蹤 AWVALID/WVALID 交握, 待位址與資料皆到齊即發出寫入分派並回覆 BRESP。鎖存位址以組合邏輯拆解為單元解碼 (a[15:13])、unit-0 分割 (a[12]) 與字位址切片 (a[12:2]); 1-of-9 寫致能解多工器將解碼結果轉為單週期 WEN 脈衝, 共用組態匯流排 (waddr·wdata) 則廣播至九個目標 (MVAU0 閾值、MVAU8 分類器權重、Super1-5 adapter、MVAU6/MVAU7 閾值)——僅 WEN 有效之單元寫入生效。

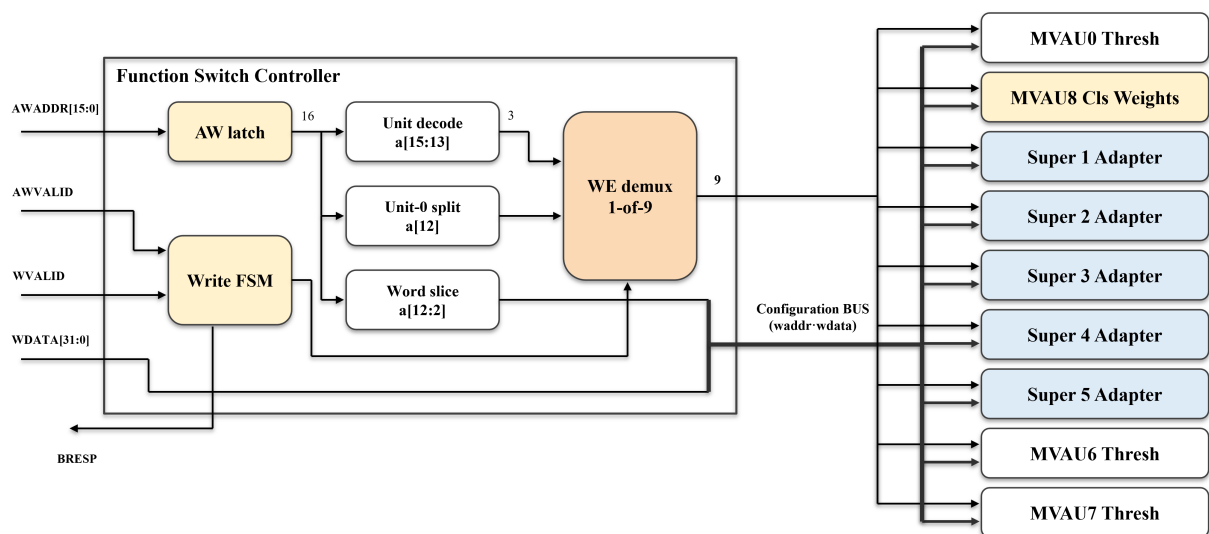


Figure 4.9: Function Switch Controller 之內部電路。

4.5.1 位址映射

圖 4.10 以視覺化方式呈現 64 KB 位址空間。每個 8 KB 插槽承載一組 MVAU 模組之 Adapter 暫存器（enable 在 word 0、Q8 閾值在 word 1152、sign bit 在 word 1408），或一組每任務參數區塊（MVAU0 閾值、MVAU8 分類器權重、MVAU6/MVAU7 閾值）。

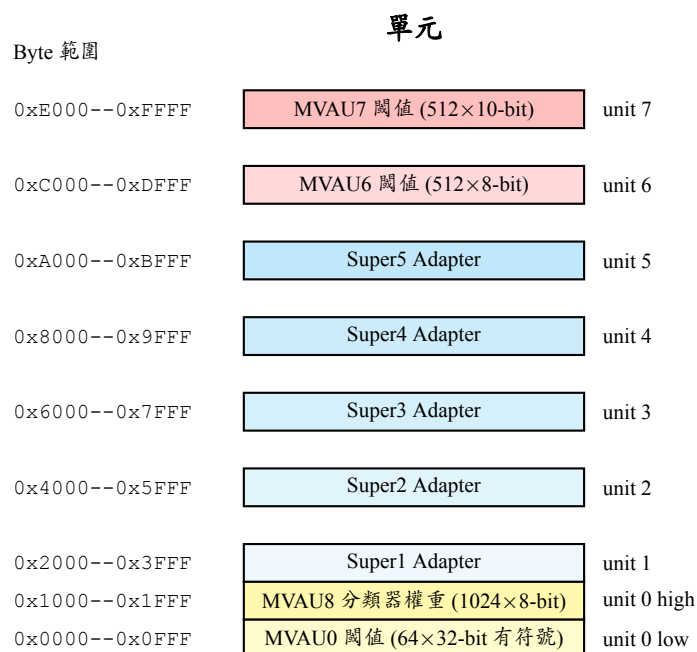


Figure 4.10: cfg_hub 64 KB 位址映射。

4.5.2 執行時期切換流程

圖 4.11 整理端到端切換流程。切換之前提條件為 Output DMA done bit 拉起, 確保前一批推論已自串流管線排空（實測驗證詳見第 5.6 節）。使用者驅動接著以 MMIO 批次寫入任務特定參數 blob（6,757 字, 重寫約 1.86 ms; 切換後正確性另行驗證），之後即可立即啟動下一批, 不需顯式 flush 管線。使用者空間驅動於啟動時將 cfg_hub 位址空間映射一次, 並將 6,757 個字之寫入合併為少量連續之整批 MMIO 傳輸, 而非逐字呼叫; 將此每字軟體開銷降至最低, 正是達成第 5.6 節所報 1.86 ms 組態重寫延遲的關鍵。

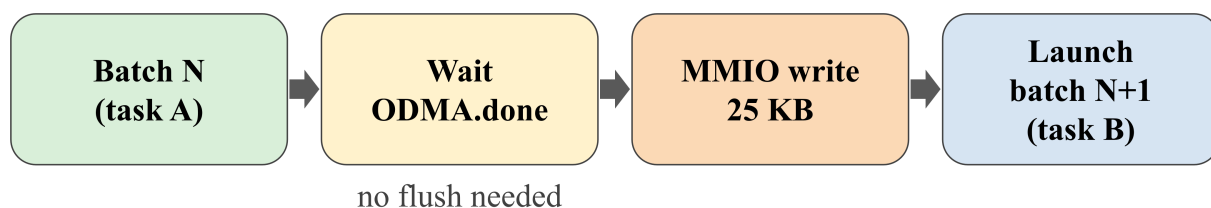


Figure 4.11: 端到端執行時期任務切換序列。

4.5.3 RTL 修改

以下三項 RTL 修改在不擾動串流資料流的前提下實現上述流程：

- **MVAU0**：原本寫死的 16 組閾值多工器改為 16 個分散式 RAM bank, 各 bank 對 `cfg_hub` 暴露寫入埠。
- **Super1–5 Stream Adder + Threshold**：Q8 閾值 RAM 與符號極性位元改為 `cfg` 可寫之分散式 RAM, 並於 word 0 新增 `adapter_enable` 暫存器開控 Adapter 貢獻; 停用時 Super1–5 回歸純主幹行為（供 CIFAR-10 模式使用）。
- **分類器 memstream (MVAU8)**：於 `_imp` 包裝內加入 `cfg→memstream` 狀態機驅動晶片內 `memstream`, 支援 CIFAR-10 與 SVHN 間差異之輸出投影權重執行時期重載。

因共用主幹一次性凍結、持久性每任務 blob 存於晶片外並於切換時複製至 `cfg` 可寫之晶片內記憶體, MARS 之部署 fabric 成本與部署任務數無關——單一 bitstream 即服務全部任務 ($O(1)$), 且 fabric floorplan 不變。前人任務切換方案則付出每任務或每變體之部署成本, 但各類形式不同: 部分重組態設計每任務需多一份 partial bitstream (`.bit`) 並使 fabric floorplan 更複雜 (可重組態區), 而非晶片內記憶體變多; 晶片內模型選擇設計則將固定之合成時變體集合常駐晶片 (每次多工選定變體之參數)。圖 4.12 以示意方式呈現此對比。

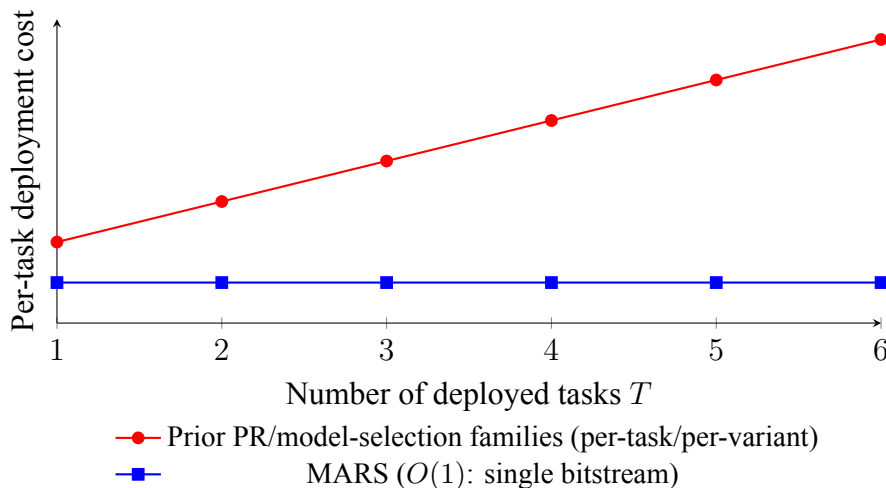


Figure 4.12: 部署成本對任務／變體數之示意: MARS $O(1)$ 晶片內成本 vs 前人每任務／每變體成本 (量化軸列於表 5.25–5.27)。

4.6 MVAU + Adapter 整合系統之管線化

主幹 MVAU 之管線化與逐層 PE/SIMD 折疊皆遵循標準 FINN 串流資料流方案 [9,10], 兩者均非本論文之貢獻。MARS 特有之處在於: 將 adapter 側流整合進此已折疊之資料路徑而不破壞它——Adapter IP 與 Stream Adder + Threshold 模組本身為管線化設計 (第 4.3–4.4 節), 並經逐層 FIFO 與外露之 MVAU 部分和串流做週期同步, 使整合系統於 FINN 所選折疊因子下仍在 100 MHz 收斂時序, 且開啓 adapter 後吞吐量與純主幹相差在 $\approx 0.5\%$ 內 (第 5.5 節)。throughput 與 compactness build 之完整逐層折疊因子作為實測數據於第 5.4.3 節 (表 5.14) 報告。

4.7 驗證方法論

驗證策略採三層級階層式方法, 涵蓋逐模組 RTL 模擬、頂層串流資料流模擬與端到端板上驗證:

1. **模組層 RTL 模擬**: 各 Super Wrapper (Super1–Super5) 獨立測試, golden 向量由與硬體運算位元精確一致之軟體模型產生 (XNOR-popcount、累加、Q8 閾值比較、符號/極性處理)。golden 模型直接讀取與 FPGA RAM 初始化相同之硬體佈局權重映像, 而非自匯出前之軟體張量重新計算——以消除位元層級之表示法不一致假錯。主幹權重於各任務間完全相同, adapter 訓練從不更新它們。五個模組合計涵蓋 43,520 筆測試向量 (各層輸出範圍之逐通道輸出, 依表 5.14 之 PE 因子折疊), 每一模組均 100% 通過。
2. **頂層資料流模擬**: 完整串流管線——Input DMA、MVAU0、五組 Adapter 強化 Super Wrapper (Super1–5)、MaxPool、MVAU6–MVAU8、LabelSelect、Output DMA——作為單一 block design 模擬, 推入一批 SVHN 測試影像穿過整條資料流; 分類決策與 PyTorch golden 一致, 證實 Adapter 整合、FIFO 同步與 Stream Adder + Threshold 融合於系統層級與軟體行為一致。
3. **端到端板上驗證**: 產生之 bitstream 載入目標 FPGA, 以 Python 驗證驅動, batch size 1000 處理 10,000 張 SVHN 測試影像 (10 次平均)。本 throughput build 為吞吐量/能效量測配置: 端到端 1,866 img/s (驅動+FPGA 為 1,933 img/s), batch size 1000 下攤提之單張平均時間 0.54 ms。板上準確率正確性則由第 5.6.3 節之 compactness build 於正規 cifar10_1w1a 主幹驗證 (五個資料集板上與軟體相差 0.69 個百分點以內, 表 5.23); 同一 cfg_hub 機制亦於該 build 之 Adapter 停用模式驗證執行時期閾值重載不引入額外數值誤差。

Chapter 5

實驗與評估

本章自四個面向評估 MARS 加速器: 軟體端遷移學習準確率 (第 5.3 節)、FPGA 資源使用與功耗 (第 5.4 節)、跨平台效能比較 (第 5.5 節) 與任務數擴展分析 (第 5.7 節)。

5.1 端到端設計流程

圖 5.1 為 MARS 之端到端軟體到硬體流程。軟體端凍結共用之 1W1A VGG-like 主幹, 並針對每個任務 ($\#1 \dots \#N$) 訓練一組多分支 Adapter (含 RC); 模型於目標資料集上建立、訓練與評估。FINN compiler 隨後生成 Backbone IP, 部署至硬體後將自訂 Adapter IP 融合進主幹, 並由 Runtime Function Switch Controller 提供單一 bitstream 之任務切換。此流程細分為六階段:

1. **階段 1 —主幹預訓練**: VGG-like 主幹 (6 卷積 + 3 FC) 於 CIFAR-10 上以 PyTorch 進行 1W1A 量化感知訓練。權重量化至 $\{-1, +1\}$ (1-bit 有符號)、啟動量化至 $\{-1, +1\}$ (1-bit 有符號)、輸入為 8-bit (Q1.7 格式) [6,9,45]。
2. **階段 2 —Adapter 微調**: 凍結主幹權重, 僅於目標資料集 (SVHN 或 FashionMNIST) 上訓練 Adapter 參數 (down/up projection 權重、RC 偏置、縮放因子 α)。Checkpoint 依 BNN-PYNQ 原始 trainer 流程: 每輪訓練後評估並保留最高分之 epoch(best.tar); 此逐輪評估使用 test 集, 故所有報告之準確率為以 test 集選出之最佳 epoch 操作點。因所有配置 ($M=1-4$ 、有/無 RC、全部主幹與目標) 採同一規則, 比較在程序上一致; 惟以 test 集選最佳 epoch 仍可能同時使絕對準確率與相對差值產生偏差。絕對準確率應視為操作點上界而非 held-out 估計。

3. **階段 3 —權重匯出**：主幹權重以 FINN 交錯格式匯出;Adapter 權重轉為十六進制 .dat 檔供 RAM 初始化,up-projection 權重以 PE 對齊打包。
4. **階段 4 —FINN 主幹生成**：FINN 框架將量化主幹轉換為串流資料流加速器,產生帶串流介面之 HLS MVAU IP Core。
5. **階段 5 —RTL 整合**：於 RTL 區塊設計層將 FINN 產生之 MVAU 替換為 Super Wrapper;原 MVAU 之內部閾值邏輯解耦,並連接自訂 Adapter IP、Stream Splitter、FIFO 與 Stream Adder + Threshold 模組。
6. **階段 6 —FPGA 驗證**：將整合後設計合成、擺放與繞線;於 Python 建立 golden 模型,與 FPGA 輸出逐 bit 比對。各模組資料路徑為位元精確 (43,520 筆向量,100% 通過);於完整 10,000 張影像之端到端層級,板上結果與軟體參考相差在 0.69 pp 以內 (表 5.23),微小殘差與 Q8 閾值捨入之累積一致,而非 bit 層級錯誤。

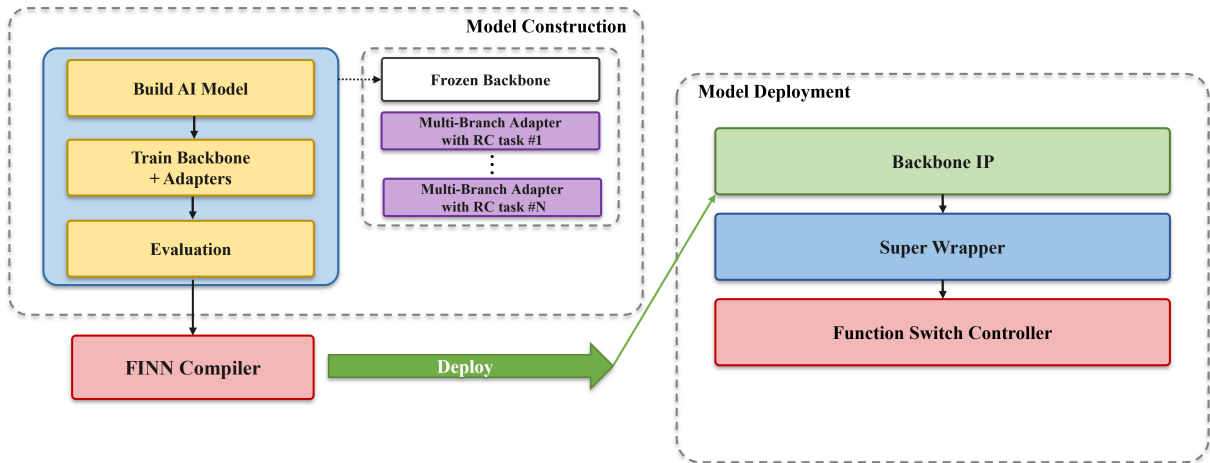


Figure 5.1: MARS 端到端軟體到硬體流程。

5.2 實驗設定

5.2.1 資料集與任務

本論文使用五個 32×32 等級之影像分類資料集，彙整於表 5.1。每個資料集皆可作為預訓練後凍結之 *backbone* (來源)，或作為被適應之下游 *target*。其涵蓋刻意不同之視覺領域：一般自然物件 (CIFAR-10、STL10、CINIC10)、街景數字 (SVHN) 與灰階服飾 (FashionMNIST)，使遷移同時涵蓋領域內與跨領域。

Table 5.1: 本論文使用之資料集（皆調整為 $32 \times 32 \times 3$ 輸入）。

資料集	類別	訓練／測試	原生解析度	型態	領域／備註
CIFAR-10	10	50,000 / 10,000	32×32	RGB	一般自然物件（預設 backbone）
SVHN	10	73,257 / 26,032	32×32	RGB	街景門牌數字
STL10	10	5,000 / 8,000	96×96	RGB	ImageNet 衍生物件（降採樣）
FashionMNIST	10	60,000 / 10,000	28×28	灰階	服飾（縮放、複製通道）
CINIC10	10	90,000 / 90,000	32×32	RGB	CIFAR-10 + 下採樣 ImageNet；總計 270k ($90k \times 3$)

FashionMNIST 由 28×28 灰階縮放並複製通道為 $32 \times 32 \times 3$ ，STL10 由 96×96 降採樣至 32×32 ，使每個資料集皆吻合 VGG-like 輸入幾何；CINIC10 作為大規模泛化驗證。硬體側上板部署全部五組任務專屬模型（各對應一資料集：CIFAR-10、SVHN、FashionMNIST、STL10、CINIC10）作為 runtime 任務切換驗證（單一 bitstream 共用一凍結 1W1A 主幹，第 5.6.3 節）；此五個目標亦於軟體完整評估（bit-width 掃描與 RC 跨資料集驗證，第 5.3.5 節）。

5.2.2 平台與工具

- **訓練**：PyTorch + Brevitas, 以 1W1A 量化感知訓練。
- **FPGA**：PYNQ-Z2 (XC7Z020)、Vivado 2022.2、FINN v0.9、100 MHz 時脈。
- **GPU 基準**：NVIDIA RTX 4090 與 Jetson Orin NX 8GB, 用於跨平台比較。

5.2.3 量測指標

- **準確率**：目標測試集上之 Top-1 分類準確率。
- **吞吐量**：端到端 frames per second (img/s)。
- **GOPS**：giga-operations per second, $\text{GOPS} = \text{FPS} \times \text{OPs/image} / 10^9$ 。
- **能效**：img/s/W（端到端 FPS 除以晶片總功耗）。
- **資源使用**：XC7Z020 上之 LUT、FF、BRAM、DSP 使用量。

5.3 軟體端遷移學習結果

本節報告以軟體端評估 Adapter 遷移學習方法之結果。論述順序為：問題定義（1-bit 遷移之準確率落差，第 5.3.1 節）、機制分析（二值化下 down convolution 偏置項之角色，第 5.3.2 節）、準確率恢復（在最具表達力之 Adapter 配置下進行多分支擴展，第 5.3.3 節）、泛化驗證（三組額外 target dataset，第 5.3.5 節）、變異性量化（multi-seed 重跑，第 5.3.6 節），最後為部署配置及其 *FPGA-friendly* 成本（第 5.3.7 節）。除非另行說明，所有實驗皆採用：transfer 200 epochs、學習率 0.005、STEP scheduler milestones 為 100 與 150、batch size 100、ADAM 最佳化器、SqrHinge loss、隨機 seed 2024；主幹為以 CIFAR-10 預訓練 500 epochs 之 1W1A VGG-like 網路；除非另行指明，source-target 配對為 CIFAR-10 $\rightarrow$ SVHN。

基線政策。 本章中， $M=1$ 單分支 Conv-Adapter 瓶頸 [22]（搭配先前工作預設之 `bias=True`）為基線，於表格中標示為先前工作、於圖中標為 $M=1$ ；本論文相對此基線之演算法新增僅限於：(a) 多分支 Adapter 分支擴展 $M \in \{2, 3, 4\}$ （ $M=2-4$ ）；(b) 受控之 `bias` 切換 ablation——命名此項為 RC，並孤立其 1-bit 專屬效應；(c) FPGA 友善之 RC 整合為累加器 reset 值（第 3.3 節）。後續表格中報之 $M=1$ 數值為先前工作配置於本論文 1W1A 主幹之重現，不列為本論文之貢獻。

5.3.1 1-bit 遷移之準確率落差

本章所據之 1-bit 遷移準確率落差已於第 3.1 節（表 3.1）建立：單分支 Conv-Adapter [22] 於 ≥ 2 -bit 與 Full-FT 差距僅 $\approx 1-5$ pp，卻於 1-bit 仍明顯不足，而 Full-FT 仍維持有效。故本論文須回答「1-bit 下缺少了什麼」；本節後續以三步驟回應：對 down-convolution 偏置進行控制性 ablation（第 5.3.2 節）、於最具表達力之 Adapter 配置下做多分支擴展（第 5.3.3 節）、跨額外 target dataset 進行驗證（第 5.3.5 節）。

5.3.2 Down-convolution 偏置作為 1-bit 專屬之校正項（RC）

於 1W1A 量化下，down convolution 之輸出送入二值 sign 函數。若無可學習偏移項，sign 函數退化為「閾值固定為零」之硬決策，Adapter 無法逐通道調整此閾值。於 ≥ 2 -bit 啟動下，此偏移之實測效果很小（至多 1.4 pp，表 3.1），可能係因額外量化階層可加以補償；此處視為假說而非已驗證之機制。第 3.3 節將此偏移命名為殘差校正（*Residual Correction, RC*），並於硬體匯出管線中作為獨立每層元件

adapter_{l}_rc.dat 輸出。本節以寬版軟體 Adapter 搭配四種多分支寬度, 明確評估 RC 之貢獻; FPGA 部署之 MARS (於第 5.3.7 節定義) 之相同消融則於該節呈現 (表 5.9)。

寬版 *Conv-Adapter* [22] 配置之 Adapter—— 3×3 down convolution、 ≥ 2 -bit 採 ReLU (1-bit 採 signed)、per-channel α 、bottleneck 寬度 $C_{\text{out}}/4$ ——為寬版單分支配置, 結構上即先前工作 *Conv-Adapter* [22] 之 bottleneck (表 3.1 之 No-RC $M=1$ 列採之)。本小節與下一小節之軟體端分析皆採此配置; FPGA 部署之 MARS 待其設計取捨於第 5.3.7 節量化後再行介紹。RC 消融僅將 down convolution 之 bias 關閉/開啓 (開啓時 bias 以 Int8 量化), 其餘 Adapter 設計、主幹、最佳化器、隨機 seed 均固定。

Table 5.2: CIFAR-10 $\rightarrow$ SVHN(1W1A)1-bit 殘差校正消融。

Adapter	M	No-RC	With RC	Δ_{RC} (pp)	Full-FT
Conv-Adapter [22] (2024)	1	63.06%	79.23%	+16.17	
	2	66.12%	83.01%	+16.89	94.91%
MARS	3	67.79%	83.48%	+15.69	
	4	69.08%	84.44%	+15.36	

由表 5.2 得到三項觀察。第一, 偏置項於每個分支數 M 貢獻 +15 至 +17 pp, 無任何一格落於實驗噪聲之內。第二, 多分支擴展與 RC 並非替代品: 無論 M 為何, 關閉 RC 均使準確率下降 15 pp 以上。第三, 對照表 3.1 (第 3.1 節) 之 No-RC/RC 欄可確認此貢獻為 1-bit regime 之專屬現象: 跨位元寬度, 移除偏置於 1-bit 損失最高 +16.2 pp, 於 ≥ 2 -bit 至多 1.4 pp (多數在 ± 0.3 pp 內)。相同消融於部署 MARS 之結果, 及其隨各 target Full-FT 參考值之變化, 於第 5.3.7 節 (表 5.9) 呈現。

Conv-Adapter 瓶頸結構 [22] 之偏置項先前僅於全精度主幹下被研究, 當時其角色被周邊高精度算術所掩蓋。表 5.2 補上其於先前工作未涉足之 1W1A regime 中之角色描繪。

5.3.3 多分支擴展與軟體參考值

接下來考察: 在解除 FPGA-friendly 之 bottleneck 寬度限制後, 第 5.3.2 節之 RC 項與多分支擴展共同作用可將 1-bit 落差縮小至何程度。具體而言, 將 mid-channel 公式由 $C_{\text{in}}/4$ 替換為 $C_{\text{out}}/4$ 會於 $C_{\text{in}} \neq C_{\text{out}}$ 之層 (Conv3 與 Conv5) 放大 bottleneck; 此選擇僅用於軟體參考值, 第 5.3.7 節之部署 bitstream 全層仍採 $C_{\text{in}}/4$ 。

於寬版 *Conv-Adapter* [22] 幾何下，MARS 多分支擴展給出表 5.3 之軟體參考值，其單分支 $M=1$ 欄即 Conv-Adapter [22] 基線。

Table 5.3: CIFAR-10 backbone 之多分支擴展 (寬版幾何、軟體、1W1A、RC)。

	Conv-Adapter [22] (2024)	MARS					
Target	$M=1$	$M=2$	$M=3$	$M=4$	Δ_{multi} (pp)	Full-FT*	
SVHN	79.23	83.01	83.48	84.44	+5.21	94.91	
STL10	67.50	67.48	67.68	68.15	+0.65	71.05	
FashionMNIST	82.71	83.57	83.63	83.60	+0.89	92.36	
CINIC10	64.50	64.71	64.85	65.02	+0.52	68.02	

* Full-FT：全參數 1-bit 微調（不使用 adapter）；同訓練預算下之各目標實證參考值。

於寬版 Conv-Adapter [22] 幾何下，MARS 多分支擴展搭配 RC 於 $M = 4$ 達 84.44%（表 5.3），為本論文於 CIFAR-10 $\rightarrow$ SVHN 1-bit 取得之最高遷移準確率，作為對照第 5.3.7 節部署 MARS 之寬版軟體參考值。相較於 1-bit Full-FT 之 94.91%，仍有 ≈ 10.5 pp 殘差落差——此為不受限制之網路於相同 1-bit 權重、啟動與訓練協定下所達之準確率。於其餘三個目標上，相同的多分支擴展同樣為正但幅度小許多——FashionMNIST +0.89 pp、STL10 +0.65 pp、CINIC10 +0.52 pp——顯示其效益主要集中於單分支 1-bit 落差最大之 SVHN，於單分支準確率已趨飽和之目標上則迅速收斂。

5.3.4 backbone（來源）資料集之影響

前述結果皆固定以 CIFAR-10 為來源。為檢驗多分支 Adapter 機制與絕對遷移準確率是否取決於提供凍結 backbone 之資料集，將相同之 1-bit 單／多分支 Adapter 掃描以其餘四個資料集輪流作為來源重複進行（表 5.4）。

Table 5.4: 多分支擴展對其他來源 backbone 之推廣 (寬版幾何)。

Backbone	Target	Conv-Adapter [22] (2024)	MARS				Δ_{multi} (pp)	Full-FT*
		$M=1$	$M=2$	$M=3$	$M=4$			
SVHN	CIFAR-10	46.34	49.54	49.56	50.41	+4.07		81.70
SVHN	STL10	35.16	37.48	37.60	38.46	+3.30		63.14
SVHN	FashionMNIST	81.47	82.40	82.86	83.02	+1.55		92.32
SVHN	CINIC10	32.76	34.47	36.84	39.70	+6.94		67.85
STL10	CIFAR-10	68.92	69.97	70.58	70.99	+2.07		83.01
STL10	FashionMNIST	84.71	85.77	86.28	86.35	+1.64		92.73
STL10	SVHN	73.22	77.53	80.16	80.85	+7.63		95.32
STL10	CINIC10	56.07	56.74	57.39	57.36	+1.29		70.18
FashionMNIST	CIFAR-10	30.42	30.54	32.64	33.26	+2.84		78.37
FashionMNIST	STL10	30.84	31.94	33.20	31.81	+0.97		59.11
FashionMNIST	SVHN	39.16	43.29	45.68	45.68[†]	+6.52		94.36
FashionMNIST	CINIC10	24.82	25.33	27.28	26.34	+1.52		64.63
CINIC10	CIFAR-10	78.05	78.08	78.47	78.59	+0.54		80.92
CINIC10	SVHN	67.43	70.60	71.12	72.30	+4.87		94.62
CINIC10	STL10	66.42	66.86	67.33	67.83	+1.41		70.38
CINIC10	FashionMNIST	79.81	80.89	81.26	81.59	+1.78		91.99

* Full-FT：各來源之全參數 1-bit 微調——每格自其來源 backbone 初始化、固定 200-epoch 遷移預算；此為各（來源，目標）組合之參考值，非該目標之絕對上界（同一目標隨來源而變，如 STL10 介於 59.11–71.05）。

[†] 於 FashionMNIST→SVHN， $M=3$ 與 $M=4$ 之 best-epoch 準確率相同（45.68%，11,892/26,032 正確）：兩者於此困難之灰階 → 彩色遷移飽和於相同 best-epoch 測試操作點，故多分支擴展於 $M=3$ 之後無進一步可測增益。

跨來源 backbone 可見兩個現象（表 5.4）。其一，多分支擴展於幾乎每個 backbone→target 組合皆為正，顯示正向多分支效應不限於特定來源資料集或該來源之 checkpoint（皆屬所評估之 VGG-like 架構）（如 SVHN→CIFAR-10 +4.07 pp、STL10→CIFAR-10 +2.07 pp）。其二，絕對遷移準確率強烈取決於來源：自然影像 backbone（CIFAR-10、STL10）提供可廣泛遷移之特徵，於多數 target 達高準確率，而灰階之 FashionMNIST backbone 對自然影像目標遷移甚差（如於 CIFAR-10 僅 33.26%），因其單通道低解析特徵無法涵蓋這些目標所需之色彩與紋理統計。此不對稱性使自然影像 backbone（CIFAR-10、STL10，以及 CIFAR-10 衍生之 CINIC10）成為可行來源，其中 CINIC10 表現如較嘈雜之 CIFAR-10。**MARS 採用 CIFAR-10 作為凍**

結 backbone，基於兩個與部署相關之理由：其一，CIFAR-10 原生即為 32×32 ，直接吻合 VGG-like 之輸入幾何，而 STL10 原生為 96×96 需降採樣，造成預訓練與部署 32×32 輸入管線之解析度不一致；其二，1W1A VGG-like backbone 於 BNN/FINN 文獻中慣以 CIFAR-10 預訓練，提供標準且可直接比較之特徵萃取器。本章其餘部分因此採用 CIFAR-10 backbone，並於第 5.3.7 節帶入 bitstream。

幾何優劣排序之來源相依性。 寬版幾何於 CIFAR-10 部署主幹上達最高準確率（表 5.7 與表 5.8），惟此排序於所有來源主幹並不一致。將表 5.4 與表 5.18 之部署幾何掃描相比，可見其取決於來源：於 FashionMNIST 來源之配對（及多數 CINIC10 來源配對）上，較精簡之部署幾何追平甚或超越寬版幾何，FashionMNIST→SVHN 最高達 17 pp（表 5.8 列出）——此效應遠超該格之 seed 間變異（部署 $M=4$ 準確率於 seed 2024 / 2025 分別為 62.70%、63.17%），並於一次獨立重跑中重現。此反轉僅限於 source-target 錯配，且其幅度隨錯配嚴重度遞增：灰階 FashionMNIST 來源最大、CIFAR 衍生之 CINIC10 輕微、而於良好匹配之自然影像來源則幾乎不出現：於 SVHN 與 STL10 主幹上寬版幾何於所有目標皆居前，於 CIFAR-10 主幹上則於 1-bit 落差最大處領先（SVHN，+4.6 pp），其餘目標兩幾何持平（ ± 0.5 pp 內）。一個可能的解釋為較大之二值 adapter（每分支參數約 $5.6 \times 3 \times 3$ 核、per-channel α ）於嚴重錯配下之訓練動態：額外容量可能阻礙 1-bit 最佳化；責任設計軸尚未隔離，留待未來工作（第 6 章）。故部署幾何不僅是硬體妥協：其準確率代價僅限於良好匹配之來源（CIFAR-10→SVHN -4.6 pp），於錯配來源則持平或更佳。

5.3.5 跨資料集泛化

為驗證第 5.3.1–5.3.3 節之發現不限於 SVHN，本節於 MARS 下，使用相同訓練排程評估三組額外 target dataset：STL10（10 類，5,000 訓練 / 8,000 測試， 96×96 縮放至 32×32 ）、FashionMNIST（10 類， 28×28 灰階重採樣並複製通道至 $32 \times 32 \times 3$ ）、CINIC10（10 類，90,000 訓練 / 90,000 測試， 32×32 ；採用官方互斥之 train 與 test 子集，未使用 90,000 張之 valid 子集）。四組 target dataset 共用相同之 CIFAR-10 預訓練主幹。

跨資料集之 RC 結果（ $M=4$ 、RC on/off，以及各 target 之 Full-FT 參考值）以部署 MARS 呈現於表 5.9（第 5.3.7 節）。

各資料集之趨勢與第 3.3 節之詮釋一致：RC 之增益與 No-RC Adapter baseline 上方可用之「準確率餘量」成正比。SVHN 上 Full-FT 達 94.91%、No-RC Adapter 為 59.79%，

可填補 ≈ 35 pp，RC 補回其中 20 pp。FashionMNIST 上 Full-FT 參考值為 92.36%，RC 補回 7.4 pp。STL10 與 CINIC10 之 1-bit 主幹 Full-FT 參考值僅 71.05% 與 68.02%，No-RC Adapter 已逼近該參考值，RC 對應的可恢復空間僅 +0.6 與 +1.0 pp。

5.3.6 Multi-seed 變異性

Headline 數據以 seed 2024 報告。為界定變異性，本節以 seeds 2025 至 2028 重跑兩組代表性配置，於相同超參下計算平均與標準差 ($n = 5$)，列於表 5.5。

Table 5.5: CIFAR-10 $\rightarrow$ SVHN(1W1A) 多 seed 測試準確率。

配置	測試準確率
Full-FT (無 Adapter)	$94.92 \pm 0.10\%$
MARS, $M = 4 + \text{RC}$	$79.93 \pm 0.31\%$

MARS ($M = 4 + \text{RC}$, 79.93 ± 0.31) 與 Full-FT 參考值 (94.92 ± 0.10) 相差約 14.99 pp；因兩配置共用同一組五個 seed，採配對分析：各 seed 之差值為 15.14、15.10、15.28、14.47、14.94 pp，配對平均 14.99 ± 0.31 pp (95% CI [14.60, 15.37] pp)，遠超此協定下之 seed 層級變異。此配置於表 5.18 之單-seed headline 值 (79.81%) 亦落於此五-seed 包絡 (79.93 ± 0.31) 之內。表 5.3 之 MARS 寬版軟體參考值 ($M = 4 + \text{RC} = 84.44\%$) 為 seed 2024 單點量測，本表未進行 multi-seed 重跑。Adapter 與 Full-FT 之 ≈ 15 pp 落差遠超部署配置所觀察之 multi-seed 變異。另將表 5.18 之二十組部署幾何配對視為成對樣本： $M=4$ 對 $M=1$ 之差於 20/20 組皆為正，平均 +3.36 pp、中位數 +2.68 pp、bootstrap 95% 信賴區間 [+2.21, +4.68] pp；因各組共用五個資料集、非獨立重複，此為描述性離散度摘要而非推論性檢定。

多分支增益之多-seed 顯著性。 為使多分支 Adapter 效應具更穩固之統計基礎，將兩個代表性部署遷移 (CIFAR-10 $\rightarrow$ SVHN、CIFAR-10 $\rightarrow$ FashionMNIST) 於部署幾何下以 $n = 5$ 個 seed ({2024–2028}) 在 $M=1$ 與 $M=4$ 複現。對 seed 配對之 $M=4 - M=1$ 差值做 paired t -test，顯示於此 checkpoint 選點協定下多分支增益一致且顯著——SVHN $t \approx 34$ ($p < 10^{-5}$)、FashionMNIST $t \approx 17$ ($p < 10^{-4}$)——故 $M=1 \rightarrow M=4$ 之提升遠在 seed 間變異之外，而非單一 seed 假象；此顯著性描述 best-epoch 操作點之 seed 間變異，而非 held-out 泛化。作為無選擇之交叉檢核，直接讀取同批 run 訓練 log 之最終 (第 200 個) epoch——即完全不作 checkpoint 選擇之 last-epoch 協定——配對

$M=4 - M=1$ 增益為 SVHN $+6.87 \pm 0.50$ pp、FashionMNIST $+2.99 \pm 0.40$ pp，與 best-epoch 數字一致（甚至略大），而 best-epoch 選點僅使絕對準確率膨脹 0.5–0.8 pp；故此二遷移而言，正向之 $M=1 \rightarrow 4$ 增益並非僅由 best-epoch 選點所致；其餘準確率結果仍保有 test 集選點操作點之限制。

表 5.6 列出各格平均值與配對差摘要。

Table 5.6: $M=1 \rightarrow 4$ 增益之五-Seed 配對摘要（部署幾何, Seeds 2024–2028）。

目標	$M=1$ (平均 $\pm$ SD)	$M=4$ (平均 $\pm$ SD)	配對增益 (平均 $\pm$ SD)	95% CI (pp)
SVHN	73.17 ± 0.21	79.81 ± 0.27	$+6.64 \pm 0.43$	[+6.10, +7.18]
FashionMNIST	80.56 ± 0.27	83.25 ± 0.09	$+2.69 \pm 0.36$	[+2.25, +3.13]

準確率單位為%；CI 為配對 t 分布之 95% 區間 ($df = 4$)。此 $n=5$ 複現獨立於單-seed headline 訓練，絕對水準與表 5.18 或有些微差異；配對之 $M=4 - M=1$ 差值於同一 seed 內計算，可消除此批次位移並降低 seed 間變異，惟仍受同一 best-epoch test 集選點程序影響。

5.3.7 FPGA-deployed 配置與 SW-HW 差距

部署版 MARS 配置（已於第 5.3.2 節定義： 1×1 down convolution、signed binary 啓動、scalar α 、bottleneck 寬度 $C_{in}/4$ ）即帶入 bitstream。本小節量化此選擇背後之資源對準確率取捨。四項選擇皆以「將 Adapter Super Wrapper 相對於原 FINN MVAU 管線之 FPGA 額外資源降至最低」為原則挑選；其準確率代價與第 5.3.3 節之軟體參考值對照。表 5.7 條列每一設計軸上 MARS 與 Conv-Adapter [22] 之差異及其為何更精簡；表 5.8 接著量化由此產生之準確率取捨，晶片資源與功耗結果見第 5.4 節。

Table 5.7: 部署 MARS 相對 Conv-Adapter [22] 所改之五個設計軸。

設計軸	Conv-Adapter [22] (寬版幾何), 2024	MARS (部署)	MARS 為何更精簡
Down-conv (空間)	3×3	1×1	1×1 僅需 $1/9$ 權重與乘法，且不暫存鄰近像素——面積大幅縮小。
Conv MAC	浮點乘加 (模擬 ± 1)	XNOR + pop-count (0 DSP)	以位元邏輯 (XNOR + pop-count) 取代乘加——不需 DSP，只用基本邏輯閘。
啓動	ReLU (1-bit sign)	取 signed (1-bit)	1-bit 取號只是單一位元判斷、無額外算術——與二值資料路徑天然契合。
α 縮放	per-channel	scalar	一個共用常數取代每通道一組——折入貢獻 LUT，免存逐通道參數。
Bottleneck (mid)	$C_{\text{out}}/4$	$C_{\text{in}}/4$	中間通道更小且各層一致——權重與記憶體更少、控制更單純。

此軟體對硬體之準確率取捨因資料集而異——各 target 與其 Full-FT 參考值之距離不同——故逐 target 隨部署結果報告，而非分攤至個別設計軸。惟其中一軸可乾淨隔離：於 1-bit 單分支、關閉 RC、50-epoch 預算、固定其他軸下單獨比較 down-conv kernel (3×3 對 1×1)，差異僅 ≤ 1 pp 且在 seed 噪音內 (CIFAR-10 $\rightarrow$ SVHN 為 63.97% 對 65.06%)；此操作點位於 No-RC 單分支水準，可對照表 5.2 之 No-RC $M=1 = 63.06\%$)，故 $3 \times 3 \rightarrow 1 \times 1$ 空間縮減非此落差之主因，落差較可能主要來自 α 參數化與 mid-channel 寬度兩軸（此二軸未個別隔離，完整 factorial 列為未來工作，見第 6

章)。(XNOR+popcount 為模擬 ± 1 乘積之精確 1-bit 等價,是唯一完全不帶準確率代價之改動。)

表 5.8 將寬版與部署幾何於各分支數 $M=1-4$ 並列,涵蓋全部二十組 backbone-target 配對:於相似之自然影像來源上寬版領先(如 CIFAR-10 $\rightarrow$ SVHN, $M=4$ 為 84.44% 對 79.81%),於差異大之來源上排序反轉(FashionMNIST $\rightarrow$ SVHN,45.68% 對 62.70%; 第 5.3.4 節)。

Table 5.8: 寬版與部署幾何於各分支數之對照 (全部 20 組 backbone-target 配對); $\Delta_{M4} =$ 部署 - 寬版 ($M=4$)。第一部分: CIFAR-10、SVHN、STL10 來源 (接續次頁)。

Backbone	Target	方法 / 組態	$M=1$	$M=2$	$M=3$	$M=4$	Δ_{M4}
CIFAR-10	SVHN	Conv-Adapter [22] (2024)	79.23%	—	—	—	
		MARS(寬版幾何)	—	83.01%	83.48%	84.44%	-4.63
		MARS(部署)	73.72%	77.16%	78.77%	79.81%	
	FashionMNIST	Conv-Adapter [22] (2024)	82.71%	—	—	—	
		MARS(寬版幾何)	—	83.57%	83.63%	83.60%	-0.06
		MARS(部署)	80.42%	82.00%	82.82%	83.54%	
	STL10	Conv-Adapter [22] (2024)	67.50%	—	—	—	
		MARS(寬版幾何)	—	67.48%	67.68%	68.15%	-0.09
		MARS(部署)	67.46%	67.58%	68.00%	68.06%	
	CINIC10	Conv-Adapter [22] (2024)	64.50%	—	—	—	
		MARS(寬版幾何)	—	64.71%	64.85%	65.02%	+0.43
		MARS(部署)	64.80%	65.16%	65.38%	65.45%	
SVHN	CIFAR-10	Conv-Adapter [22] (2024)	46.34%	—	—	—	
		MARS(寬版幾何)	—	49.54%	49.56%	50.41%	-2.04
		MARS(部署)	43.18%	45.13%	46.93%	48.37%	
	FashionMNIST	Conv-Adapter [22] (2024)	81.47%	—	—	—	
		MARS(寬版幾何)	—	82.40%	82.86%	83.02%	-1.13
		MARS(部署)	78.51%	79.76%	80.56%	81.89%	
	STL10	Conv-Adapter [22] (2024)	35.16%	—	—	—	
		MARS(寬版幾何)	—	37.48%	37.60%	38.46%	-3.36
		MARS(部署)	34.41%	34.60%	35.45%	35.10%	
	CINIC10	Conv-Adapter [22] (2024)	32.76%	—	—	—	
		MARS(寬版幾何)	—	34.47%	36.84%	39.70%	-1.67
		MARS(部署)	33.59%	35.21%	34.29%	38.03%	
STL10	CIFAR-10	Conv-Adapter [22] (2024)	68.92%	—	—	—	
		MARS(寬版幾何)	—	69.97%	70.58%	70.99%	-2.20
		MARS(部署)	66.65%	67.90%	68.74%	68.79%	
	FashionMNIST	Conv-Adapter [22] (2024)	84.71%	—	—	—	
		MARS(寬版幾何)	—	85.77%	86.28%	86.35%	-2.16
		MARS(部署)	81.48%	82.96%	83.58%	84.19%	
	SVHN	Conv-Adapter [22] (2024)	73.22%	—	—	—	
		MARS(寬版幾何)	—	77.53%	80.16%	80.85%	-5.20
		MARS(部署)	68.15%	73.18%	74.17%	75.65%	
	CINIC10	Conv-Adapter [22] (2024)	56.07%	—	—	—	
		MARS(寬版幾何)	—	56.74%	57.39%	57.36%	-0.80
		MARS(部署)	54.26%	55.33%	56.17%	56.56%	

Table 5.8: (續) 寬版與部署幾何於各分支數之對照。第二部分: FashionMNIST、CINIC10 來源。

Backbone	Target	方法 / 組態	$M=1$	$M=2$	$M=3$	$M=4$	Δ_{M4}
FashionMNIST	CIFAR-10	Conv-Adapter [22] (2024)	30.42%	—	—	—	
		MARS(寬版幾何)	—	30.54%	32.64%	33.26%	+3.50
		MARS(部署)	33.63%	34.67%	35.84%	36.76%	
	STL10	Conv-Adapter [22] (2024)	30.84%	—	—	—	
		MARS(寬版幾何)	—	31.94%	33.20%	31.81%	+2.30
		MARS(部署)	32.64%	32.29%	33.64%	34.11%	
	SVHN	Conv-Adapter [22] (2024)	39.16%	—	—	—	
		MARS(寬版幾何)	—	43.29%	45.68%	45.68%	+17.02
		MARS(部署)	51.37%	58.19%	59.42%	62.70%	
	CINIC10	Conv-Adapter [22] (2024)	24.82%	—	—	—	
		MARS(寬版幾何)	—	25.33%	27.28%	26.34%	+3.22
		MARS(部署)	27.75%	28.37%	29.55%	29.56%	
CINIC10	CIFAR-10	Conv-Adapter [22] (2024)	78.05%	—	—	—	
		MARS(寬版幾何)	—	78.08%	78.47%	78.59%	+0.13
		MARS(部署)	78.21%	78.60%	78.87%	78.72%	
	SVHN	Conv-Adapter [22] (2024)	67.43%	—	—	—	
		MARS(寬版幾何)	—	70.60%	71.12%	72.30%	+4.04
		MARS(部署)	69.14%	73.88%	75.39%	76.34%	
	STL10	Conv-Adapter [22] (2024)	66.42%	—	—	—	
		MARS(寬版幾何)	—	66.86%	67.33%	67.83%	-0.41
		MARS(部署)	67.21%	67.60%	67.54%	67.42%	
	FashionMNIST	Conv-Adapter [22] (2024)	79.81%	—	—	—	
		MARS(寬版幾何)	—	80.89%	81.26%	81.59%	+1.29
		MARS(部署)	80.23%	81.86%	82.17%	82.88%	

粗體為該配對 $M=4$ 較佳之幾何。寬版 FashionMNIST→SVHN 之重複 45.68% 為 best-epoch 飽和, 見表 5.4 之†註; 幾何優劣排序取決於來源 (第 5.3.4 節)。

對部署 MARS 施以相同之 RC 消融 (表 5.9) 顯示 1-bit 效果更大 (SVHN 為 +20 至 +23 pp), 且於其餘三個 target, 可恢復之 headroom 隨各 task 之 1-bit Full-FT 參考值而變。

Table 5.9: 部署 MARS 之殘差校正消融 (1-bit)。

Target	M	No-RC	With RC	Δ_{RC} (pp)	Full-FT
SVHN	1 [†]	50.68%	73.72%	+23.04	94.91%
	2	55.14%	77.16%	+22.02	
	3	57.41%	78.77%	+21.36	
	4	59.79%	79.81%	+20.02	
FashionMNIST	4	76.18%	83.54%	+7.36	92.36%
STL10	4	67.50%	68.06%	+0.56	71.05%
CINIC10	4	64.42%	65.45%	+1.03	68.02%

[†] $M=1$ (含 RC) 即先前工作之單分支 Conv-Adapter 基線 [22] (基線政策, 第 5.3 節)。

表 5.10 報告 MARS 於四組 CIFAR-10 $\rightarrow$ target 遷移情境 (SVHN、FashionMNIST、STL10、CINIC10) 之軟體端準確率，以說明此結果並非單一 target dataset 之特例。RC 全程啟用，四個 MARS 列掃描分支數 M ；表格下方並列出四個 target 之 Full-FT 參考值 (解凍整網路、不使用 Adapter) 作為對照。

Table 5.10: MARS 跨資料集遷移準確率 vs. 分支數 M 。

配置	Source	Targets			
	CIFAR-10	SVHN	FashionMNIST	STL10	CINIC10
純主幹 (無 Adapter) retention	81.46%	28.68%	49.13%	63.93%	62.78%
MARS, $M = 1$	—	73.72%	80.42%	67.46%	64.80%
MARS, $M = 2$	—	77.16%	82.00%	67.58%	65.16%
MARS, $M = 3$	—	78.77%	82.82%	68.00%	65.38%
MARS, $M = 4$	—	79.81%	83.54%	68.06%	65.45%
Full-FT (無 Adapter, 參考值)	—	94.91%	92.36%	71.05%	68.02%

得到兩項觀察。第一， $M = 1 \rightarrow M = 4$ 之單調提升於所有四個 target 均成立，但提升幅度與「距 Full-FT 參考值之剩餘空間」成正比：SVHN 由 $M = 1$ 至 $M = 4$ 提升 +6.09 pp，而 STL10 與 CINIC10 分別僅提升 +0.60 與 +0.65 pp (其 1-bit 主幹之 Full-FT 參考值僅 71.05 與 68.02%，見第 5.3.5 節)。第二，先前所述之 SW-HW 差距 (部署之 MARS 79.81% 對寬版幾何 84.44%，於 SVHN) 為每個 target 於 CIFAR-10 主

幹之 FPGA-friendly 設計代價；由於各 target 與其 Full-FT 參考值之距離不同，此差距因資料集而異，應與第 5.4–5.5 節所報告之 FPGA 部署效益做權衡。

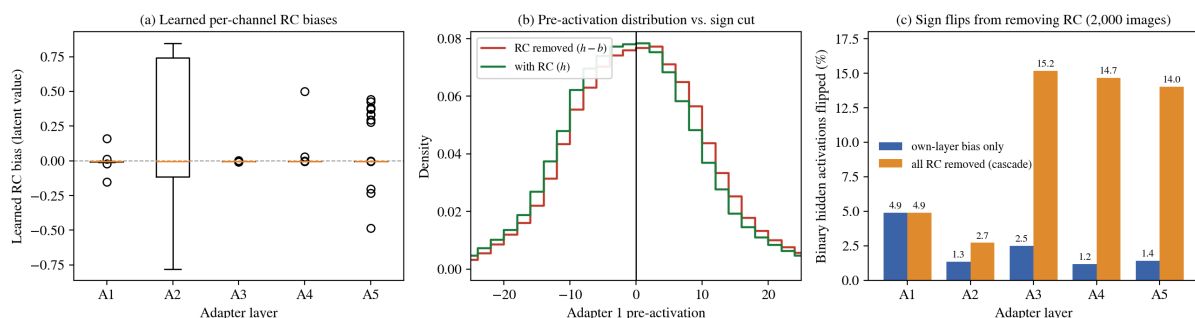


Figure 5.2: RC 之機制層級探查 (部署幾何 CIFAR-10→SVHN checkpoint, 2,000 張測試影像)。

為在消融之外補充機制層級證據，圖 5.2 直接探查一個訓練完成之部署幾何 $M=1$ checkpoint (seed 2024)。面板 (a) 為各層學得之逐通道 RC 偏置：非零、正負皆有、幅度依層而異。面板 (b) 匯集 Adapter 1 於 2,000 張 SVHN 測試影像上之 down-projection 前激活分佈：相當比例之機率質量緊鄰符號切點，故即使小於一個單位之切點位移亦會改變二值決策。面板 (c) 加以量化：僅清零單層之 RC 即翻轉該層 1.2–4.9% 之二值隱藏激活；全部移除 RC 則經凍結二值主幹連鎖放大，使較深 adapter 之二值編碼改變 14–15%。此等觀察與「RC 之作用在於安置二值化切點、而非增加表示容量」之假說一致，惟其本身尚不足以隔離自切點位置到最終準確率之因果路徑。

Table 5.11: MARS 多分支擴展貢獻 (四個 target)。

Target	MARS				Δ_{multi} (pp)	Full-FT
	$M=1$	$M=2$	$M=3$	$M=4$		
SVHN	73.72%	77.16%	78.77%	79.81%	+6.09	94.91%
FashionMNIST	80.42%	82.00%	82.82%	83.54%	+3.12	92.36%
STL10	67.46%	67.58%	68.00%	68.06%	+0.60	71.05%
CINIC10	64.80%	65.16%	65.38%	65.45%	+0.65	68.02%

表 5.11 將同一份 MARS 數據改以分支數編排，呈現各 target 相對單分支基線之多分支增益 Δ_{multi} 。

5.4 FPGA 資源使用與功耗

5.4.1 跨配置資源比較

表 5.12 比較同一 MARS 架構於 XC7Z020 上四個 build 配置之實作後 FPGA 資源使用。兩個變數軸：整合 MVAU + Adapter 資料路徑之 *throughput* 對 *compactness* 折疊（第 5.4.3 節），以及 *cfg_hub* runtime 切換基礎架構之無對有（第 5.6 節）。第一列為 FINN 產生之 *throughput* 純主幹（無 adapter、無 *cfg_hub*），即 *throughput* 資源零基線；第二列為 *throughput* build 整合 Adapter IP 與 *cfg_hub*（2 資料集）；第三列為 *compactness*（PE = 1）純主幹，即 *compactness* 零基線；第四列為真正部署之 *compactness* build，承載 N 任務 runtime *cfg* 切換（第 5.6.3 節）。

Table 5.12: 四個 MARS build 配置之 FPGA 資源使用。

配置	Function Switch	Occupied Slices	Slice LUTs	Slice Reg. (FF)	BRAM
Backbone (Throughput)	否	11,648 (87.58%)	25,322 (47.60%)	31,621 (29.72%)	99.5 (71.07%)
MARS (Throughput, 2-task)	是	13,272 (99.79%)	41,729 (78.44%)	53,285 (50.08%)	99 (70.71%)
Backbone (Compactness)	否	8,926 (67.11%)	15,651 (29.42%)	22,219 (20.88%)	85 (60.71%)
MARS (Compactness, N-task)	是	12,957 (97.42%)	39,967 (75.13%)	40,926 (38.46%)	84 (60.00%)

於 XC7Z020, 每個 slice 含 4 個 LUT 與 8 個 FF, 且只要其中任一被使用即計為被佔用; 故當分散式 RAM (LUTRAM) 與 control-set 限制使設計散佈於擺放網格時, 被佔用 slice 之使用率可遠高於 Slice-LUT 或 FF 之使用率——降低 LUT 數量未必能釋放 slice, 除非剩餘邏輯能被更密集地 pack。整合 adapter (含 *cfg_hub*) 使 slice 使用率逼近飽和, 雖 LUT 使用率不到 80%——Adapter IP 之分散式 RAM 為主結構飽和擺放網格: *throughput* build 達 13,272 / 13,300 (99.79%), 真正部署之 *compactness* build 達 12,957 / 13,300 (97.42%)。 *cfg_hub* 本體僅佔 19 LUT, 相對裸 adapter 整合幾乎不增加資源。 *compactness* 純主幹 (無 adapter) 僅 15,651 LUT / 8,926 slice: 均勻 PE = 1 折疊釋出之 slice 餘裕, 再投入承載單一組完整 runtime 可寫之 active 任務狀態, host 常駐任務影像依序載入其中 (第 5.6.3 節)。

主要觀察：

- **零 DSP 開銷**：兩配置皆使用零 DSP, 證實所有算術 (XNOR、popcount、LUT 貢獻縮放) 以組合邏輯實作。
- **BRAM 持平**：加入 Adapter 未增加 BRAM (−0.5 tile)。此為所有 Adapter down/up projection 權重與貢獻 LUT 實作於分散式 RAM (LUT-RAM) 之直接結果, 保留 BRAM 供主幹 memstream 使用。

- **LUT 使用**：部署之 compactness build 使用 39,967 LUT (75.13%)。後佈局層級分析顯示約 25,593 LUT (晶片約 48%) 歸屬於五個承載 adapter 之 Super Wrapper, 主體為 adapter 資料路徑 (權重 RAM 與 XNOR-popcount 樹) 與閥值比較邏輯。
- **Slice 飽和**：97.42% slice 使用率 (部署之 compactness build;throughput build 為 99.79%) 意味進一步新增必須配合邏輯層級最佳化釋出 slice 空間, 而非僅釋出 LUT。此為在 XC7Z020 上擴展 Adapter IP 至 $M \geq 2$ 配置的主要限制 (若不更換為更大平台)。

圖 5.3 為 MARS compactness build 於 XC7Z020 上之實作後佈局, 依功能群組著色 (色塊對照見圖側圖例)。此佈局將上述資源數據具象化:adapter 邏輯 (黃) 與主幹 MVAU(綠) 佔據並交錯分佈於整個晶粒, 而 Function Switch Controller(紫) 僅為零星少數 cell, 與其 19 LUT 之成本相符。

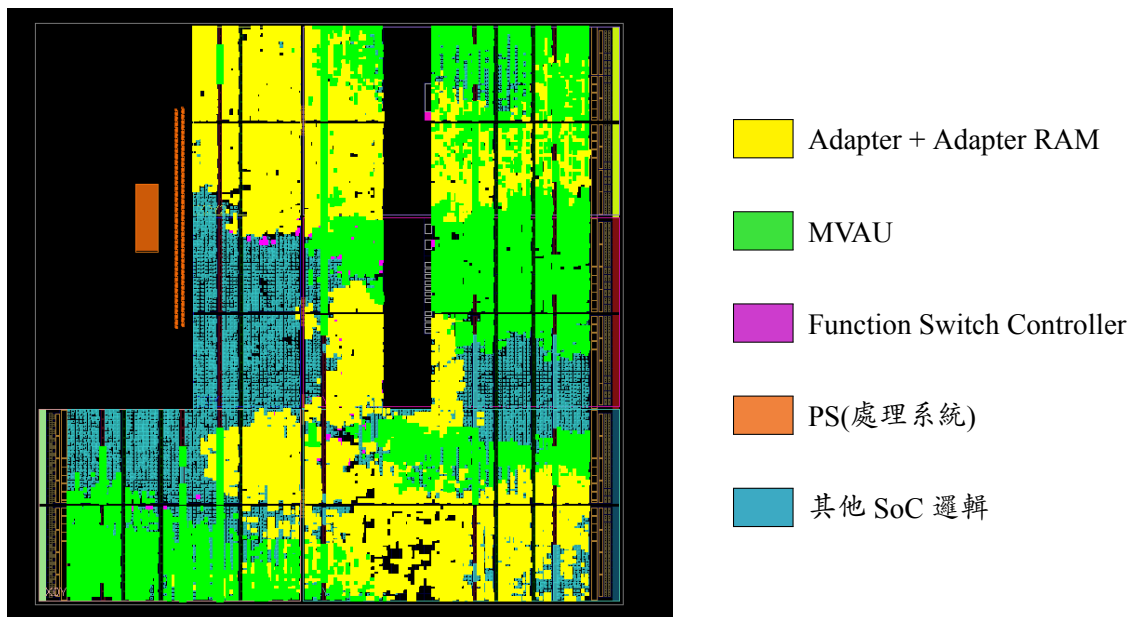


Figure 5.3: MARS compactness build 於 XC7Z020 之實作後佈局。

5.4.2 功耗

Table 5.13: 四個 MARS build 配置之功耗。

Build	總功耗 (W)	動態 (W)	靜態 (W)
Backbone (Throughput)	2.114	1.949	0.165
MARS (Throughput, 2-task)	2.215	2.046	0.168
Backbone (Compactness)	1.782	1.629	0.152
MARS (Compactness, N-task)	1.802	1.649	0.153

於 throughput build, 整合 Adapter 連同 cfg_hub 使晶片總功耗僅增加 0.101 W (+4.8%; 2.114 → 2.215 W), 動態項由 1.949 W 升至 2.046 W; runtime 切換之 cfg_hub 本體功耗可忽略。(不含 cfg_hub 之 Adapter 啓用 throughput build 量測為 2.214 W, 即第 5.5 節能效比較所採之數值。) 分解 Adapter 啓用 throughput build 之 PL 功耗, 串流資料流 (StreamingDataflowPartition_1, 使用者邏輯) 僅耗 0.732 W, ARM PS 子系統佔 1.256 W——剩餘多為常駐 PS。Compactness 版功耗較低 (1.78–1.80 W), 因均勻 PE = 1 折疊所實例化之平行邏輯遠少。實務上, 使用者面向的推論管線保持於次瓦級別, 此為邊緣部署相關的數據。下方跨平台比較採 FPGA 晶片總功耗, 以盡可能貼近 GPU/Jetson 量測所用之裝置層級功耗口徑; 惟方法上仍不對稱: FPGA 為 Vivado 估計值, GPU/Jetson 為執行時實測。本節所有吞吐量與能效數據皆屬 throughput build (異質 PE 最高 32); 為 N 任務 runtime 切換而提出之 compactness 設計點 (第 5.6.3 節) 刻意對卷積資料路徑時間折疊, 以約 107 img/s 運行, 故 throughput build 仍為能效比較之代表組態。

5.4.3 逐層 PE 配置：throughput 對 compactness

表 5.14 比較 throughput build 與 compactness build 之逐層 PE 折疊因子 (同一整合 MVAU + Adapter 資料路徑兩版本)。throughput build 於卷積層 (Conv1–Conv5) 選擇顯著更高的 PE 因子以展開整合資料路徑、維持串流吞吐量 (如上述報告); compactness build (用於第 5.6.3 節之 runtime 任務切換) 所有卷積單元 (MVAU0 與 Super1–5) 均使用 PE = 1, 將釋出之 slice/LUT 餘裕再投入 cfg 可寫之 adapter 與閾值儲存。全連接單元 MVAU6–MVAU8 因不含 adapter, 兩版本均保留原折疊配置。

Table 5.14: throughput 與 compactness build 之逐層 PE 折疊。

層	throughput PE	compactness PE
MVAU0	16	1
Super1	32	1
Super2	16	1
Super3	16	1
Super4	4	1
Super5	1	1
MVAU6	1	1
MVAU7	1	1
MVAU8	5	5

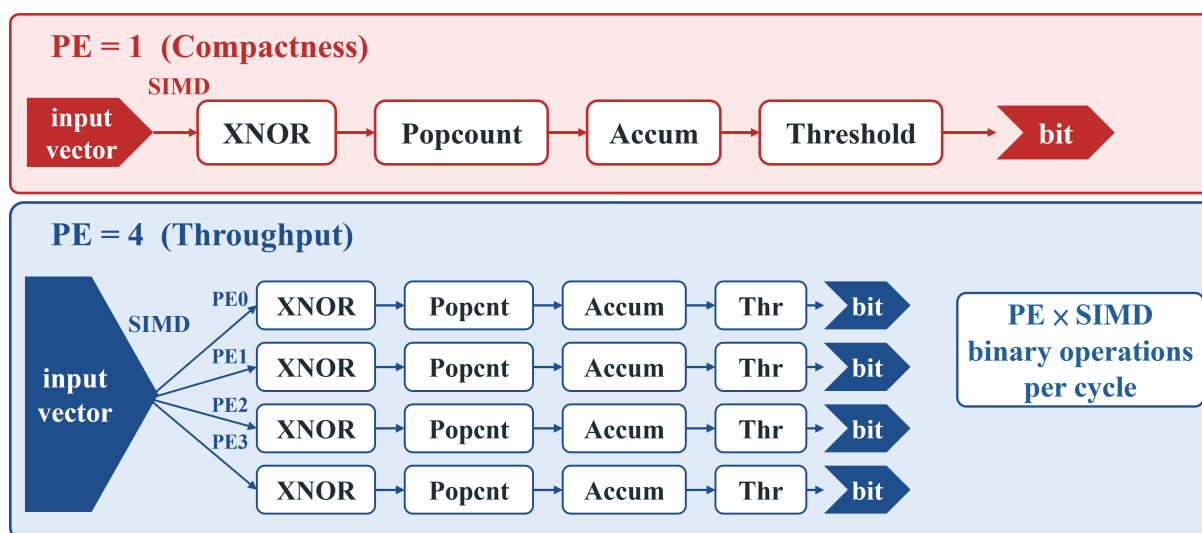


Figure 5.4: 兩種 build 之折疊差異:PE = 1(compactness) 對 PE 平行通道 (throughput, 以 PE = 4 為例)。

圖 5.4 將此折疊差異視覺化。均勻 PE = 1(compactness) 下, 每個卷積單元僅例化一條 XNOR–popcount–累加–閾值通道, 輸出通道逐一串行計算, 邏輯最省, 釋出之 slice 餘裕再投入 cfg 可寫之每任務狀態;throughput build 則於每單元例化 PE 條平行通道 (圖以 PE = 4 為例, 實際逐層因子最高達 32, 表 5.14), 各通道有獨立累加器與閾值比較器, 每週期完成 PE×SIMD 個二值運算。兩 build 之通道 RTL 完全相同, 僅例化通道數不同——此 PE 折疊差異為約 17.4× 吞吐差距與上述互補 slice 餘裕之主要架構來源。

5.4.4 多分支擴展成本與準確率-資源權衡

第 3 章預測 Multi-Branch Adapter 擴展之參數量呈線性成長; 本小節直接於硬體驗證此預測。兩種 build 之 $M=2-4$ 變體皆以 RTL 實作, 並走與部署加速器完全相同之 FINN Zynq 流程: 每個 adapter 站點之 Super Wrapper 例化 M 份不變的 Adapter 資料路徑, 其 Q8 貢獻於共用閾值比較前相加 (即式 (3.8) 之硬體實現); 五顆站點 IP 重新打包、串流 partition 重新匯出, 且完全相同之 Zynq 頂層設計 (PS、DMA 引擎與 interconnect 皆不變動) 以與部署 build 相同之合成策略進行合成。每個變體皆先以與部署設計相同之逐 MVAU RTL 模擬協定驗證: 兩種 build、全部五個站點、 $M \in \{1, \dots, 4\}$, RTL 輸出對照載入真實訓練完成之部署幾何分支權重之整數 golden 模型皆逐位元一致 (40/40 模擬全數通過)。由於 $M \geq 2$ 之設計超出 XC7Z020 容量, 擺放與繞線無法完成; 故表 5.15 之所有擴展數據——包含 $M=1$ ——皆於合成 (synthesis) 階段報告, 使所有比較處於流程之同一階段。

Table 5.15: 多分支 MARS 於 XC7Z020 之合成後資源與功耗擴展。

Build	M	Slice LUTs	Slice Reg. (FF)	BRAM	功耗 (W)
Throughput (2-task)	1	73,169 (137.5%)	65,717 (61.8%)	99	2.135
	2	109,731 (206.3%)	86,727 (81.5%)	99	2.193
	3	145,061 (272.7%)	107,471 (101.0%)	99	2.337
	4	187,521 (352.5%)	128,698 (121.0%)	99	2.468
Compactness (N-task)	1	47,931 (90.1%)	42,922 (40.3%)	84	1.841
	2	71,841 (135.0%)	60,390 (56.8%)	84	1.897
	3	95,593 (179.7%)	77,830 (73.2%)	84	1.915
	4	125,538 (236.0%)	95,285 (89.6%)	84	1.969

所有 build 之 DSP 使用量皆為零、BRAM 數量恆定: 每新增一個分支皆完全以 LUT/FF 邏輯與 distributed RAM 實現, 不影響 memstream 記憶體庫。百分比相對於 XC7Z020(53,200 LUTs、106,400 FFs)。功耗為合成後 Zynq 頂層 netlist(含 PS) 於 100 MHz 之 Vivado vectorless 估計。

表 5.16 以合成報告之資源值與 vectorless 功耗估計值重列 compactness build 每 M 之參數負擔與成本, 以支持第 3.4 節之線性成本主張。

Table 5.16: compactness build 之每分支參數量與 post-synthesis 成本。

M	新增參數	佔主幹 (1,546,690) 比例	Slice LUT(% XC7Z020)	功耗 (W)
1	58,533	+3.78%	47,931 (90.1%)	1.841
2	117,066	+7.57%	71,841 (135.0%)	1.897
3	175,599	+11.35%	95,593 (179.7%)	1.915
4	234,132	+15.14%	125,538 (236.0%)	1.969

由此可得三點觀察。其一, 資源成長如第 3.4 節所稱線性且可預測:compactness build 每增一分支花費 +23.8k–29.9k LUTs,throughput build 為 +35.3k–42.5k LUTs, 且 BRAM 恆定、DSP 全程為零。其二,XC7Z020 立即耗盡: $M=2$ 時 LUT 需求即已達器件之 135%(compactness) 與 206%(throughput), 任何擺放最佳化皆無法彌補——多分支部署需要更大的器件, 將第 5.4.1 節預期之限制加以量化。其三, 功耗並非瓶頸:compactness build 自 $M=1$ 至 $M=4$ 估計總功耗僅上升 0.13 W(+7.0%), 因常時開啓之 PS 與主幹佔主導; 此權衡受面積限制, 而非功耗。

合成階段之數字會高估最終成本, 因擺放與實體最佳化會跨模組邊界修剪並重新打包邏輯。表 5.17 以 $M=1$ build 校準此效應——其實作可完成, 且表 5.12 與表 5.13 之已發表實作後數據可供對照: 擺放與繞線於 compactness build 回收合成階段 16.6% 之 LUTs, 於 throughput build 回收 43.0%(其 PE 平行之 adapter 儲存提供更多重新打包空間)。因此上述合成階段之擴展數據為保守上界——即便如此樂觀解讀, 所有 $M \geq 2$ 之組態仍超出器件容量:compactness build 仍在其上 (以相同回收率換算 $M=2$ 約 113% LUT),throughput build 則遠超出。

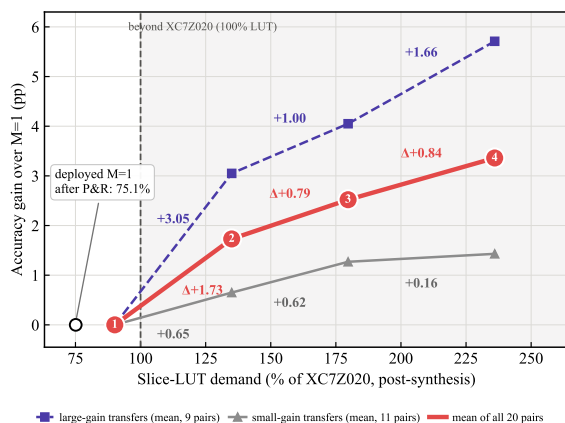
Table 5.17: $M=1$ build 之合成與實作階段使用率對照。

Build	階段	Slice LUTs	Slice Reg. (FF)	功耗 (W)
Throughput (2-task)	合成 (synthesis)	73,169 (137.5%)	65,717	2.135
	實作 (implementation)	41,729 (78.4%)	53,285	2.215
Compactness (N-task)	合成 (synthesis)	47,931 (90.1%)	42,922	1.841
	實作 (implementation)	39,967 (75.1%)	40,926	1.802

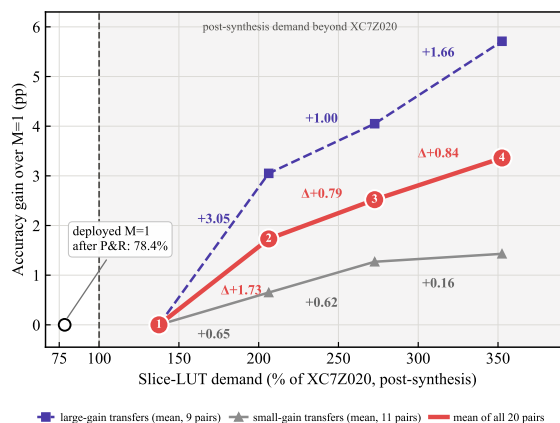
實作列為表 5.12 與表 5.13 之已發表實作後 build(occupied slices 13,272 / 12,957, 即 99.8% / 97.4%)。

Occupied-slice 數僅於擺放後存在, 故合成階段各列無此定義。

權衡中的準確率面以二十組部署幾何遷移對量化: 五個主幹 (CIFAR-10、SVHN、STL10、FashionMNIST、CINIC10) 與四個目標資料集之全部組合, 每組皆以與第 5.3 節相同之協定在 $M=1-4$ (含 RC) 下訓練。表 5.18 列出全部二十組; CIFAR-10 主幹之四列與表 5.11 相同, 其餘十六組為本研究新訓練。與使用寬版軟體幾何之跨主幹研究 (表 5.4) 不同, 此處所有數字皆採用表 5.15 硬體成本所對應之部署幾何。所有 adapter 分支皆為 cfg 可寫, 固定大小之主幹記憶體跨任務不變; 故硬體成本對每一組遷移皆相同, 二十條準確率軌跡共用同一成本軸。



(a) Compactness(N-task)



(b) Throughput(2 資料集)

Figure 5.5: 多分支擴展之準確率增益對合成後 LUT 需求 (二十組遷移對); 大增幅定義為 $\Delta_{\text{multi}} = \text{Acc}_{M=4} - \text{Acc}_{M=1} \geq 3 \text{ pp}$ (九對), 其餘十一對為小增幅組。

Table 5.18: 二十組主幹-目標遷移對之部署幾何多分支準確率。

主幹	目標	$M=1$	$M=2$	$M=3$	$M=4$	Δ_{multi} (pp)
CIFAR-10	SVHN	73.72%	77.16%	78.77%	79.81%	+6.09
	FashionMNIST	80.42%	82.00%	82.82%	83.54%	+3.12
	STL10	67.46%	67.58%	68.00%	68.06%	+0.60
	CINIC10	64.80%	65.16%	65.38%	65.45%	+0.65
SVHN	CIFAR-10	43.18%	45.13%	46.93%	48.37%	+5.19
	FashionMNIST	78.51%	79.76%	80.56%	81.89%	+3.38
	STL10	34.41%	34.60%	35.45%	35.10%	+0.69
	CINIC10	33.59%	35.21%	34.29%	38.03%	+4.44
STL10	CIFAR-10	66.65%	67.90%	68.74%	68.79%	+2.14
	FashionMNIST	81.48%	82.96%	83.58%	84.19%	+2.71
	SVHN	68.15%	73.18%	74.17%	75.65%	+7.50
	CINIC10	54.26%	55.33%	56.17%	56.56%	+2.30
FashionMNIST	CIFAR-10	33.63%	34.67%	35.84%	36.76%	+3.13
	STL10	32.64%	32.29%	33.64%	34.11%	+1.47
	SVHN	51.37%	58.19%	59.42%	62.70%	+11.33
	CINIC10	27.75%	28.37%	29.55%	29.56%	+1.81
CINIC10	CIFAR-10	78.21%	78.60%	78.87%	78.72%	+0.51
	SVHN	69.14%	73.88%	75.39%	76.34%	+7.20
	STL10	67.21%	67.60%	67.54%	67.42%	+0.21
	FashionMNIST	80.23%	81.86%	82.17%	82.88%	+2.65
相對 $M=1$ 之平均增益 (pp)		—	+1.73	+2.52	+3.36	—

所有數值皆採 FPGA 部署之 adapter 幾何 (含 RC; $\Delta_{\text{multi}} = M=4 - M=1$); CIFAR-10 各列即表 5.11, 其餘十六組依相同 200 epoch 協定訓練。表 5.4 採寬版軟體幾何; 兩表並非逐格可比, 且幾何優劣排序本身取決於來源 (見第 5.3.4 節之範圍說明)。全部二十組皆為單一 seed (2024); 代表性 $n=5$ 配對檢定僅於 CIFAR-10→SVHN 與 CIFAR-10→FashionMNIST 報告 (第 5.3.6 節)。

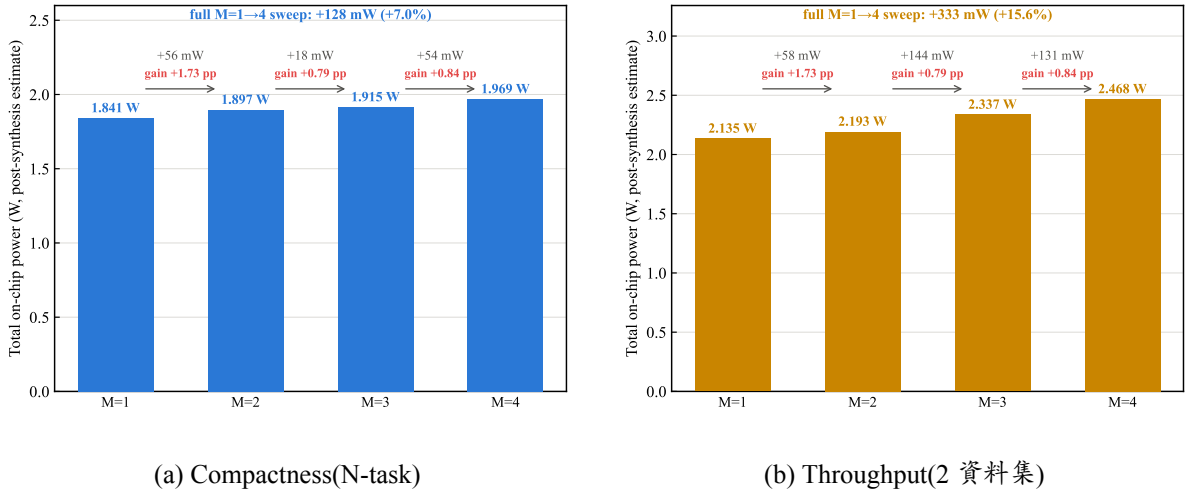


Figure 5.6: 各分支數之總晶片功耗(合成後估計)。

圖 5.5 與圖 5.6 呈現由此而得之權衡, 且數據使性價比排序一目瞭然。圖 5.5 各面板同時繪出全部二十組之平均(粗線)與大增幅($\Delta_{\text{multi}} \geq 3$ pp, 九對)、小增幅(十一對)兩組之平均, 其成本軸為表 5.15 之合成後需求——此值高估最終佔用: 已部署之 $M=1$ build 於擺放繞線後可放入器件 (75.1%/78.4% LUT, 空心標記; 表 5.17), 故灰底不可行區僅涉及 $M \geq 2$ 各點。就二十組之平均而言, 第一個新增分支值 +1.73 pp, 而第二、第三個分支僅各值 +0.79 與 +0.84 pp——但在 compactness build 上每個新增分支之代價幾乎相同 (+23.8k–29.9k LUTs)。以性價比計, 第一步每 10k LUTs 換得 0.72 pp, 達其後各步 (0.33/0.28 pp) 之兩倍以上: $M=2$ 即是單位硬體換得準確率之高峰, 以約三分之一之總增量面積取得可得多分支總增益之 51%。於所評估之 $M=1-4$ 掃程內, 功耗並非決定性約束——面積才是: 整個掃程在 compactness build 僅增加 128 mW(+7.0%), throughput build 為 333 mW(+15.6%)(圖 5.6), 即每挽回一個百分點僅需數十 mW。

因此建議之操作點取決於平台與任務。在 XC7Z020 上, 問題由可行性直接決定: $M=1$ 於合成階段已佔用 90% 之器件 LUT(實作後 slice 佔用 97.4%, 表 5.12), 故已部署之單分支組態是唯一可實現之點, 也仍是此平台上之正確選擇。在所評估之分支增量中, $M=2$ 對準確率敏感之任務提供最高之每合成階段 LUT 準確率報酬, 且在困難遷移上具決定性——僅此一步 FashionMNIST→SVHN 即挽回其總增益 +11.3 pp 中之 +6.8 pp; 其於更大 Zynq 器件上之可部署性與效率仍未經驗證。 $M=3-4$ 以約一半之效率換取剩餘準確率, 僅在準確率壓倒一切面積考量時才值得; 而於所評估範圍內, 擴展超過 $M=4$ 之報酬遞減, 與第 3.4 節瓶頸容量飽和之論述一致。數據另給出兩點限定。其一, 報酬高度依任務而異(表 5.18): 單分支落差本已很小之遷移(如以 STL10 為目標者)在任何分支數下增益皆不足 1 pp, 故分支數宜視為逐任務之部署參數——容易之目標維持 $M=1$, 困難之目標於器件允許時移至 $M=2$ 。其二, throughput build 上每個分支需多耗約 1.5 倍邏輯

(+35k–42k LUTs) 而準確率相同, 故 compactness 摺疊亦是承載多分支擴展之較佳宿主。

5.4.5 Golden 模型驗證

端到端 golden 模型驗證於真正、已上板驗證之 runtime 切換 build——即於單一 bitstream、單一凍結共用主幹上 runtime N 任務切換之精簡 compactness build——進行。CIFAR-10、SVHN、FashionMNIST、STL10、CINIC10 五者各以 10,000 張測試影像 (STL10 為 8,000 張) 與其軟體參考相差 0.69 個百分點以內, 且無需逐資料集重新 build; 完整表格與量測協定見第 5.6.3 節 (表 5.23)。此三層級模式——各 MVAU RTL 模擬 (43,520 筆, 100% 通過)、頂層資料流模擬與 PyTorch golden 一致、上板端到端執行——證實各 MVAU RTL 層級為位元精確、頂層資料流模擬與軟體一致、上板準確率與軟體參考差距在 0.69 pp 以內。

5.5 跨平台效能比較

量測方法 本節所有跨平台數據皆遵循同一量測協定, 使所報數值於協定層級可比 (相同輸入、batch 與前處理)。每個資料集以 10,000 張測試影像、batch 1000 (十個 batch)、相同 /255 NCHW 前處理量測, 且每個數值皆為 10 次重複之平均。吞吐量為端到端速率 (主機資料準備、傳輸/DMA、運算與讀回), batch 1000 下之攤提單張時間為其倒數 (此非單張請求延遲)。FPGA 之 pipeline 與 compactness build 由同一量測迴圈驅動, 僅差在所載入之 bitstream 及 (切換 build 之) 每資料集一次 cfg_hub 寫入; GPU 與 Jetson 則於其原生推論堆疊上以相同 batch 執行先前工作之 Conv-Adapter 基線 (1W1A 精度對齊); 故表 5.20 為精度、輸入尺寸與 batch 協定對齊之部署層級比較, 而非同一模型之平台掃描。FPGA 功耗為 Vivado 後佈局估計, GPU 功耗為實測 (RTX 經 NVML、Jetson 經 tegrastats)。切換時間另行量測, 為在已編程之 bitstream 上一次整包 cfg_hub 改寫 (6,757 字、無任何 bitstream 重載) 之牆鐘時間, 以 50 次隨機順序切換之平均 ± 標準差呈現。

表 5.19 與 5.20 分別比較四種平台配置在純主幹與 adapter 工作負載下的推論效能; 表 5.20 中 FPGA 跑 MARS, GPU 與 edge-GPU 跑前入 Conv-Adapter (3 × 3) 於相同 1W1A 精度。所有平台均以 batch size 1000、10,000 張影像、端到端、10 次量測取平均; 表 5.19 使用 CIFAR-10, 表 5.20 使用 SVHN, 故每張表於各平台間遵循一致之量測協定 (相同資料集、影像數、batch 與前處理)。吞吐量本即不受資料內容影響 (latency 取決於 tensor shape, 而非 pixel 值)。

Table 5.19: 純主幹: 跨平台效能 (CIFAR-10, batch 1000)。

指標	Backbone (RTX 4090)	Backbone (Jetson Orin NX)	Backbone (MARS Compactness)	Backbone (MARS 管線化)
FPS (img/s)	8,359	4,090	107.1	1,859
攤提每張時間 (ms)	0.120	0.244	9.338	0.538
GOPS	994	486.3	12.73	221.0
功耗 (W)	101.1	19.54	1.782	2.114
GOPS/W	9.83	24.9	7.14	104.5
能效 (img/s/W)	82.7	209.3	60.1	879.4

Table 5.20: 資料集、精度、輸入尺寸與 batch 對齊、惟 adapter 幾何不同之部署層級比較:MARS (FPGA) 對 Conv-Adapter [22] (GPU/Jetson),SVHN,batch 1000。

指標	Conv-Adapter [22] (2024) (RTX 4090)	Conv-Adapter [22] (2024) (Jetson Orin NX)	MARS (Compactness, N-task)	MARS (Throughput, 2-task)
Adapter 幾何	3×3 寬版	3×3 寬版	1×1 部署	1×1 部署
FPS (img/s)	9,388	2,752	107.1	1,866
攤提每張時間 (ms)	0.107	0.363	9.338	0.536
功耗 (W)	132.48	19.23	1.802	2.214
能效 (img/s/W)	70.86	143.1	59.4	842.7

FPGA 功耗為 Vivado 後佈局估計值,GPU/Jetson 為執行時實測 (NVML/tegrastats); 故能效倍率為工作負載與量測方法相依之結果。各平台之模型幾何不同:FPGA 執行部署版 MARS 幾何 (1×1 centre-selection adapter,RTL 執行 3.01 M adapter MACs),GPU/Jetson 執行先前工作之 Conv-Adapter(3×3 寬版幾何)、同為 1W1A 精度——屬部署層級比較而非同模型掃描。兩幾何之軟體準確率亦不同: 寬版幾何之 best-epoch CIFAR-10→SVHN 參考值為 79.23%(表 3.1), 部署幾何為 73.72%(表 5.10); 故能效倍率比較的是具部署代表性之實作, 而非準確率對齊之模型。

5.5.1 分析

1. **邊緣之限制是絕對功耗, 而非吞吐量。**資料中心 GPU 於此小型二值網路達到較高之絕對吞吐量——執行 Conv-Adapter 之 RTX 4090 之 9,388 img/s 對 throughput build 之 1,866 img/s——但其功耗為 FPGA NPU 之約 60× (132 W 對 2.2 W)。對本論文所針對之 sub-watt 至數瓦邊緣封套,132 W 之裝置、甚至 19 W 之 Jetson 皆無法塞入, 而 2.2 W 之 FPGA 可以。故此比較是可部署功耗等級之比較, 而非峰值速度之比較。
2. **於可部署等級內之能效。**於 adapter 任務,throughput build 達 842.7 img/s/W,

為執行 Conv-Adapter 之 RTX 4090 (70.86 img/s/W) 之 $11.9\times$ 、Jetson Orin NX (143.1 img/s/W) 之 $5.9\times$; 於主幹任務達 879.4 img/s/W。compactness build (59.4 img/s/W) 並非以能效為目標之配置: 其以吞吐量換取容納 cfg 可寫 per-task 狀態 (單一 bitstream 切換所需) 之 LUT 餘裕。

3. **折疊組態與兩個 build 之角色。** throughput build 之主幹採用 FINN 標準之異質逐層 PE/SIMD 折疊 [9, 10]; 本論文不將此折疊列為貢獻。compactness build 採均勻 PE=1。所得約 $17.4\times$ 之吞吐量差 (107 $\rightarrow$ 1,859 img/s) 為 runtime 切換設計點之代價, 而非本論文所提之最佳化。本論文之管線貢獻為將 adapter 側流整合並以週期精準之 FIFO 同步進折疊資料路徑, 使啓用 adapter 後之吞吐量維持在主幹之約 0.5% 以內 (1,859 $\rightarrow$ 1,866 img/s, 屬量測間變異)。
4. **板上準確率對等。** 板上正確性由第 5.6.3 節之 compactness build 於正規 cifar10_1w1a 主幹證實——五個資料集板上與軟體相差 0.69 個百分點以內 (表 5.23); 殘餘差距與定點及閾值量化效應之累積一致, 而非觀察到之 bit 層級 RTL 不一致。Throughput build 之 Adapter IP RTL 與之同源、模組層級 bit-equivalent。

逐平台吞吐、功耗與能效數字彙整於表 5.19 與表 5.20; 鑑於 adapter 幾何與功耗量測方法不同, 以原始數字呈現而不另作倍率圖。

5.5.2 跨平台比較之公平性與範圍

上述跨平台數據應視為特定工作負載之量測, 而非「FPGA 普遍優於 GPU」之一般宣稱。為使比較可防守, 明確說明以下條件與限制。

- **相同精度與代表性部署方法。** FPGA 執行 MARS; GPU 與 edge-GPU 執行前人 Conv-Adapter (3×3) 於相同 1W1A 精度、相同 SVHN 工作負載——此為部署層級比較 (各方法於其代表平台、相同位元寬), 而非同模型之跨平台掃描。
- **相同 batch 且完整批次化。** 各平台皆以 batch 1000、相同 10,000 張影像量測, 重複 10 次取平均。此使 GPU 與 Jetson 脫離 kernel-launch 受限區、避免低使用率——故所報 $11.9\times$ 能效優勢係於完整批次化協定下量測——但我們不宣稱 batch 1000 對每一平台皆為全域吞吐最佳; 就絕對吞吐量而言, RTX 4090 仍數倍快於 FPGA。
- **端到端計時。** FPS 為端到端 (含 host-to-device 傳輸、運算與 device-to-host 回傳), 故 GPU/Jetson 之數值未因排除 CPU 前處理或資料搬移而被美化。

- **功耗量測之不對稱。** GPU 與 Jetson 功耗為執行時實測 (RTX 4090 以 NVML 板級功耗、Jetson 以 `tegrastats VDD_IN` 軌), 而 FPGA 為 Vivado 後佈局之晶片總功耗估計值, 非牆上插座實測。FPGA 數值相對 GPU 可能偏樂觀, 比較時應將此不對稱納入考量。
- **本比較主張與不主張者。** 其顯示於 sub-watt 至數瓦封套——本研究量測之 GPU/SoC 組態皆未達此等級——內, 於所量測之平台中, MARS 於此特定 1W1A 串流工作負載上取得最高能效; 其不主張 PYNQ-Z2 於一般 AI 工作負載上優於 GPU, 該情境下 batch、runtime framework 與功耗量測方法皆會改變結論。

5.6 執行時期模型切換

本論文於同一 Super Wrapper 架構下提供兩種硬體建置, 通篇貫穿使用。兩者皆為 *pipelined* 串流資料流設計; 其差別僅在於 per-layer PE/SIMD 折疊, 而非是否管線化。**throughput build** 採用 FINN 預設之異質折疊 (PE 最高 32), 用於跨平台吞吐量/能效比較 (第 5.5 節); **compactness build** 將各 MVAU 之 PE 均設為 1——此為刻意收小整體資源, 使單一組完整 runtime 可寫之 active 任務狀態能舒適放入裝置——用於本節之 runtime 任務切換展示。兩版本 RTL 除折疊參數與 `cfg` 可寫之 adapter/閾值儲存外完全相同, 分別以吞吐量/slice 與 LUT 餘裕作為設計取舍。

2 任務切換與 N 任務切換之差別 本論文中兩個 build 所標示之任務數 (throughput build 標為 2-task, compactness build 標為 N -task) 代表同一 `cfg_hub` 機制之兩種深度, 而非兩種不同機制。於 throughput build, 其中一個 adapter 任務於合成時即已備妥: SVHN 之 adapter down/up 投影權重、RC 偏置與 α 貢獻 LUT 皆烘入權重記憶體之開機內容。其板上驗證之切換協定因此僅重寫區分兩個既備模型之狀態——各層 `adapter_enable` 位元、融合 Q8 閾值與符號位元、MVAU0/MVAU6/MVAU7 閾值及分類器投影 (3,781 筆字寫入, 約 15 KB)——並於恰好兩個模型間撥切: backbone 原生任務 (adapter 停用) 與合成時烘入之 adapter 任務。此 build 之 adapter 權重記憶體同樣為 `cfg` 可寫, 故經同一路徑載入更多任務於架構上可行, 但於此折疊組態下既未執行、亦未上板驗證。相對地, compactness build 不烘入任何任務: 權重記憶體開機為空, 每一個任務相依位元組——含 adapter down/up 權重、RC 與貢獻 LUT——皆於執行時期以完整約 25 KB 之每任務 blob 載入 (6,757 筆字寫入; 第 5.6.3 節)。可達模型集合因此僅由主機端組態 blob 決定, 此即令 throughput build 之兩模型撥切提升為真正 N 任務切換之關鍵。

表 5.21 彙整兩者差異。

Table 5.21: 2 任務與 N 任務切換對照。

	Throughput(2-task)	Compactness(N -task)
折疊方式	FINN 異質 PE ≤ 32	均勻 PE = 1
合成時烘入	一個 adapter 任務 (開機狀態)	無——記憶體開機為空
每次切換搬移	閾值 + 分類器 (約 15 KB)	完整 blob(約 25 KB, 含 adapter 權重)
可達模型	恰好兩個	任意符合固定主幹/張量/head 介面之任務
驗證	板上, 2 任務	板上, 5 任務 (N 為架構性質)

本節呈現於 PYNQ-Z2 上另建之 runtime 切換 bitstream 變體實作與驗證。於 PYNQ 類平台, 每任務呼叫 FPGA 部分重組態之實測延遲為 5.2 ms (ZyCAP [25]) 與 97–114 ms (PCAP [24])。本實作不採此法, 而是於執行時期透過統一 AXI-Lite 位址映射, 僅重新編程所有任務相依參數——MVAU0 閾值、MVAU6/MVAU7 閾值、分類器權重, 以及五組 Adapter 區塊 (enable、down/up 投影 RAM、RC 偏置、閾值、符號、貢獻 LUT)。關鍵觀察: 因所有合格下游任務皆共用同一凍結 1W1A VGG-like 主幹, 任務間之二值卷積與 FC 權重相同, 只有 BatchNorm 融合閾值、分類器輸出投影與 Adapter 貢獻需不同。此觀察使單一 bitstream 之執行時期任務切換具可行性, 也正是第 3.1 節所述缺口之架構層回應。

5.6.1 cfg_hub 架構

配置中樞為 AXI-Lite 從站, 於部署之 compactness build 映射基址 0x40010000、容量 64 KB。16-bit byte 位址拆解為 3-bit 單元選擇 (byte_addr[15:13]) 與 11-bit 字位址 (byte_addr[12:2]), 經組合邏輯多工扇出至八個目標單元:

Table 5.22: cfg_hub 位址映射 (基址 0x40010000, 64 KB)

單元	Byte 範圍	目標
0 low	0x0000--0x0FFF	MVAU0 閾值 (64 字, 32-bit 有符號)
0 high	0x1000--0x1FFF	MVAU8 分類器權重 (1024 字, 8-bit; memstream AXI-Lite bridge)
1–5	0x2000--0xBFFF	Super1–5 Adapter: word 0 為 enable, 4+ 為 RC, 128+ 為 down 投影, 640+ 為 up 投影, 1152+ 為 thresh, 1408+ 為 sign, 1664+ 為貢獻 LUT
6	0xC000--0xDFFF	MVAU6 閾值 (512 字, 8-bit)
7	0xE000--0xFFFF	MVAU7 閾值 (512 字, 10-bit)

以下三項 RTL 層修改使此機制可行, 且不影響串流資料流：

1. **MVAU0**：原本寫死的閾值多工器改為 16 個分散式 RAM bank, 每 bank 皆對 `cfg_hub` 暴露寫入埠。
2. **Super1–5 Stream Adder + Threshold**：Q8 閾值 RAM 與符號 bit 改為 `cfg` 可寫之分散式 RAM, 並於 word 0 加入 `adapter_enable` 暫存器閘控 Adapter 貢獻; 停用時 Super1–5 回到純主幹行為; 於 compactness build, 各 Super wrapper 之 adapter 權重記憶體 (down/up 投影、RC、貢獻 LUT) 亦為 `cfg` 可寫之分散式 RAM, 使任一任務之完整 adapter 可於執行時期載入 (第 5.6.3 節)。
3. **分類器 memstream (MVAU8)**：晶片內 memstream 由 `cfg_hub` 經 `_imp` 內之 `cfg`→AXI-Lite 狀態機驅動, 支援 CIFAR-10 與 SVHN 差異之輸出投影權重執行時期重載。

因所有合格任務之二值主幹權重相同 (烘入電路、任務間不變), 整體任務切換僅需傳輸每任務組態約 6,757 筆 `cfg-hub` 字寫入 (實體 blob 約 25 KB)：MVAU0/MVAU6/MVAU7 閾值、分類器投影與五組 Adapter 區塊。亦無需顯式 flush 管線——唯一前提是重新編程前觀察到 Output DMA done bit, 此後串流管線已無任務相依狀態。此機制之量化板上準確率與切換延遲結果報告於真正之 N 任務 compactness build (第 5.6.3 節)。

5.6.2 實作層觀察：Vivado OOC 合成快取

於迭代 `cfg_hub` 設計時, 遇到一項非顯而易見的工程發現：當一個 stitched IP (MVAU_hls_0/6/7、memstream AXI-Lite bridge 與 `_imp` 包裝) 的 RTL 被修改時, 只 `reset synth_1` 與 `impl_1` 並不足夠——Vivado 為每一個封裝 IP 保有獨立之 out-of-context (OOC) 合成快取 (如 `top_StreamingDataflowPar_0_0_synth_1`), 頂層合成即便原始碼較新也會重用舊 netlist。症狀為 build 正常完成, 但 `cfg_hub` 寫入對推論沒影響。解法 (已內嵌於專案建置腳本) 是枚舉所有具 `IS_SYNTHESIS` 屬性的 run, 在 `reset impl_1` 前先 `reset` 之。此點列為方法論備忘, 避免未來擴展 Super Wrapper 時再次默默失敗。

5.6.3 compactness 變體：板上 N 任務切換與可擴展性

前述 throughput build 採 FINN 預設之異質折疊（PE 最高 32）且僅以 2-task 組態驗證。為探究單一 bitstream、cfg_hub 驅動之切換原理在任務數量上之可擴展性，本論文將相同 Super Wrapper 架構改折疊為均勻 PE = 1 資料路徑，重建為一獨立、自洽之第二設計點。將每個卷積單元降為 PE = 1（全連接單元 MVAU6–MVAU8 保留原折疊，MVAU8 之 PE = 5，表 5.14），以吞吐量（卷積資料路徑時間折疊）換取大量 slice/LUT 餘裕，並將此餘裕投入使 CIFAR-10、SVHN、FashionMNIST、STL10、CINIC10 五個資料集所對應之每任務模型狀態，於單一 bitstream、單一凍結共用 1W1A VGG-like 主幹上完全可 runtime 切換。此 build 採用正規 cifar10_1w1a 1W1A CIFAR-10 主幹檢查點（軟體 CIFAR-10 準確率 81.46%），本論文全篇上板之準確率正確性驗證均以此 compactness build 為依據；throughput build 之 Adapter IP RTL 與本 build 同源，角色限於吞吐量／能效量測（第 5.6 節）。本節之可擴展性主張針對切換機制，與絕對主幹準確率無關。此 build 之 cfg_hub 映射於 0x40010000；切換機制其餘與第 4.5 節相同。

為何可擴展至任意任務數量。 共用主幹之二值卷積與 FC 權重烘入 bitstream、任務間從不改變；不同者僅為 BatchNorm 融合之 MVAU0/MVAU6/MVAU7 閾值、分類器輸出投影權重，與五層 Adapter 區塊（enable/down/up/RC/閾值/符號/貢獻量）。這些皆為 AXI-Lite cfg_hub 之後可寫 RAM，而 cfg_hub 本體僅佔 19 LUT（表 5.24）。新增任務因此並非硬體變更，而僅是離線產生並於切換時串流之另一份離線組態（儲存 blob:25,088 bytes $\approx$ 25 KB；匯流排 payload: $6,757 \times 4 = 27,028$ bytes——差異來自 byte-packed 分類器權重等窄於 32-bit 之欄位寫入時擴為完整字）。故晶片成本對任務數為 $O(1)$ （僅主機端組態儲存線性成長），前提為新任務共用凍結主幹、採相同 $32 \times 32 \times 3$ 輸入格式、為分類任務，且具相同 10 類輸出 head 介面。此處展示之五個資料集為此架構性質之一個實證，而非其上限。

板上準確率。 表 5.23 報告五個資料集於單一 compactness bitstream 上之板上準確率，各以 10,000 張測試影像量測（STL10 為其完整 8,000 張測試集；皆為 10 次重複之平均），並與在相同影像、相同 /255 NCHW 前處理下評估之軟體參考比較。五者皆與軟體相差 0.69 個百分點以內（此差距與定點及閾值量化效應一致），且無需逐資料集重新 build；量測間變異甚小（每資料集標準差 ≤ 0.09 pp, CIFAR-10 完全可重現）。此軟體參考欄為相同部署 adapter 配置之 PyTorch 軟體準確率（相同 1W1A 配置），於相同 /255 NCHW 前處理與相同凍結主幹下評估，與表 5.10 之 $M=1$ 軟體參考一致。板上殘餘落差

與一般定點量化損失一致, 並非訓練或前處理造成之誤差。

Table 5.23: compactness build: 板上 N 任務切換準確率。

資料集	模式	軟體參考	板上	差距 (pp)
CIFAR-10	純主幹	81.46%	80.99%	-0.47
SVHN	MARS	73.72%	73.21%	-0.51
FashionMNIST	MARS	80.42%	79.98%	-0.44
STL10	MARS	67.46%	66.77%	-0.69
CINIC10	MARS	64.80%	64.64%	-0.16

軟硬體差異從何而來。 此部署涉及兩種應分清楚之軟硬體差異。其一為刻意之設計取舍：MARS 捨棄了 Conv-Adapter [22] 中最不利於硬體之選擇, 產生表 5.7 逐軸摘要之軟硬體準確率差距。其二即上方表 5.23 所報——同一部署配置於 FPGA 實跑與於 PyTorch 評估之殘差量化誤差 (≤ 0.69 pp); 模組級資料路徑為 bit-exact (43,520 向量 100% 通過), 故其殘餘落差與定點及閾值量化 (Q8) 效應之累積一致, 而非 RTL 不一致。

資源與切換成本。 表 5.24 給出 compactness N-task bitstream 於 XC7Z020 之後佈局資源。均勻 PE = 1 折疊維持 39,967 LUT、零 DSP 與 84 BRAM tiles, 並提供單一 active 任務之 cfg 可寫儲存 (五份 host 常駐任務影像輪流載入此儲存); 時序於 100 MHz 達成 (post-route WNS +0.055 ns), 晶片總功耗估計 1.802 W。端到端吞吐量約 107 img/s, 為 compactness (PE = 1) 時間折疊之預期結果, 也是 throughput build (第 5.5 節) 仍為能效比較代表組態之原因。完整每任務重組態寫入 6,757 個配置字, 以準確率已驗證之批次驅動需 1.86 ± 0.04 ms (50 次隨機順序切換之平均 $\pm$ 標準差)。此值非次毫秒: 更快之連續段批次路徑經量測會破壞 adapter-on 準確率 (cfg-write FSM 於超過其單拍接受率時丟字), 故此處報告準確率正確之驅動。即便如此, 約 1.86 ms 不需重組態控制器; 將此 build 嚴格壓至 1 ms 以下屬 RTL 層改動 (cfg_hub burst 或雙 bank 翻轉), 列為未來工作。

Table 5.24: compactness N-task bitstream:XC7Z020 後佈局資源。

資源	使用量
LUT (總計)	39,967
– 作為邏輯	30,696
– 作為記憶體 (LUTRAM)	8,282
FF	40,926
BRAM (tiles)	84
DSP	0
adapter_cfg_hub 單獨	19 LUT
晶片總功耗 (Vivado 估計)	1.802 W
時序 @ 100 MHz	達成 (WNS +0.055 ns)

Vivado 之 LUT 統計分為 as logic、as memory、route-through 並含跨功能合併; 子類別互有重疊, 故不必然加總等於 LUT 總數。

此設計點證明 cfg_hub 切換原理本質上並不限於任意固定任務數: N 之限制因素為主機端組態儲存與共用主幹之前提, 而非晶片資源。

5.7 任務數擴展分析

邊緣 FPGA 上的任務切換問題於文獻中有三類獨立方法, MARS 應對此三類發表系統進行比較。第一類是部分重組態式時間多工: ACNNE [24] 以 PCAP 於執行時期置換 CNN 卷積層 bitstream; Rangsikunpum 等 [25] 以 ZyCAP 控制器切換逐類細粒度分類模型; Boudjadar 等 [26] 將 CNN 自適應與動態 PR 共同設計; Kang & Park [27] 則以遮罩式部分更新結合動態重組態。第二類是固定共享前端搭配可程式後端: FixyNN [28] 硬接線特徵萃取器、僅留分類器可程式, 明確認知主幹共享可避免每任務硬體複製。第三類為晶片內動態模型選擇: NeuroAdaptiveNet [29] 將固定之合成時精簡模型變體集合存於晶片內, 每次推論以 k-Nearest-Centroid 選擇器 (microsecond 級) 挑選變體, 將其參數多工至共用 PE 陣列、無部分重組態 (所報 52.85% LUT 節省為相對於以最大模型為尺寸之靜態設計)。三類各有代價——PR 控制器帶來毫秒到百毫秒切換延遲; FixyNN 於 tape-out 永久固定特徵萃取器; NeuroAdaptiveNet 之晶片內成本隨其固定 (合成時決定) 且全部常駐晶片之變體集合成長。

表 5.25 以量化方式將 MARS 置於上述系統中。

Table 5.25: 任務切換 FPGA 推論:MARS 與已發表方法之比較。

方法	年份	切換延遲	Fabric-PR 控制器	任務儲存成長	適應層級
ACNNE [24]	2024	97–114 ms	是 (PCAP)	每任務 部分 bitstream	層區塊置換
CoarseToFine [25]	2024	5.2 ms	是 (ZyCAP)	每任務 部分 bitstream	模型層級置換
Boudjadar 等 [26]	2025	PR-bound	是 (DPR)	每任務 部分 bitstream	模型層級置換
Kang & Park [27]	2025	PR-bound	是 (DPR)	每任務 部分更新遮罩	部分模型更新
FixyNN [28]	2019	N/A (固定 FE)	否	每任務 後端硬體	固定 FE; 每任務後端
NeuroAdaptiveNet [29]	2025	0.23–0.67 μ s 選擇器/控制延遲	否	全部變體參數常駐晶片 (合成時固定)	模型/變體選擇; 無 FE 層級 adapter 適應
MARS	2026	約 1.86 ms	否	$O(1)$ 晶片內; 每任務約 25 KB 主機端 blob	FE 層級 adapter

由表 5.25 可直接得出三項觀察：第一, 決定性優勢為結構性而非僅速度數字：MARS 以單一 bitstream 在 $O(1)$ 晶片成本下服務任意數量之合格任務——每多一任務僅是一份約 25 KB 之離線組態——而 PR 類需每任務產生、儲存並串流層級或模型級之部分 bitstream; FixyNN 式共享 FE 需為每任務重新實例化整個後端硬體; NeuroAdaptiveNet 則將固定之合成時模型變體集合全部常駐晶片、以選擇器每輸入選一個, 晶片成本隨集合成長且集合於合成時固定。第二, 組態重寫時間約 1.86 ms(切換後正確性另行驗證), 且完全不需 FPGA 重組態控制器。第三, MARS 對目標 FPGA 的需求為單一 bitstream 加上存於 DRAM 的每任務參數 blob; PR 類另需重組態控制器與部分 bitstream 倉庫——此為邊緣部署的重要簡化。

表 5.25 中有兩個結構性面向值得獨立列出, 因其為此優勢之根源: bitstream 儲存需求 (表 5.26) 與重組態控制器需求 (表 5.27)。

Table 5.26: T 個執行時期任務之 bitstream 儲存需求。

做法	年份	所需 bitstream	$T=5$ 範例
PCAP-based PR [24]	2024	1 base + T partial	6
ZyCAP-based PR [25]	2024	1 base + T partial	6
Masking-based PR [27]	2025	1 base + T partial	6
晶片內模型選擇 [29]	2025	1 份 bitstream (K 個變體全存晶片內)	1
MARS	2026	1 份 bitstream	1

對晶片內模型選擇 [29], K 為合成時固定之變體集合, 並非可於執行時期擴充之任務數; 新增集合外之任務需重新合成設計, 且晶片內參數記憶體隨 K 成長。

Table 5.27: 重組態控制器需求 vs. MARS。

做法	年份	重組態控制器
ACNNE [24]	2024	PCAP (須)
CoarseToFine [25]	2024	ZyCAP (須)
Boudjadar et al. [26]	2025	Dynamic PR 控制器 (須)
Kang & Park [27]	2025	Dynamic PR 控制器 (須)
NeuroAdaptiveNet [29]	2025	無 (晶片內參數多工; 變體集合於合成時固定)
MARS	2026	無 fabric 重組態控制器; 僅 AXI-Lite cfg 寫入

於更大型 FPGA 上硬體驗證 $M \geq 2$ 多分支配置為未來工作。表 5.25 之切換結果為真正、已上板驗證之 compactness build (第 5.6.3 節) 所量: 於單一 bitstream 上 runtime 切換任務模型, 組態重寫約 1.86 ms (切換後正確性另行驗證), 並於架構上可在 $O(1)$ 晶片成本下擴展至任意數量之同主幹、同輸入尺寸、同輸出 head 分類任務。此 build 嚴格次毫秒切換屬 RTL-bound, 列為未來工作 (更快之批次路徑經實測會破壞 adapter-on 準確率)。本節比較因此僅限於實際於 PYNQ-Z2 上建置並量測之兩個設計點。

1-bit Adapter 之硬體驗證狀態。 針對已發表系統之量化對比——切換延遲 (表 5.25)、bitstream 數量 (表 5.26)、是否需重組態控制器 (表 5.27)——已彙整於前述各表。其未涵蓋之唯一維度——1-bit Adapter 是否確實經硬體驗證——則隔離於表 5.28。MARS 不具優勢之維度 (例: 可訓練參數比例, 3.78% 與 Houlsby 的約 3% 同量

級、高於 LoRA-Edge 1.49%) 不主張為優勢。

Table 5.28: 引用範圍內 1-bit Adapter 之硬體驗證狀態。

文獻	年份	Adapter 位元	主幹位元	是否硬體驗證?
Jie 等 [23]	2023	1-bit	FP32	否
Conv-Adapter [22]	2024	FP32	FP32	否
MARS	2026	1-bit	1-bit (1W1A)	是 (M=1)

「硬體驗證」指 PYNQ-Z2 上部署之 $M=1$ adapter 組態; $M \geq 2$ 多分支 adapter 經 RTL 模擬與合成後評估, 未於 XC7Z020 上板驗證。

5.8 證據層級總結

在進入限制之前, 表 5.29 彙整各驗證層級所確立之內容。簡言之: 準確率機制 (RC 與多分支擴展) 於軟體層以二十組 backbone-target 配對確立; $M \geq 2$ 配置另於 RTL 模擬 (位元精確) 與合成後 (資源/功耗) 層級驗證, 但超出 XC7Z020 容量, 故未上板驗證; 部署之 $M=1$ compactness build 完整上板驗證, 含五組任務模型間之執行時期切換; 跨平台吞吐/能效數據於 throughput build 以對齊之 batch-1000 協定端到端實測, 其中 FPGA 功耗為 Vivado 估計值。

Table 5.29: 主張—證據總結: 各主張之證據層級、工作負載與限制。

主張	證據層級	資料集／工作負載	限制
RC 貢獻 (寬版 +16.2 pp、部署 +23.0 pp, $M=1$)	軟體 bias 開/關消融	CIFAR- 10→SVHN, 1W1A; 其餘 目標 +0.6–7.4 pp	單一 seed; 主張限於 1-bit
多分支增益 (平均 +3.36 pp, 最高 +11.33 pp)	軟體單一 seed; 兩組 配對 $n=5$ paired t 檢 定	20 組配對, 部署幾何	$M \geq 2$ 未上板驗證
$M \geq 2$ 硬體成本	RTL 模擬 (位元精 確, 40/40)+ 合成後	$M=2-4$, 兩種 build	超出 XC7Z020; 無佈局 繞線
$M=1$ 板上正確性	板上, 完整測試集 $\times$ 10 次 (10,000 張; STL10 為 8,000)	五資料集, compactness build	與軟體差 0.69 pp 內 (定 點/閾值量化)
執行時期任務切換	板上, 50 次隨機切換	五組任務模型, 單一 bitstream, 1.86 ± 0.04 ms	非次毫秒; 限同主幹 $32 \times 32 \times 3$ 分類
吞吐量／能效	端到端實測, batch 1000	1W1A SVHN: MARS (FPGA) 對 Conv- Adapter (GPU/Jetson)	FPGA 功耗為 Vivado 估 計; GPU/Jetson 為實測

5.9 限制

MARS 有若干需要承認的限制; 凡相關之處, 皆於各項中註明具體之下一步或已發表之補足基線。

- **單一主幹架構。**所有實驗使用 FINN 參考之 VGG-like 主幹 (6 卷積 + 3 FC)。推廣至更深或具分支拓樸 (如 ResNet、MobileNet、EfficientNet) 需重新推導 Super Wrapper 之 signal 切分、FIFO 深度排程與 cfg_hub 位址映射; 觀念上同三項技術 (多分支 Adapter 擴展、RC、cfg_hub) 皆適用, 但本論文之實證證據未推及 VGG-like 之外。
- **任務類型限於影像分類。**已驗證之機制涵蓋同一輸入格式 ($32 \times 32 \times 3$) 之分類任務切換。偵測、分割或非影像 modality 需不同 output head 與 cfg_hub 位址映射形貌; cfg_hub 介面可推廣, 但已展示之任務不行。

- **資料集規模**。軟體端目標資料集為 SVHN、FashionMNIST、STL10、CINIC10——皆 32×32 級。準確率主張不擴及 ImageNet 規模, 該規模下 1W1A 主幹準確率本身大幅較低 [6], 且多分支 Adapter 擴展與 RC 之角色尚未於該規模特徵化。於 ImageNet 等級 workload 上重新評估 MARS 列為未來工作。
- **量化空間**。bit 寬度掃描涵蓋 $\{1, 2, 4, 8\}$ -bit 權重/啟動值組合; INT3、所報告點之間之中間位寬、ternary ($\{-1, 0, +1\}$)、逐層混合精度等點不在內; RC 之「1-bit 專屬」陳述因此刻劃 (1-bit, ≥ 2 -bit) 二分之 step, 而非 INT3/INT4 邊界效應之細緻陳述。
- **統計樣本量**。multi-seed 變異檢查 (表 5.5 與 $M=1$ 對 $M=4$ 檢定, 第 5.3.6 節) 使用 $n=5$ seeds ($\{2024-2028\}$); 涵蓋完整每 (M, RC) cell 矩陣之 paired t -test 因預算受限未執行。Table 5.2 之軟體端結果為單一 seed 並如實標示。
- **Checkpoint 以 test 集選點**。依 BNN-PYNQ 原始 trainer 流程, 每輪訓練後以 test 集評估並保留最高分之 epoch(best.tar); 故所有絕對準確率為 200 epoch 中之最佳操作點、具樂觀偏差; 因所有配置 ($M=1-4$ 、有/無 RC、全部主幹與目標) 採同一選點程序, 比較在程序上一致; 惟以 test 集選最佳 epoch 仍可能同時使絕對準確率與相對差值產生偏差; 無選擇之 last-epoch 交叉檢核於兩個多 seed 遷移上仍重現配對多分支增益 (第 5.3.6 節)。validation 選點協定留待未來實驗。
- **切換延遲尚未嚴格次毫秒**。真正之 N 任務 compactness build 達成 1.86 ms 之組態重寫切換 (切換後正確性另行驗證); 一更快之連續切片批次驅動曾實作並量測, 但於 adapter-on 模式破壞準確率——因 cfg-write FSM 於超出單拍接受速率時會丟字。達成嚴格次毫秒 (與每推論粒度切換) 需 RTL 改動——cfg_hub burst 接收或 dual-bank ping-pong——而非純軟體最佳化。
- **$M \geq 2$ 尚未上板驗證**。多分支 Adapter 擴展曲線 ($M \in \{2, 3, 4\}$) 已於軟體建立, 且於硬體層面達到 RTL 模擬與合成後階段 (第 5.4.4 節): 每個多分支變體皆通過逐 MVAU RTL 模擬之位元級一致驗證, 其合成階段之資源使用量由 Vivado 報告、功耗由 vectorless 合成後分析估計。惟擺放與繞線受 XC7Z020 slice 容量所限 ($M=1$ 時已達 97.4% 佔用), 故多分支準確率之上板量測需更大型 FPGA (如 Zynq UltraScale+ ZCU104、Alveo 級)。
- **RC ablation 範圍**。第 5.3.2 節之 RC ablation 僅於 1W1A 並涵蓋兩種 adapter 幾何 (寬版與 deployed) 執行; 對四個 FPGA 友善設計軸 (kernel、activation、 α 參數

化、mid-channel 公式) 以 bias on/off 進行完整 factorial 掃描將進一步銳化逐軸貢獻歸因, 但本論文未執行。

- **無與商用 NPU 之逐項比較。**平台比較涵蓋 GPU (RTX 4090)、嵌入式 SoC (Jetson Orin NX) 與 PYNQ-Z2 上之 MARS;Hailo-8、Arm Ethos-U55/U85 等商用邊緣 NPU 未量測。嚴格受控之比較需於各平台部署相同之 1W1A 主幹, 而其各自之工具鏈原生並不支援, 列為未來工作。
- **至大型 FPGA 之可擴展性。**評估限於 XC7Z020。於 Zynq UltraScale+ 或 Alveo 等平台之行爲——尤其上述 `cfg_hub burst` RTL 改動是否仍必要, 或 AXI-Lite 單拍是否已飽和 slack——尚待描述。

Chapter 6

結論與未來工作

貢獻總結。 本論文提出 MARS, 一種將參數高效率遷移學習整合進 FINN 串流資料流推論管線之超低位元 AI 硬體加速器架構, 並進一步延伸為具 runtime 任務切換能力之系統。關鍵貢獻與發現如下：

1. **將參數高效 adapter 整合進超低位元串流資料流加速器。** 本論文將 adapter 設計為正交的加性側分支, 相加進凍結之超低位元主幹, 在不重新編譯或重新合成電路、亦不更動主幹權重的情況下加上任務適應能力——此為本論文所綜述之串流資料流加速器與軟體端 adapter 中前人未曾展示之結合。Adapter IP 不需 DSP slice 或專用乘法器——新增之算術與儲存全數以 LUT、正反器與分散式 RAM 實作——且於模組層級驗證為 bit-exact (第 5.4 節)。
2. **針對超低位元情境之多分支 adapter 擴展。** 將單一瓶頸 adapter 拓寬為 M 條平行分支——各分支獨立完成 down 投影、二值啓動與 up 投影, 其縮放後貢獻於共用閾值級前相加——提升遷移準確率, 與補回二值化所削弱之有效容量一致, 使其高於單分支 Conv-Adapter 基線 [22], 同時維持小的每任務參數負擔 (表 5.11 與表 5.18)。於第 5.4.4 節之二十組 backbone-target 全掃描中, 平均多分支增益為 +3.36 pp (最高 +11.33 pp, FashionMNIST→SVHN; 皆為單一 seed、以 test 集選 best-epoch 之操作點軟體實驗, $M \geq 2$ 之硬體部署因 XC7Z020 容量限制僅驗證至 RTL 模擬與合成後層級)。
3. **將任務身分與 bitstream 解耦, 達成單一 bitstream 執行時期任務切換。** 共用之超低位元主幹一次性凍結入電路, 所有任務相依參數搬移至執行時期可寫之晶片內組態記憶體 (cfg_hub); 故切換任務僅為單一不變 bitstream 上之參數記憶體改寫, 無 fabric 重組態、無重組態控制器, 單一 bitstream 即可在 $O(1)$ 晶片成本

下服務任意數量之合格同主幹、同輸入尺寸、同輸出 head 分類任務, 主機端組態儲存則隨任務數 $O(T)$ 成長。此架構性質以五組任務模型於 compactness build (CIFAR-10、SVHN、FashionMNIST、STL10、CINIC10) 上板展示, 任務切換顯著快於 PR 類重組態且不需重組態控制器 (第 5.6.3 節、表 5.25)。

除上述架構性貢獻外, 第 4 章與第 5.5 節顯示部署設計運行於邊緣功耗封套內, 且於所量測之 1W1A 工作負載上相較 GPU 與嵌入式 GPU 基線具競爭力之能效, 並完整六階段設計流程經三層級驗證 (per-MVAU 向量、頂層串流模擬、與板上執行)。此等為支持上述貢獻之佐證, 而非另立之貢獻。

未來工作。 以下方向值得進一步探索：

1. **Bare-metal / AXI4-burst 切換驅動：**目前約 1.86 ms 之組態重寫切換時間主要由單拍 AXI-Lite cfg 寫入路徑主導。於真正之 compactness N-task build (第 5.6.3 節) 目前切換時間之剩餘瓶頸在 cfg-write FSM 之單拍接受率、而非軟體：曾實作並量測更快之連續段批次驅動, 但發現其會破壞 adapter-on 準確率, 因 cfg-write FSM 於超過其單拍接受率時丟字。於該 build 嚴格達成次毫秒切換因此需 RTL 改動 (cfg_hub burst 接受或可原子翻轉之雙 bank ping-pong), 此為將 N 任務切換提升至逐推論粒度之具體下一步。
2. **大型 FPGA 上之多分支 Adapter 硬體驗證：**將目前 $M=1$ 的硬體驗證延伸到 $M=2-4$ 配置, 於 slice/LUT 預算充裕的大型 FPGA 平台 (如 Zynq UltraScale+ ZCU104、Alveo U250) 上消除 XC7Z020 的擺放飽和限制, 直接板上量測軟體預測之準確率增益 (SVHN 79.81%、FashionMNIST 83.54%) 而非推估。
3. **更廣的架構支援：**將 Super Wrapper 設計推廣至不同主幹架構 (如 FINN 中的 ResNet 模型)。
4. **ASIC 探索：**研究 Adapter IP 的 ASIC 實作, 不具可重組態約束時可進一步最佳化面積與功耗。
5. **Adapter 感知訓練：**發展針對量化硬體表示明確最佳化之訓練策略, 有望改善 RC 與多分支 Adapter 擴展之互動, 並縮小 0.69 個百分點的軟硬準確率差距。
6. **參數化 IP 庫：**將參數化 Adapter IP 與 cfg_hub 執行時期切換基礎架構作為開源模組庫發佈, 擴展 FINN 加速器。

7. **較大 n 之顯著性檢定**： $n = 5$ paired t -test 已於兩個代表性部署遷移提供多分支增益之初步統計證據（第 5.3.6 節）；將其延伸至完整 ($M=1-4$, RC 開/關) 矩陣、對每個 (M , RC) 格做 paired t -test 以於全面附上正式顯著性與誤差棒，仍為未來工作（第 5.3.3 節）。
8. **完整 factorial 設計軸消融**：對 Adapter 設計軸（kernel 大小、 α 參數化、mid-channel 寬度）做完整 factorial 掃描，以隔離各軸對寬版至 deployed 落差之個別貢獻，補足第 5.3.7 節僅隔離單軸（kernel）之分析。

Bibliography

- [1] L. Gao, Z. Luo, and L. Wang, “Convolutional Neural Network Acceleration Techniques Based on FPGA Platforms: Principles, Methods, and Challenges,” *Information*, vol. 16, no. 10, art. no. 914, Oct. 2025, doi: 10.3390/info16100914.
- [2] P. Antunes and A. Podobas, “FPGA-Based Neural Network Accelerators for Space Applications: A Survey,” arXiv:2504.16173, Apr. 2025.
- [3] A. Nechi et al., “FPGA-Based Deep Learning Inference Accelerators: Where Are We Standing?,” *ACM Trans. Reconfigurable Technol. Syst.*, vol. 16, no. 4, art. no. 60, Oct. 2023, doi: 10.1145/3613963.
- [4] J. Jiang et al., “FPGA-Based Acceleration for Convolutional Neural Networks: A Comprehensive Review,” arXiv:2505.13461, May 2025.
- [5] R. Li, “Dataflow & Tiling Strategies in Edge-AI FPGA Accelerators: A Comprehensive Literature Review,” arXiv:2505.08992, May 2025.
- [6] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, “XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, Oct. 2016, pp. 525–542.
- [7] I. Hubara et al., “Quantized Neural Networks: Training Neural Networks with Low Precision Weights and Activations,” *J. Mach. Learn. Res.*, vol. 18, no. 187, pp. 1–30, Apr. 2018.
- [8] H. Qin et al., “Binary Neural Networks: A Survey,” *Pattern Recognit.*, vol. 105, art. no. 107281, Sep. 2020, doi: 10.1016/j.patcog.2020.107281.
- [9] Y. Umuroglu et al., “FINN: A Framework for Fast, Scalable Binarized Neural Network Inference,” in *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, Feb. 2017, pp. 65–74.

- [10] M. Blott et al., “FINN-R: An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks,” *ACM Trans. Reconfigurable Technol. Syst.*, vol. 11, no. 3, art. no. 16, Sep. 2018, doi: 10.1145/3242897.
- [11] S. A. Alam et al., “On the RTL Implementation of FINN Matrix Vector Unit,” *ACM Trans. Embed. Comput. Syst.*, vol. 22, no. 6, art. no. 94, Nov. 2023, doi: 10.1145/3547141.
- [12] H.-I. Liu et al., “Lightweight Deep Learning for Resource-Constrained Environments: A Survey,” *ACM Comput. Surv.*, vol. 56, no. 10, art. no. 267, Jun. 2024, doi: 10.1145/3657282.
- [13] Y. Su, K. P. Seng, L. M. Ang, and J. Smith, “Binary Neural Networks in FPGAs: Architectures, Tool Flows and Hardware Comparisons,” *Sensors*, vol. 23, no. 22, art. no. 9254, Nov. 2023, doi: 10.3390/s23229254.
- [14] J.-Y. Zhan et al., “FPGA-Based Acceleration for Binary Neural Networks in Edge Computing,” *J. Electron. Sci. Technol.*, vol. 21, no. 2, art. no. 100204, Jun. 2023, doi: 10.1016/j.jnlest.2023.100204.
- [15] A. M. Z. Khaki and A. Choi, “Optimizing Deep Learning Acceleration on FPGA for Real-Time and Resource-Efficient Image Classification,” *Appl. Sci.*, vol. 15, no. 1, art. no. 422, Jan. 2025, doi: 10.3390/app15010422.
- [16] L. Jungemann, B. Wintermann, H. Riebler, and C. Plessl, “FINN-HPC: Closing the Gap for Energy-Efficient Neural Network Inference on FPGAs in HPC,” in *Proceedings of the International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies (HEART)*, May 2025, pp. 103–116.
- [17] B. X. B. Yu et al., “Visual Tuning,” *ACM Comput. Surv.*, vol. 56, no. 12, art. no. 297, Jul. 2024, doi: 10.1145/3657632.
- [18] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2016, pp. 770–778.
- [19] N. Houlsby et al., “Parameter-Efficient Transfer Learning for NLP,” in *Proceedings of the International Conference on Machine Learning (ICML)*, Jun. 2019, pp. 2790–2799.

- [20] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, Apr. 2022.
- [21] R. Belanec, B. Pecher, I. Srba, and M. Bielikova, “PEFT-Bench: A Parameter-Efficient Fine-Tuning Methods Benchmark,” arXiv:2511.21285, Nov. 2025.
- [22] H. Chen et al., “Conv-Adapter: Exploring Parameter Efficient Transfer Learning for ConvNets,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Jun. 2024, pp. 1551–1561.
- [23] S. Jie, H. Wang, and Z.-H. Deng, “Revisiting the Parameter Efficiency of Adapters from the Perspective of Precision Redundancy,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, Oct. 2023, pp. 17217–17226.
- [24] C.-H. Huang, S.-W. Tang, and P.-A. Hsiung, “ACNNE: An Adaptive Convolution Engine for CNNs Acceleration Exploiting Partial Reconfiguration on FPGAs,” in *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, May 2024, pp. 1–5.
- [25] A. Rangsikunpum, S. Amiri, and L. Ost, “A Reconfigurable Coarse-to-Fine Approach for the Execution of CNN Inference Models in Low-Power Edge Devices,” *IET Comput. Digit. Tech.*, vol. 2024, no. 1, art. no. 6214436, Jan. 2024, doi: 10.1049/cdt2/6214436.
- [26] A. Boudjadar, S. Islam, and R. Buyya, “Dynamic FPGA Reconfiguration for Scalable Embedded AI: A Co-Design Methodology for CNN Acceleration,” *Future Gener. Comput. Syst.*, vol. 169, art. no. 107777, Aug. 2025, doi: 10.1016/j.future.2025.107777.
- [27] M. Kang and D. Park, “Runtime-Robust Edge Inference System with Masking-Based Partial Update on Dynamic Reconfigurable FPGA,” *Sensors*, vol. 25, no. 24, art. no. 7448, Dec. 2025, doi: 10.3390/s25247448.
- [28] P. N. Whatmough et al., “FixyNN: Energy-Efficient Real-Time Mobile Computer Vision Hardware Acceleration via Transfer Learning,” in *Proceedings of the Conference on Machine Learning and Systems (MLSys)*, Apr. 2019.
- [29] A. El Bouazzaoui, O. Mouhib, and A. Hadjoudja, “NeuroAdaptiveNet: A Reconfigurable FPGA-Based Neural Network System with Dynamic Model Selection,” *Chips*, vol. 4, no. 2, art. no. 24, May 2025, doi: 10.3390/chips4020024.

- [30] C. Ding et al., “LoRA-C: Parameter-Efficient Fine-Tuning of Robust CNN for IoT Devices,” arXiv:2410.16954, Oct. 2024.
- [31] H. Kwak, K. Lee, J.-J. Lee, and W. Lee, “LoRA-Edge: Tensor-Train-Assisted LoRA for Practical CNN Fine-Tuning on Edge Devices,” arXiv:2511.03765, Nov. 2025.
- [32] B. A. Arend, A. F. Ely, F. L. Kastensmidt, and M. Recamonde-Mendoza, “Predicting Hardware Resource Allocation of MVAU Modules in FINN-Generated FPGA Accelerators,” in *Proceedings of the I Escola Regional de Aprendizado de Máquina e Inteligência Artificial da Região Sul (ERAMIA-RS)*, Nov. 2025, pp. 308–311.
- [33] M. Tasci, A. Istanbulu, V. Tumen, and S. Kosunalp, “FPGA-QNN: Quantized Neural Network Hardware Acceleration on FPGAs,” *Appl. Sci.*, vol. 15, no. 2, art. no. 688, Jan. 2025, doi: 10.3390/app15020688.
- [34] S. Liang et al., “FP-BNN: Binarized Neural Network on FPGA,” *Neurocomputing*, vol. 275, pp. 1072–1086, Jan. 2018, doi: 10.1016/j.neucom.2017.09.046.
- [35] M. Courbariaux, Y. Bengio, and J.-P. David, “BinaryConnect: Training Deep Neural Networks with Binary Weights During Propagations,” in *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, Dec. 2015, pp. 3123–3131.
- [36] Z. Liu et al., “Bi-Real Net: Enhancing the Performance of 1-bit CNNs With Improved Representational Capability and Advanced Training Algorithm,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, Sep. 2018, pp. 747–763.
- [37] Z. Liu, Z. Shen, M. Savvides, and K.-T. Cheng, “ReActNet: Towards Precise Binary Neural Network with Generalized Activation Functions,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, Aug. 2020, pp. 143–159.
- [38] H.-J. Kang and B.-D. Yang, “AoCStream: All-on-Chip CNN Accelerator with Stream-Based Line-Buffer Architecture and Accelerator-Aware Pruning,” *Sensors*, vol. 23, no. 19, art. no. 8104, Sep. 2023, doi: 10.3390/s23198104.
- [39] Z. Zhao et al., “A High-Throughput FPGA Accelerator for Lightweight CNNs With Balanced Dataflow,” arXiv:2407.19449, Jul. 2024.

- [40] L. Zhang et al., “Quantized Convolutional Neural Networks: A Hardware Perspective,” *Front. Electron.*, vol. 6, art. no. 1469802, Jul. 2025, doi: 10.3389/felec.2025.1469802.
- [41] M. Nagel et al., “A White Paper on Neural Network Quantization,” arXiv:2106.08295, Jun. 2021.
- [42] J.-F. Schulte et al., “hls4ml: A Flexible, Open-Source Platform for Deep Learning Acceleration on Reconfigurable Hardware,” *ACM Trans. Reconfigurable Technol. Syst.*, vol. 19, no. 2, art. no. 19, Jun. 2026, doi: 10.1145/3801979.
- [43] T. Lu and C. Chang, “Cache-Adapter: Efficient Video Action Recognition Using Adapter Fine-Tuning and Cache Memorization Technique,” *J. Vis. Commun. Image Represent.*, vol. 111, art. no. 104543, Sep. 2025, doi: 10.1016/j.jvcir.2025.104543.
- [44] F. Manca, F. Ratto, and F. Palumbo, “ONNX-to-Hardware Design Flow for Adaptive Neural-Network Inference on FPGAs,” arXiv:2406.09078, Jun. 2024.
- [45] B. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2018, pp. 2704–2713.
- [46] A. Pappalardo, “Brevitas: A Quantization-Aware Training Library,” 2021. [Online]. Available: <https://github.com/Xilinx/brevitas>